

基于集员几何与在线决策的无线电干扰源快速定位与清除

摘 要

本文针对无线电干扰源的快速自动定位与清除问题，建立“集员几何 + 在线决策”双层框架：下层用示向度扇形求交、无信号排除、近距限定三类算子维护每个频道的候选域，上层以最小化累计虚拟时间为目标、以全频道结案为约束在线选择指令。四问共享底层算子，第三问被实现为第四问的特例，从根上保证四问求解方式一致。

针对问题一，基于集员估计将每条带 $\pm 1^\circ$ 误差的示向度化为两个半平面， n 条读数的 $2n$ 个半平面之交给出凸多边形定位区域，直径由顶点对枚举精确求得。进而由 Jung 定理证明“以直径为直径的圆一般不覆盖该区域”，覆盖比 $\kappa = 2R_{\text{MEC}}/D$ 的紧上界为 $2/\sqrt{3} \approx 1.1547$ ，并以三站互隔 120° 、同偏 0.5° 的正三角形构形构造取等反例 ($D = 45.34$ m、 $R_{\text{MEC}} = 26.18$ m)。

针对问题二，采用下一次测量后的最小外接圆半径为效用，将选点写成带可接收、交会角窗口、行程预算三约束的极小化问题。证明交会角 $\gamma = 90^\circ$ 时定位最优、偏离按 $1/\sin \gamma$ 放大，并导出关键门槛：仅当目标距两检测点均不超过 **810 m** 时，两次正交测量才足以一击清除。

针对问题三，指出“全清”硬约束的实质是为空频道出具判空证书，算出以 1000 m 圆盘覆盖 1800 m 圆域至少需 7 个检测点，据此构建“原点全扫 $\rightarrow$ 七点骨架巡回增量检测 $\rightarrow$ 机会式清除”的分层自适应策略。120 局蒙特卡洛演练全部 $\rho = 1$ ，平均定位清除时间 **521.5 s**、90% 分位 583.3 s，三次正式测试均全清。

针对问题四，定向源使无信号多出“出束”成因，排除域从圆盘退化为圆盘 $\cap$ 半平面，判空证书随之从距离覆盖升级为方向包围（相邻方位角最大间隙 $\leq 180^\circ$ ）。通过保留分层骨架、仅替换排除算子即覆盖混合场景，120 局演练仍全部 $\rho = 1$ ，平均用时 1212.96 s、约为全向场景的 **2.33** 倍。消融与相关长度标定进一步表明：分层选点是效率主源，聚簇因误差场相关长度仅 10-50 m 而暂不启用。

关键字： 集员估计 在线决策 交会定位 判空证书 方向包围

一、问题重述

1.1 问题背景

无线电干扰源的快速侦测与清除是电磁频谱管理中的一类典型任务。在一片半径 1800 m 的圆形区域 Ω 内, 随机分布着 K 个未知的无线电干扰源, K 的取值在 10 到 16 之间但对执行方不可见。每个干扰源占用 1 至 20 号频道中互不相同的一个频道, 位置固定, 其有效接收半径 R_c 落在 $[1000, 1500]$ m 区间内且同样未知; 干扰源分为全向与定向两类, 定向源仅在其定向方向两侧各 90° (合计 180°) 的扇区内可被接收, 定向方向也未知。

一只机器狗携带测向机与激光清除装置, 从区域原点出发 (测向机初始置于 1 号频道), 通过与仿真环境交互来完成任务。它可用的动作只有四条指令: 进入、检测、清除、退出。检测指令在指定频道返回三种结果之一——若收到信号且距离超过 5 m, 返回带 $\pm 1^\circ$ 误差、保留两位小数的示向度; 若距离在 5 m 以内返回“近距”; 否则返回“无信号”。清除指令当且仅当清除点与目标距离不超过 20 m 时成功, 且成败只取决于距离、与信号覆盖角无关。各动作按统一规则计入虚拟时间: 移动按 5 m/s 折算, 频道切换 1 s, 检测 5 s, 清除成功 5 s、未命中 3 s。任务追求在确保所有干扰源被清除的前提下尽量缩短总时间。

1.2 待求解的问题

本文需依次回答以下四个递进的子问题。

问题一 (定位区域的几何度量) 机器狗在若干检测点获得多条带 $\pm 1^\circ$ 误差的示向度后, 真实目标必落在这些示向度所张扇形的公共交集内。要求建立由多条示向度确定目标所在平面区域的模型, 给出计算该区域直径的方法, 并进一步论证: 以“直径为直径的圆”是否总能覆盖整个定位区域; 若不能, 给出反例并刻画覆盖所需圆半径与直径之比的紧上界。

问题二 (第二检测点的选择) 在已获得第一条示向度的基础上, 第二个检测点的位置直接决定了两次测量能把目标锁定到多小。要求建立选择第二检测点的策略模型, 说明什么样的几何配置能让定位区域最小, 并给出满足可接收、几何有效与行程约束的候选点区域。

问题三 (全向源的搜索-定位-清除) 设区域内干扰源全部为全向源, 总数未知。要求设计机器狗自动搜索、定位并清除全部干扰源的完整策略与算法, 通过仿真演练测试统计被清除比例与平均定位清除时间以检验有效性, 并完成三次正式测试、按规定格式报告结果。

问题四 (全向与定向混合场景) 将场景改为全向源与定向源混合, 定向源的个数与各自方向均未知, 其余条件与问题三相同。要求在此更困难的条件下给出检测与清除

的策略及算法，同样通过演练测试检验有效性并完成三次正式测试。

四个问题层层递进：问题一提供刻画定位精度的几何度量工具，问题二给出单步决策的选点准则，问题三、四则把两者装入面向全区域、多目标的在线决策回路，区别仅在于定向源改变了“无信号”这一观测的排除能力。

二、问题分析

2.1 总体分析

本题的三个物理内核决定了求解的走向。其一是**有界误差**：示向度误差被严格限制在 $\pm 1^\circ$ 之内，且是检测点位置的确定性函数（同点重测读数不变），这意味着“真实目标落在示向度扇形内”是一条硬性的集合约束，而非需要用概率去平均的噪声——因此定位问题的自然语言是**集员估计**^[1,2]（把目标锁进一个确定性可行集），而不是最小二乘。其二是**时间与距离同尺度**：机器狗以 5 m/s 匀速移动，故 1 s 恰对应 5 m，检测、切换、清除等一切开销都能折算为等效路程，使“定位精度”与“任务耗时”落在同一把尺子上，从而可以统一优化。其三是**全清是硬约束**：评价要求清除比 $\rho = N_{\text{cleared}}/K = 1$ ，这把任务重心从“快速清掉看得见的源”转移到“为每个频道拿到确定性结论”——有信号的频道要把候选域缩到清除半径内并清除，无信号的频道则必须出具一份“该频道在全区域内确无未清源”的判空证书。

这把“时间与距离同尺度”的换算基准贯穿全文优化，其指令代价与折算关系汇总于表 1。

表 1 指令代价与时间-距离折算

指令 / 动作	耗时构成	虚拟耗时 (s)	等效行程 (m)
进入	进入目标区域，不计	0	0
退出	主动结束测试，不计	0	0
移动	直线距离 d 除以速度 5 m/s	$d/5$	d
频道切换	仅在目标频道与当前不同时计入	1	5
检测动作	三种返回（方向 / 无信号 / 叠加）同价	5	25
清除成功	光学定位 3 + 激光清除 2	5	25
清除未发现	仅光学定位 3	3	15
一次完整检测	移动 + 切换 + 检测动作	$d/5 + 1 + 5$	$d + 30$
一次成功清除	移动 + 清除动作	$d/5 + 5$	$d + 25$

注：等效行程 = 虚拟耗时 $\times$ 5 m/s。由此得两条选点经验：多测一次的代价相当于绕行 30 m，故在 30 m 尺度内的重复检测总是划算；一次「未发现」的试清除只相当于 15 m 行程，比再交会一次（ ≥ 30 m）便宜，这是近距离改用沿射线试清除的折算依据。总虚拟时间 T_{total} 为四类耗时的非加权求和，且被拒指令不推进虚拟时钟。

由表 1，机器狗以 5 m/s 匀速运动，1 s 恰等价于 5 m 路程，于是一次检测相当于绕行 25 m、一次频道切换相当于 5 m、一次未命中的试探清除相当于 15 m。正是这把统一的尺子，使“少走路”与“少测量、少试错”能在同一个目标函数里权衡。

据此，我们把四问统一到一个“集员几何 + 在线决策”的双层框架下求解。下层维护信息状态：为每个频道维护一个候选域，用三类算子随观测逐步收缩它——示向度扇形求交、无信号排除、近距圆盘限定^[9]。上层做在线决策：在当前信息状态下选择下一条指令（去哪里、测哪个频道、还是就地清除），以最小化累计虚拟时间为目标，以“每个频道最终必须结案”为约束。问题一给出下层的度量工具（定位区域及其直径、最小外接圆），问题二给出上层的单步选点准则，问题三、四把两者装进完整决策回路，二者的唯一区别是无信号排除算子的形式。这一承接关系如图 1 所示。



图 1 全文建模路线图与各问承接关系

沿图 1 的主线可以看出四问并非并列，而是“工具—准则—回路—推广”的单链：问题一的直径与最小外接圆是判断“定位是否够准、能否一击清除”的唯一度量；问题二据此回答“下一步该走到哪”；问题三把“测量—逼近—清除”缝成面向多目标的巡回；问题四只替换其中的排除算子便自然涵盖问题三。正因如此，我们在实现上把问题三写成问题四的特例（用“全部为全向”的先验开关关闭方向项），从根上保证两问求解方式一致。

2.2 问题一的分析

问题一的核心是把“多条带误差示向度”翻译成几何对象。每条示向度读数 $\tilde{\theta}_i$ 对应一个以检测点 S_i 为顶点、轴向 $\tilde{\theta}_i$ 、半张角 1° 的无界扇形，而扇形可写成两个半平面之交； n 条读数便给出 $2n$ 个半平面，其交集是一个凸多边形定位区域 Ω_n 。凸集的直径必在顶点对上取到，故直径可由顶点枚举求得。至于“直径圆是否覆盖”，答案是一般不覆盖：由 Jung 定理，平面点集的最小外接圆半径 $R_{\text{MEC}} \leq D/\sqrt{3}$ ，故覆盖比 $\kappa = 2R_{\text{MEC}}/D \leq 2/\sqrt{3} \approx 1.1547$ ，等号在正三角形构形处取到——这正是需要构造的紧界反例。

2.3 问题二的分析

问题二要回答“第二检测点放哪最好”。第一条读数把目标锁进一条极窄扇形，第二点的价值取决于它把扇形切得多小。设目标到两点距离为 r_1, r_2 、两示向度交会角为 γ ，则定位区域直径随 $1/\sin \gamma$ 放大， $\gamma = 90^\circ$ 时最小、偏离直角时急剧恶化。于是选点第一原则是让交会角接近 90° （大致垂直于首条示向度横移），第二原则是在此前提下尽量靠近目标（ D 随距离线性增长）。可行的第二点须同时满足可接收、交会角窗口、行程预算三条约束，其交集即候选区域。

2.4 问题三的分析

问题三把上述工具装进面向全域的在线决策回路。由于每频道至多一源，任务天然分解为 20 个单目标子问题，但它们共享同一只机器狗和同一条时间轴，真正的决策是“在每个落脚点把检测时隙分给哪个频道、下一步走向哪里”，本质是一个带巡回路径的信息采集规划^[10, 12]。“全清”硬约束要求为无信号频道出具“1000 m 排除圆盘覆盖整个 Ω ”的证书，其最小规模经计算为 7 个检测点。代价核算表明“全站点全频道扫描”过于昂贵，须采用分层自适应结构：原点全扫、七点骨架巡回增量检测、机会式清除与补测。

2.5 问题四的分析

问题四只改了一条物理设定，却切断了问题三最关键的推理。定向源使“无信号”多出第三种成因——“检测点落在 180° 覆盖扇区之外”，于是单次无信号的排除域从“圆盘”退化为“圆盘 $\cap$ 半平面”，判空证书必须从“距离覆盖”升级为**方向包围**（候选位置周围检测点的最大方位角间隙不超过 180° ）。这也解释了机器狗为何必须能走出 Ω ：包围边界附近的候选位置时，部分检测点必然落在区域之外。

2.6 模型总体框架

综合上述分析，本文构建的系统总体架构如图 2 所示：底层是与仿真环境交互的指令与计时层，中层是按频道维护的集员信息状态层，顶层是以最小化虚拟时间为目标、以全频道结案为约束的在线决策层。

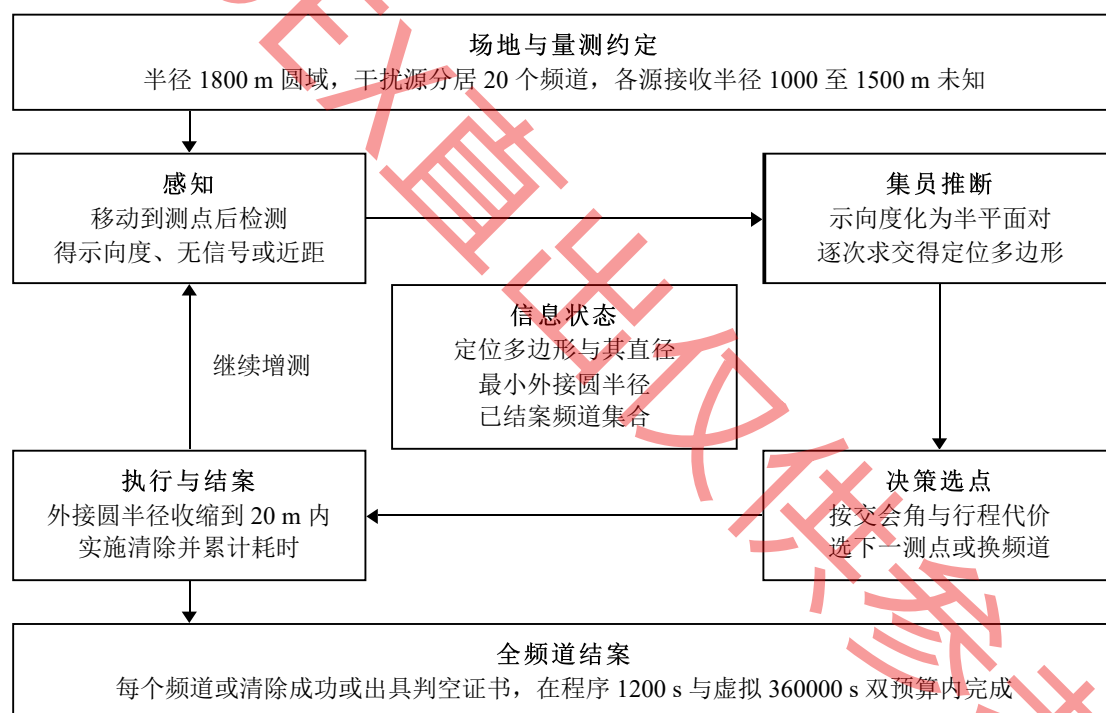


图 2 干扰源自动定位清除系统总体架构

图 2 中三层自下而上贯通：指令层每执行一次检测或清除，其返回值就驱动信息状态层更新对应频道的候选域；决策层再据全部频道的当前候选域，用“单位时间信息增益”排序选出下一条指令回灌指令层。四个子问题在这一架构里只占据不同模块——问题一、二实现信息状态层的几何算子与单步准则，问题三、四实现决策层的巡回与判空逻辑——从而共享同一套底层代码。

三、模型假设

为使模型可解且与题面设定严格一致，本文作出如下假设。每条假设都注明其依据（来自题面给定条件或必要的建模简化）与作用对象，汇总于表 2。

- H1. 测向误差是位置的确定性有界函数。示向度误差 $\varepsilon(S) \in [-1^\circ, +1^\circ]$ 只依赖于检测点位置 S ，同一位置重复检测读数完全一致，不同位置可不同。此假设直接来自题面“误差不超过 $\pm 1^\circ$ ”与“同点重测一致”的规定，是把定位问题建成集员估计（而非统计滤波）的前提。
- H2. 判空一律采用保守排除半径 $R_{\text{ex}} = 1000 \text{ m}$ 。由于各源接收半径 $R_c \in [1000, 1500]$ 未知，判定“某处无源”时只能以下界 1000 m 为准，取更大值会漏清。此为规则性假设，保证 $\rho = 1$ 不被侥幸破坏。
- H3. 每个频道至多一个干扰源。频道号在 1-20 内互不相同，故 20 个频道构成 20 个相互独立的单目标子问题，共享机器狗与时间轴。
- H4. 机器狗可移动至区域外。坐标分量绝对值仅受一个远大于 Ω 的上界约束，允许走出 1800 m 圆域，这对问题四包围边界附近的候选位置是必要的。
- H5. 清除成败仅由距离决定。清除点与目标距离不超过 20 m 即成功，与信号覆盖角无关。故定向性只增加“找”的难度，不改变“清”的难度。
- H6. 各类动作耗时严格按给定规则折算为虚拟时间。移动 $d/5$ 秒、切换 1 s （仅频道改变时）、检测 5 s 、清除成功 5 s /未命中 3 s ；被拒指令不推进虚拟时间。此为计时规则假设，与仿真计时逐行对齐。
- H7. 在误差场空间相关长度未标定前，不启用近距聚簇多测。由于 H1 下同点重测不产生新信息，只有当簇内间距达到相关长度量级时聚簇才有效；在标定完成前默认关闭该手段以免徒增开销。
- H8. 正式测试与本地演练是两类数据，不可混同。正式测试结果只能来自真实运行，演练统计仅用于策略调参与期望估计，不得作为正式测试的硬判据。

表 2 模型假设及其依据

编号	假设	依据	作用对象
H1	测向误差 $\varepsilon(S) \in [-1^\circ, 1^\circ]$ 为位置的确定性函数，同点重测不变	G02、G03（原文）	定位区域按集员而非统计方式构造
H2	判空统一采用保守排除半径 $R_{\text{ex}} = 1000 \text{ m}$	G04 区间下界	判空证书所需检测点数
H3	源位置在 Ω 内均匀随机、频道于 1–20 不重复、 K 在 10–16 均匀	附录 3「每次随机生成一个案例」；均匀性为最大熵默认	仅用于本地演练与期望代价估计，不进入硬判据
H4	机器狗可移动到 Ω 之外，仅受 $ x , y \leq 2 \times 10^6$ 限制	附录 3、附件 2 §1.1	定向源方向包围证书的可行性
H5	忽略高度差，全部几何在同一平面内	附录 2(7)（原文）	所有几何推导
H6	移动为直线、匀速 5 m/s，指令耗时按 G07–G11 累加，无额外开销	附录 2、附件 1 表 1	时间目标函数
H7	误差的空间相关长度待演练标定；未标定前不启用「近距聚簇多测」增益	A2 待定，保守起见	是否采用聚簇降不确定度
H8	一个频道至多一个干扰源，且频道不随时间变化	G13（原文）	问题按频道解耦为 20 个独立子问题

表 2 中，H1、H2、H5 是贯穿四问的“地基性”假设：H1 决定了下层用集合而非概率描述信息状态，H2 决定了判空证书的半径取法，H5 则把定向源的复杂性完全隔离在信息获取环节。其余各条服务于具体环节，在相应章节被反复引用。

四、符号说明

本文正文与附录所用主要符号按“空间与目标量、感知与几何量、决策与代价量”的顺序汇总于表 3，以便随时回查。

表 3 变量与符号说明

符号	含义	类型 / 取值	单位	来源
Ω	目标分布圆域	$\{(x, y) : x^2 + y^2 \leq 1800^2\}$	—	G12
K	干扰源总数	整数， $10 \leq K \leq 16$ ，未知	个	G14
g_k	第 k 个源的位置	$g_k \in \Omega$ ，未知	m	G12
c_k	第 k 个源的频道号	$\{1, \dots, 20\}$ ，各源互异	—	G13
$R_c^{(k)}$	第 k 个源的有效接收半径	$[1000, 1500]$ ，未知	m	G04

续下页

(续表 3)

符号	含义	类型 / 取值	单位	来源
ϕ_k	第 k 个源的定向方向	$[0^\circ, 360^\circ)$ 或空 (全向)	度	G05
S_i	第 i 个检测点坐标	$ x , y \leq 2 \times 10^6$	m	G16, H4
θ_i^{true}	真实方位角	$\text{atan2}(g_y - S_{iy}, g_x - S_{ix}) \bmod 360^\circ$	度	G01
ε_i	测向误差	$\varepsilon(S_i) \in [-1^\circ, 1^\circ]$, 位置的确定性函数	度	G02, G03
$\tilde{\theta}_i$	示向度读数	$\theta_i^{\text{true}} + \varepsilon_i$, 两位小数	度	G01, G02
$W(S_i, \tilde{\theta}_i)$	单条读数的扇形约束	顶点 S_i 、轴向 $\tilde{\theta}_i$ 、半张角 1° 的无界扇形	—	F01
Ω_n	n 条读数的定位区域	$\bigcap_{i=1}^n W(S_i, \tilde{\theta}_i)$, $2n$ 半平面之交	—	F02
$V(\Omega_n)$	定位区域顶点集	有限点集, $ V \leq 2n$	m	F04
D	定位区域直径	$\max_{p, q \in V} \ p - q\ _2$	m	问题 1
R_{MEC}	最小外接圆半径	$\min_c \max_{p \in \Omega_n} \ p - c\ $	m	F05
κ	覆盖比	$2R_{\text{MEC}}/D \in [1, 2/\sqrt{3}]$	—	F05
γ	两条示向度的交会角	$(0^\circ, 180^\circ)$	度	F06
r_i	目标到第 i 个检测点的距离	> 0	m	F03
$w(d)$	距离 d 处的扇形半宽	$d \tan 1^\circ$	m	F03
R_{ex}	判空用保守排除半径	1000 (固定, 取 R_c 下界)	m	H2, A3
H_c	频道 c 的隐藏候选域	Ω 上网格位掩码, 初值全 1	—	§5.4
$N(g)$	在 g 处均无信号的检测点集	点集, 用于方向包围判据	—	F10
p_j	第 j 条指令的位置参数	同 S_i 的取值范围	m	G11
t_{move}	移动耗时	$\ p_j - p_{j-1}\ _2/5$	s	G11
t_{sw}	频道切换耗时	$1 \cdot \mathbf{1}[c_{\text{new}} \neq c_{\text{cur}}]$	s	G10

续下页

(续表 3)

符号	含义	类型 / 取值	单位	来源
t_{meas}	单次检测耗时	5 (三种返回值同价)	s	G09
t_{clr}	单次清除指令耗时	成功 5, 未发现 3	s	G07
T_{total}	定位清除总虚拟时间	$\sum(t_{\text{move}} + t_{\text{sw}} + t_{\text{meas}} + t_{\text{clr}})$	s	G15
N_{cleared}	已清除干扰源个数	整数, $\leq K$	个	G15
ρ	被清除干扰源个数的比例	N_{cleared}/K	—	G15
$\bar{T}$	平均定位清除时间	$T_{\text{total}}/N_{\text{cleared}}$	s	G15
T_{real}	程序运行时间	$\leq \min(1200, \text{窗口剩余})$	s	G16

五、问题一：交会定位区域的几何度量与覆盖判定

5.1 机理起点：从有界误差到集合约束

一次示向度检测给出的不是目标的方位，而是一个**带宽度的方位**：真实方位角落在读数 $\tilde{\theta}_i$ 两侧各 1° 的区间内。由假设 H1，这 $\pm 1^\circ$ 是硬边界而非随机噪声，因此“真实目标在检测点 S_i 看去的方位落在 $[\tilde{\theta}_i - 1^\circ, \tilde{\theta}_i + 1^\circ]$ ”是一条确定成立的集合约束。它在平面上圈定的区域，正是以 S_i 为顶点、轴向 $\tilde{\theta}_i$ 、半张角 $\Delta = 1^\circ$ 的一个无界扇形 $W(S_i, \tilde{\theta}_i)$ 。多次检测得到多个这样的扇形，真实目标必然同时落在它们全体之中——于是定位区域就是这些扇形的公共交集。这一“取交”而非“取平均”的处理，是把仅含方位信息的交会定位^[3,4]建成集员估计的出发点。

5.2 题意映射：扇形化为半平面之交

要计算交集，先把扇形写成更基本的对象。半张角为 Δ 的扇形由两条边界射线围成，每条边界射线对应一个半平面：记 $u_s = (\cos(\tilde{\theta}_i + s\Delta), \sin(\tilde{\theta}_i + s\Delta))$ 为符号 $s \in \{+1, -1\}$ 一侧的边界方向，则该侧半平面可写成

$$n_{i,s}^\top x \leq n_{i,s}^\top S_i, \quad n_{i,s} = -s(u_{s,y}, -u_{s,x}), \quad (1)$$

其法向 $n_{i,s}$ 的符号必须使真实目标位置代入两式均成立——这一符号约定看似平凡，却是实现中极易出错之处（见第十、节所述的一处实测错误与修正）。两个半平面之交即扇形 $W(S_i, \tilde{\theta}_i)$ ； n 条读数给出 $2n$ 个半平面，其交集

$$\Omega_n = \bigcap_{i=1}^n W(S_i, \tilde{\theta}_i) = \bigcap_{i=1}^n \bigcap_{s \in \{\pm 1\}} \{x : n_{i,s}^\top x \leq n_{i,s}^\top S_i\} \quad (2)$$

是至多 $2n$ 条边的凸多边形（半平面之交必为凸集），其交集可由多边形裁剪算法^[5]逐条半平面切得。图3以两个检测点为例展示了这一构造：两个扇形（各含两条边界）相交切出四边形定位区域 $P_1P_2P_3P_4$ ，真实目标 T 落在其中。

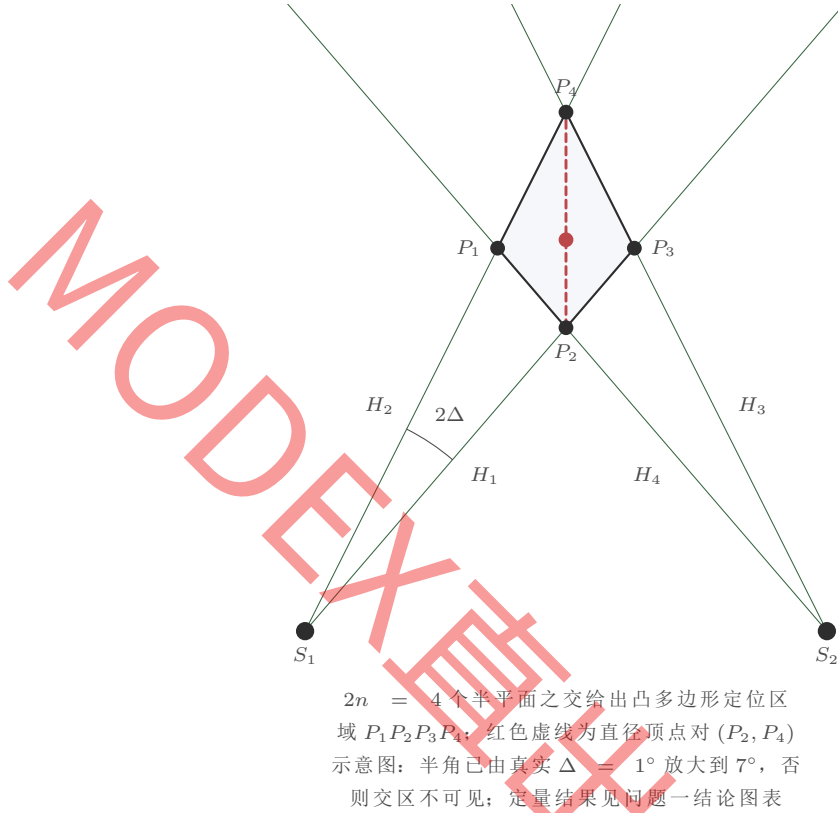


图3 交会定位的半平面交构造与定位多边形

需要说明，图3为使构造关系可见，已把半张角由真实的 1° 放大到 7° ；在真实 $\pm 1^\circ$ 下，60 单位基线所对应的交区只有约 1 单位宽，肉眼不可辨。这也从侧面说明：单靠远距离的两条窄扇形，定位区域已相当细长，其“最坏跨度”需要用直径来精确刻画。

5.3 关键推导：直径与最小外接圆

给定凸多边形 Ω_n 的顶点集 $V = \{v_1, \dots, v_m\}$ ($m \leq 2n$)，其直径定义为区域内相距最远的两点距离。由于凸集上任意两点距离的最大值必在极点（顶点）处取到，故

$$D = \max_{p, q \in \Omega_n} \|p - q\|_2 = \max_{1 \leq i < j \leq m} \|v_i - v_j\|_2, \quad (3)$$

即只需枚举顶点对。本题顶点数 m 典型为 3-6， $O(m^2)$ 枚举可在每收到一条新读数后即时重算。

直径刻画“最坏跨度”，但要判断能否一击清除，真正需要的是能罩住整块区域的最小外接圆半径

$$R_{\text{MEC}} = \min_{c \in \mathbb{R}^2} \max_{p \in \Omega_n} \|p - c\|. \quad (4)$$

最小外接圆或由某两个顶点为直径确定，或由某三个顶点外接确定，故可枚举“两点圆”与“三点外接圆”并取半径最小的可行者^[6-8]，其结果确为最小（缩小任一半径都会漏掉某顶点）。

5.4 完整模型：直径圆覆盖判定与 Jung 紧界

问题一的判定部分问的是：以直径 D 为直径（即半径 $D/2$ ）的圆，是否总能覆盖整个定位区域？等价地，覆盖比 $\kappa = 2R_{\text{MEC}}/D$ 是否恒等于 1？

答案是否。一方面 $R_{\text{MEC}} \geq D/2$ （任何能覆盖区域的圆，其直径不小于区域直径），故 $\kappa \geq 1$ ；另一方面由 **Jung 定理**，平面有界集必可被半径 $R_{\text{MEC}} \leq D/\sqrt{3}$ 的圆覆盖，于是

$$1 \leq \kappa = \frac{2R_{\text{MEC}}}{D} \leq \frac{2}{\sqrt{3}} \approx 1.1547005, \quad (5)$$

上界在正三角形构形处取到（三顶点两两距离均为 D ，外接圆半径恰为 $D/\sqrt{3}$ ）。因此半径 $D/2$ 的圆一般不能覆盖定位区域，最坏时需要把半径放大到 $D/\sqrt{3}$ 。

5.5 真实求解：紧界反例与覆盖比的实测趋势

为坐实上界 (5) 可被取到，我们构造一个显式反例：三个检测点距目标 1500 m、方位互隔 120° ，三条读数同偏 $+0.5^\circ$ 。此时定位区域 Ω_3 恰为一个正三角形，其三顶点两两等距。程序求解给出直径 $D = 45.3444$ m、最小外接圆半径 $R_{\text{MEC}} = 26.1796$ m，覆盖比

$$\kappa = \frac{2R_{\text{MEC}}}{D} = \frac{2 \times 26.1796}{45.3444} = 1.1547005 = \frac{2}{\sqrt{3}}, \quad (6)$$

与 Jung 界在 10^{-6} 相对误差内相等。半径 $D/2 = 22.6722$ m 的直径圆漏掉了全部三个顶点，每个顶点外露 3.5074 m，直观印证“直径圆不覆盖”。该构形如图 4 所示。

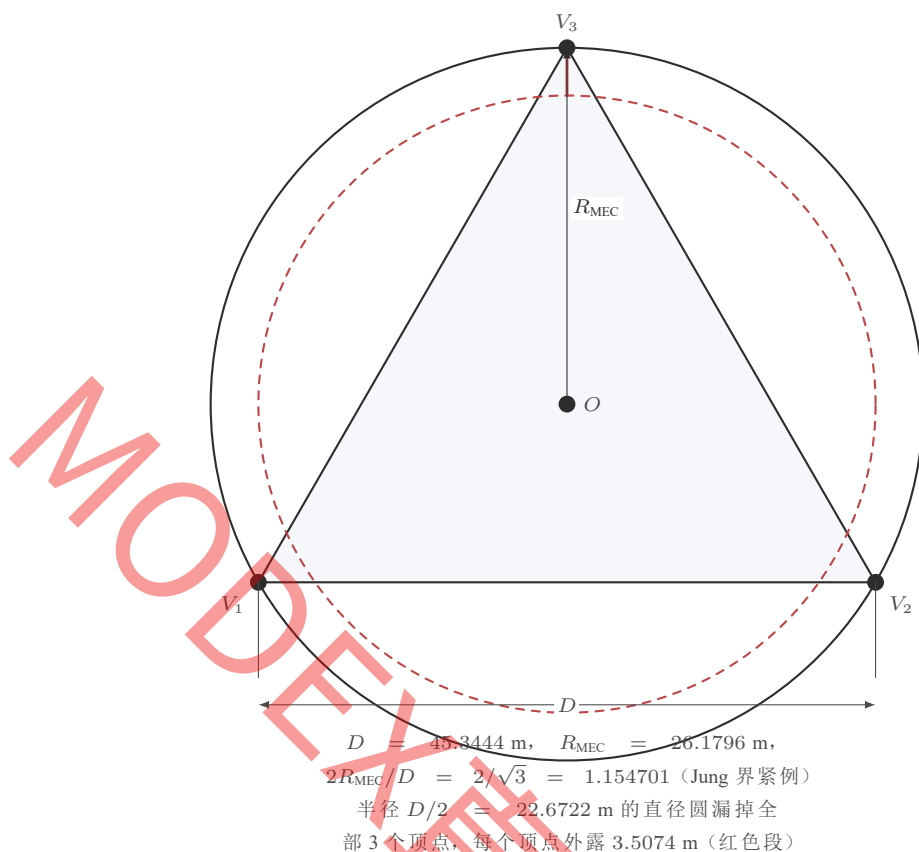


图 4 直径圆覆盖判定的反例构形

图 4 中实线圆为最小外接圆（恰过三顶点），红色虚线圆为半径 $D/2$ 的直径圆，红色实线段标出顶点在直径圆外的外露量。这一反例说明：在用几何度量判断“能否一击清除”时，必须以 R_{MEC} （而非 $D/2$ ）为准；若图省事用 D 代替，则需把安全阈值收紧到 $D \leq 20\sqrt{3} \approx 34.64 \text{ m}$ 才能保证外接圆半径不超过 20 m 。

反例证明了紧界可达，那么在随机构形下覆盖比一般有多大？我们对检测次数 $n = 2, 3, 4, 5$ 各随机采样 4000 组几何（目标在 Ω 内均匀、检测点在 $[1000, 1500] \text{ m}$ 环上随机、读数带 $\pm 1^\circ$ 随机误差），统计每个 n 下覆盖比的最大值，结果如图 5 与图 6 所示。

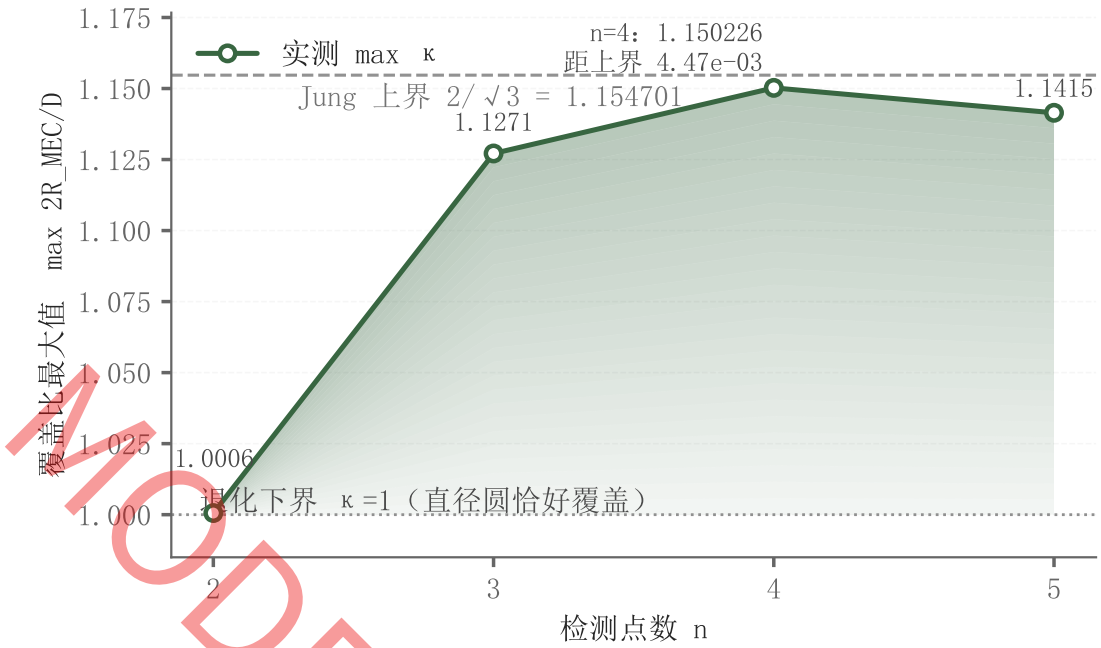


图5 比值 $2R_{MEC}/D$ 随测量次数的变化

随测量次数从2增至5,随机构形下的最大覆盖比依次为1.000571、1.127135、1.150226、1.141453 (图5),可见 $n=2$ 时定位区域近似为平行四边形、覆盖比几乎为1,而 $n \geq 3$ 时区域可呈三角形化并逼近 Jung 界。随机最大值出现在 $n=4$ (1.150226) 而非 $n=3$,说明多条读数的组合更容易“凑”出接近正三角形的极端构形。将随机分布与 Jung 界并置 (图6),全部构形的覆盖比都严格落在 $[1, 2/\sqrt{3}]$ 内,且分布右端紧贴 $2/\sqrt{3}$,从统计上再次确认式 (5) 的紧性。

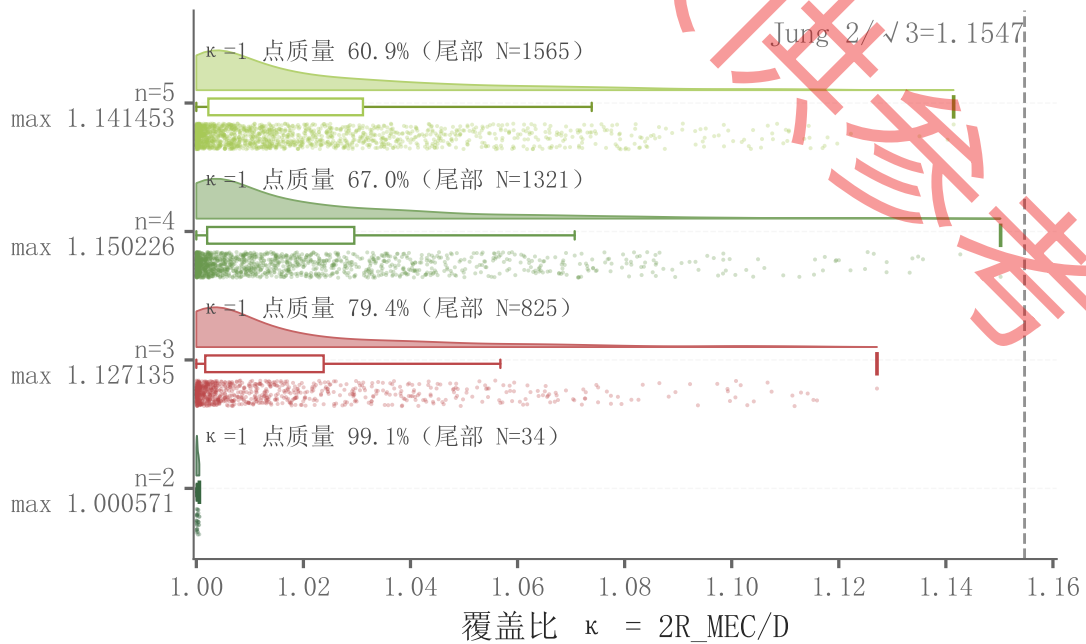


图6 随机构形下比值分布与 Jung 界

上述实测汇总于表 4。

表 4 覆盖比 $\kappa = 2R_{\text{MEC}}/D$ 的实测汇总与 Jung 紧界

读数条数 n	有效样本	κ 最大	κ 均值	κ 中位数	κ 的 90% 分位
2	3953	1.000571	1.000002	1.000000	1.000000
3	4000	1.127135	1.003604	1.000000	1.007844
4	4000	1.150226	1.006791	1.000000	1.023570
5	4000	1.141453	1.008272	1.000000	1.030481
构造反例（三站 $r = 1500\text{ m}$ 、方位互隔 120° 、读数同偏 $+0.5^\circ$ ）					
定位区域直径 D		45.3444 m（ $2n = 6$ 半平面，3 个顶点）			
最小外接圆 R_{MEC}		26.1796 m， $D/2 = 22.6722\text{ m}$ ，外伸 3.5074 m			
覆盖比 κ		1.1547005，与 Jung 界 $2/\sqrt{3} = 1.1547005$ 相对误差 $< 10^{-6}$			
直径圆是否覆盖		否（ $2R_{\text{MEC}} > D$ ）			

注： κ 的中位数在各 n 上均为 1.000000，即随机构形下定位区域绝大多数是「直径圆恰好覆盖」的钝形， κ 显著大于 1 属小概率事件；随机搜索 4000 次的最大值仍差紧界 4×10^{-8} ，等号只能由正三角形构形取到。

5.6 结果解释

表 4 与反例共同给出问题一的完整答案：定位区域是 $2n$ 个半平面所交的凸多边形，其直径由顶点对枚举精确求得；以直径为直径的圆一般不覆盖该区域，覆盖所需圆半径与直径之比的紧上界为 $2/\sqrt{3} \approx 1.1547$ ，由正三角形构形取到。这一结论的工程含义直接服务于后续三问：判断“定位是否已够准到可以一击清除”，必须以最小外接圆半径 $R_{\text{MEC}} \leq 20\text{ m}$ 为停止判据，这也是问题二选点准则与问题三、四巡回逼近的共同目标量。

六、问题二：第二检测点的选择策略与候选区域

6.1 机理起点：交会角决定定位精度

第一条示向度把目标锁进一条极窄的扇形，但沿扇形轴向仍是无界的——单条读数只能定方向、不能定距离，这正是仅含方位量测的固有局限^[3]。第二个检测点的全部价值，就在于它能把这条扇形“横切”成多小的一块。设目标到两检测点的距离分别为 r_1, r_2 ，两条示向度的交会角为 γ 。当两条窄扇形近似为半宽 $w_i = r_i \tan 1^\circ$ 的直条带时，其交集是一个平行四边形，面积与直径都随 $1/\sin \gamma$ 变化： $\gamma = 90^\circ$ 时两条带正交、交集最“紧凑”， γ 偏离直角时交集被拉成狭长条、直径急剧放大。因此选点的**第一原则**是让交会角尽量接近 90° ，即大致沿垂直于首条示向度的方向横移。

6.2 题意映射：把选点写成带约束的极小化

除了交会角，还有两点须一并考虑。其一， D 随距离 r 线性增长，故在满足交会角的前提下应尽量靠近目标（选点第二原则）。其二，移动本身要花 $\|S_2 - S_1\|/5$ 秒，须与定位收益权衡。借助表 1 的时间-距离折算，我们把选点写成如下极小化问题：

$$\min_{S_2} \frac{\|S_2 - S_1\|}{5} + \lambda \cdot \max_{g \in \Omega_1} \hat{R}_{\text{MEC}}(\Omega_1 \cap W(S_2, \theta(S_2, g))), \quad (7)$$

其中内层的 $\max_{g \in \Omega_1}$ 体现最坏情形思想：真实目标 g 只知落在首条扇形 Ω_1 内，故要保证“无论 g 在 Ω_1 何处，新测量都把外接圆压得足够小”。系数 λ （量纲 s/m）把“少走路”与“定位更准”换算到同一尺度，其取值由演练标定。

6.3 关键推导：三重可行性约束

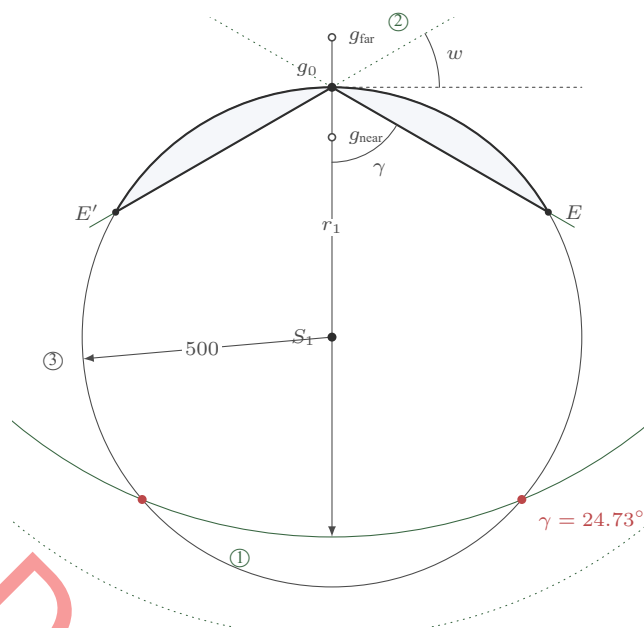
式 (7) 的可行域由三条约束求交而成，缺一不可。

约束一（可接收）。第二点必须能收到该源信号。源位置只知落在窄扇形 Ω_1 内、接收半径只有 $R_c \geq 1000$ m 的保证，故须保证“无论目标在 Ω_1 何处都能收到”，即 S_2 落在以 Ω_1 内各点为心、半径 $R_{\text{ex}} = 1000$ m 圆盘的交集中；因扇形极窄，该交集近似为沿首条示向度线段两端画圆所得的透镜形，并被 Ω 边界截断。

约束二（交会角窗口）。要求 $\gamma \in [\gamma_{\min}, 180^\circ - \gamma_{\min}]$ 。取 $\gamma_{\min} = 60^\circ$ 可把 $1/\sin \gamma$ 的放大控制在 $1/\sin 60^\circ \approx 1.155$ 倍以内，对应候选区域是一对以首条示向度为轴的对称扇环。

约束三（行程预算）。移动开销受限： $\|S_2 - S_1\| \leq B$ ，本文名义场景取 $B = 500$ m（等价 100 s）。

三条约束求交即候选区域，如图 7 所示。



- ① 可接收: S_2 到名义段两端均 $\leq r_1 = 1000$ m; ② 交会角窗 $\gamma \in [60^\circ, 120^\circ]$ (中分 90° 、半宽 $w = 30^\circ$); ③ 行程预算 $|S_2 - S_1| \leq 500$ m
- ① 只有 g_{far} 起作用, 且与 ③ 的交点处 γ 仅 24.73° 、已被 ② 排除; 预算盘内 γ 的上确界仅 90° (实测 85.91°)
- 故 ① 与上限 $\gamma \leq 120^\circ$ 全程松弛, 可行域由 $\gamma \geq 60^\circ$ 与 ③ 共同决定, 为两块 60° 圆弓形 (弦 g_0E 、 g_0E'): 解析面积 45293.04 m^2 , 5 m 网格计得 1814 点 / 45350 m^2

图 7 第二检测点候选区域的三约束求交

图 7 揭示了一个重要事实: 在名义场景 (目标名义位置距原点 500 m 、行程预算 500 m) 下, 约束一与 “ $\gamma \leq 120^\circ$ ” 其实全程松弛——预算圆盘内交会角的上确界只有 90° (网格实测最大 85.91°), 够不到 120° ; 而可接收圆盘中只有远端一支与预算圆相交, 交点处交会角仅 24.73° 、早被约束二排除。因此真正起作用的是 “ $\gamma \geq 60^\circ$ ” 与行程预算, 可行域收缩为两块 60° 的圆弓形, 解析面积 45293.04 m^2 , 与 5 m 网格计得的 1814 个可行点、 45350 m^2 在离散误差内一致。这说明在近距场景下, 选点的主导矛盾是 “交会角够不够大” 而非 “收不收得到”。

6.4 完整模型: 交会角惩罚的量化

固定 $r_1 = r_2 = r$ 、扫描交会角 γ , 可定量刻画偏离直角的惩罚。图 8 给出定位区域直径随交会角与距离的等高线。

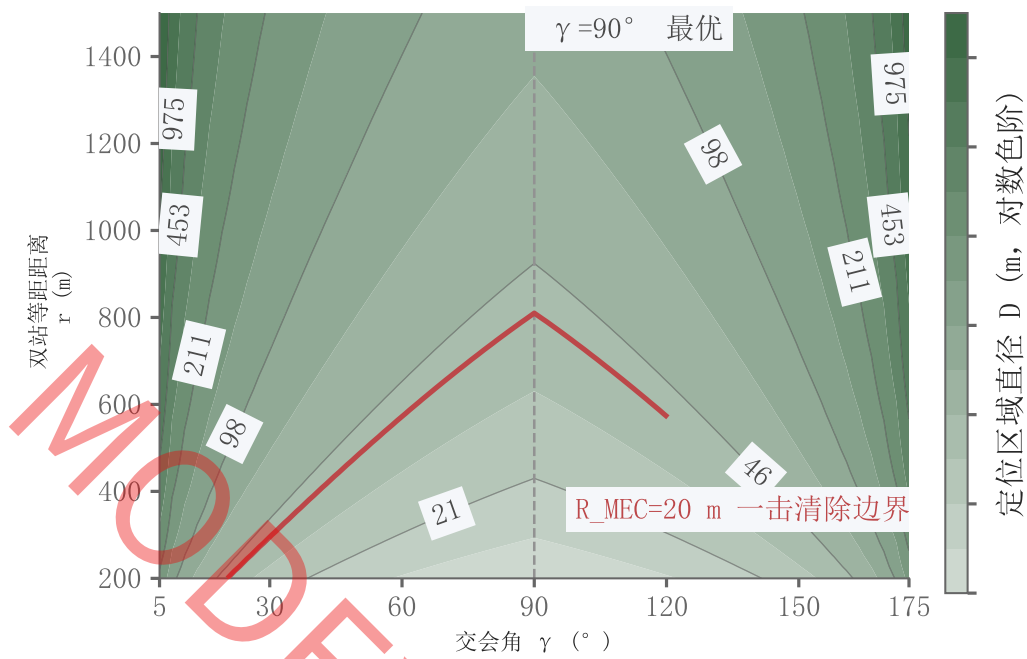


图 8 定位区域直径随交会角与距离的等高线

观察图 8 的等高线走向，直径在 $\gamma = 90^\circ$ 附近取谷值，并沿两侧随 $1/\sin \gamma$ 抬升，且距离越大抬升越陡——这与推导完全一致。把式 (7) 的效用场画在候选区域上（图 9），最优第二检测点落在“交会角接近 90° 且尽量靠近目标”的位置，恰好位于两块圆弓形贴近目标一侧的边缘。

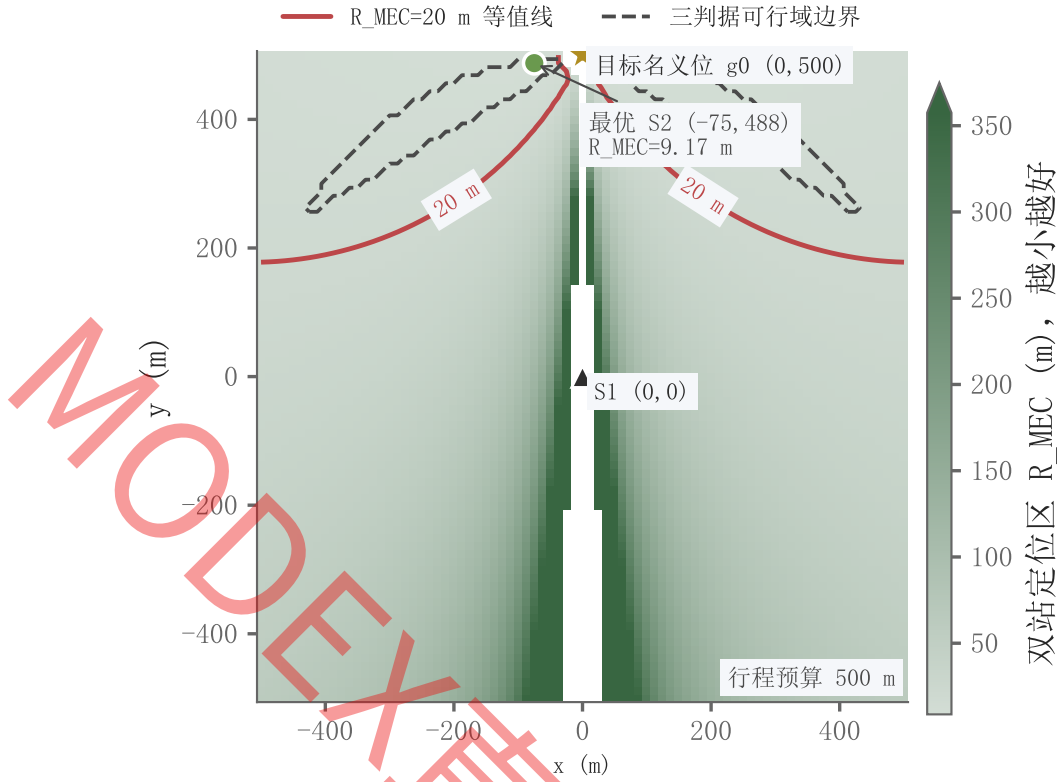


图9 候选区域上的效用场与最优第二检测点

6.5 真实求解：两次检测一击清除的距离上限

把最优几何代入可得一个直接服务于后续问题的定量结论。在 $\gamma = 90^\circ$ 、 $r_1 = r_2 = r$ 的最优配置下，定位区域近似为边长 $2r \tan 1^\circ$ 的正方形，其最小外接圆半径

$$R_{\text{MEC}} = \frac{D}{2} = \sqrt{2} r \tan 1^\circ. \quad (8)$$

令 $R_{\text{MEC}} \leq 20 \text{ m}$ 解得

$$r \leq \frac{20}{\sqrt{2} \tan 1^\circ} \approx 810.2 \text{ m}. \quad (9)$$

也就是说，只有当目标距两检测点都不超过约 810 m 时，两次正交测量才足以保证一击清除；更远的目标必须增加第三次测量或先行靠近。这一上限如图 10 所示。

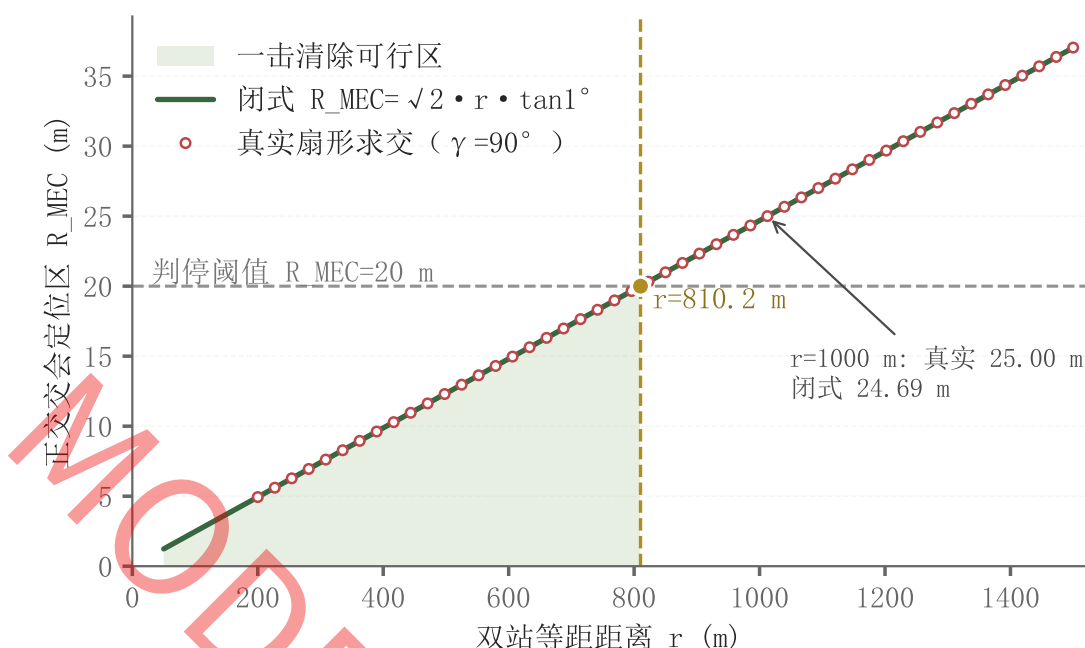


图 10 两次检测一击清除的距离上限

6.6 结果解释

综合图 8–10，问题二给出的选点策略可凝练为两条准则加一个门槛：让交会角接近 90° 、在此前提下尽量靠近目标，且只有目标在约 810 m 以内时两测才够一击清除。这个 810 m 门槛（图 10）为问题三的“测量—逼近”节奏提供了直接依据：对更远目标，与其原地反复测量，不如先沿示向度方向逼近到门槛之内再交会，这正是下一章分层策略中“机会式清除与自适应补测”的几何根源。

七、问题三：全向源的搜索-定位-清除策略

7.1 机理起点：从“清得快”到“判得空”

由于每个频道至多一个干扰源（假设 H3），全局任务天然分解为 20 个相互独立的单目标子问题；但它们共享同一只机器狗和同一条时间轴，因此真正的决策变量不是“怎么定位一个源”，而是“在每个落脚点上，把 5 s 的检测时隙分配给哪个频道、下一步走向哪里”。更关键的是，“确保全清”（ $\rho = 1$ ）这条硬约束把问题的重心从“快速清掉看得见的源”转移到“为每个频道拿到确定性结论”：对有信号的频道，结论是“候选域已缩到 $R_{\text{MEC}} \leq 20 \text{ m}$ 并成功清除”；对无信号的频道，结论必须是一份判空证书——证明该频道在整个 Ω 内确无未清源，否则无法排除“源恰在某个未探测角落、且接收半径只有 1000 m”的可能。

7.2 题意映射：按频道维护的集员信息状态

我们为每个频道 c 维护一个候选域 H_c ，用三类算子随观测收缩它：示向度求交（收到示向度返回时，用式 (2) 把新扇形交入 H_c ）、圆盘排除（收到无信号时，从 H_c 中挖去以检测点为心、半径 $R_{\text{ex}} = 1000 \text{ m}$ 的圆盘——因为若该频道有源且落在此盘内，必被收到）、近距限定（收到近距返回时，目标必在 5 m 内，立即转入清除）。频道结案的两种方式对应两类算子的终点： $R_{\text{MEC}}(H_c) \leq 20 \text{ m}$ 后清除成功，或圆盘排除累积到覆盖整个 Ω （判空）。

7.3 关键推导：判空证书的最小规模

判空的核心是一个覆盖问题^[10, 11]：要用若干半径 1000 m 的圆盘盖住半径 1800 m 的圆域 Ω ，至少需要几个圆心（检测点）？记归一化覆盖半径为“最坏格点到最近检测点距离 / 1800 ”，则判空要求它不超过 $1000/1800 \approx 0.5556$ 。对不同站点数做最优布点求解，结果列于表 5，并绘于图 11。

表 5 覆盖半径与最少判空站点数对照

站点数 n	归一覆盖半径	覆盖半径 (m)	是否满足 $\leq 1000 \text{ m}$	Kershner 最优	与最优之差
4	0.707107	1272.79	否	0.707107	3.2×10^{-7}
5	0.609441	1096.99	否	0.609383	5.8×10^{-5}
6	0.557135	1002.84	否	0.555905	1.2×10^{-3}
7	0.500106	900.19	是	0.500000	1.1×10^{-4}
8	0.445416	801.75	是	0.437340	8.1×10^{-3}
要求 ($R_{\text{ex}}/1800$)		0.555556，即覆盖半径须 $\leq 1000 \text{ m}$			
实际采用骨架		1 + 6 六边形、环半径 $\rho = 1250 \text{ m}$ ，最坏距离 951.52 m			

注： $n = 6$ 的自由最优覆盖半径为 1002.84 m ，仅超出阈值 2.84 m 便无法出证书，故最少站点数为 7。采用的 7 站骨架未取覆盖最优的 $\rho = 1550 \text{ m}$ ，而取 $\rho = 1250 \text{ m}$ ，以在满足 $951.52 \leq 1000 \text{ m}$ 的同时压缩巡站行程。

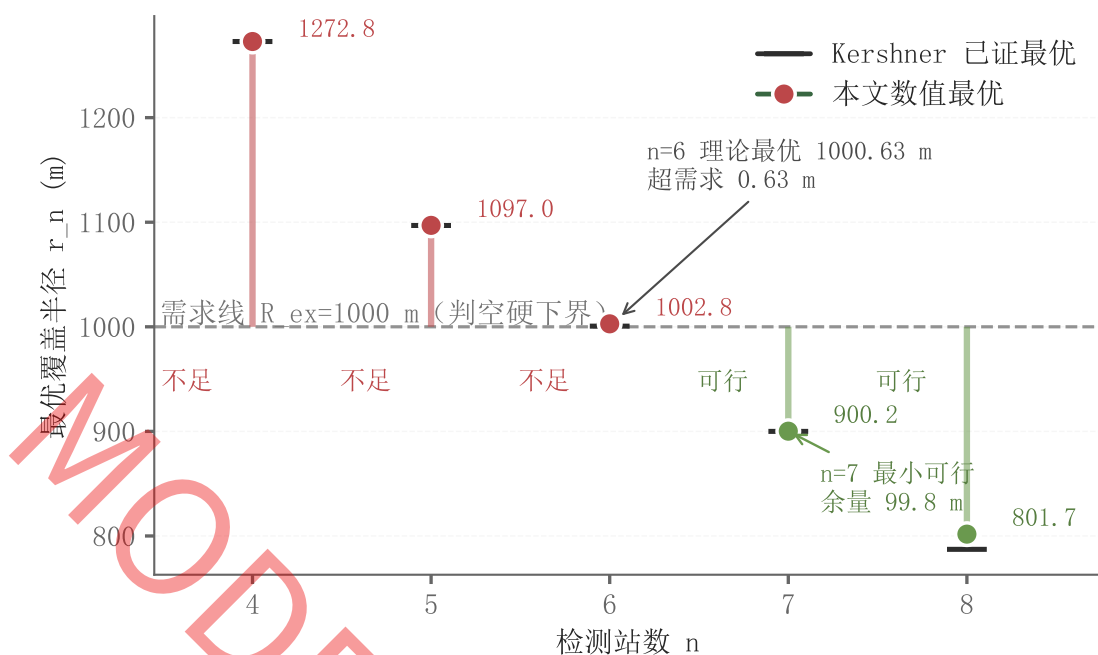


图 11 覆盖半径与最少判空站点数

表 5 给出一个略反直觉的临界现象：6 个站点的最优覆盖最坏距离为 1002.84 m，仅超阈值 2.84 m 便告失败；必须增至 7 个站点（最坏 900.19 m）才可行。因此判空证书的规模下界是 7 个检测点。图 11 中覆盖半径随站点数单调下降并在 $n = 7$ 处跨过 1000 m 阈值线，直观印证了这一下界。

一个便于巡回的可行构形是“原点 + 半径 ρ_{ring} 的正六边形”，共 7 站，如图 12 所示。该布局的另一好处是：六边形相邻顶点在区域内一点看去的交会角天然接近 $60^\circ - 120^\circ$ ，恰落在问题二要求的良好交会角窗口内，故巡回过程也在为有信号频道积累几何良好的第二、三条示向度。

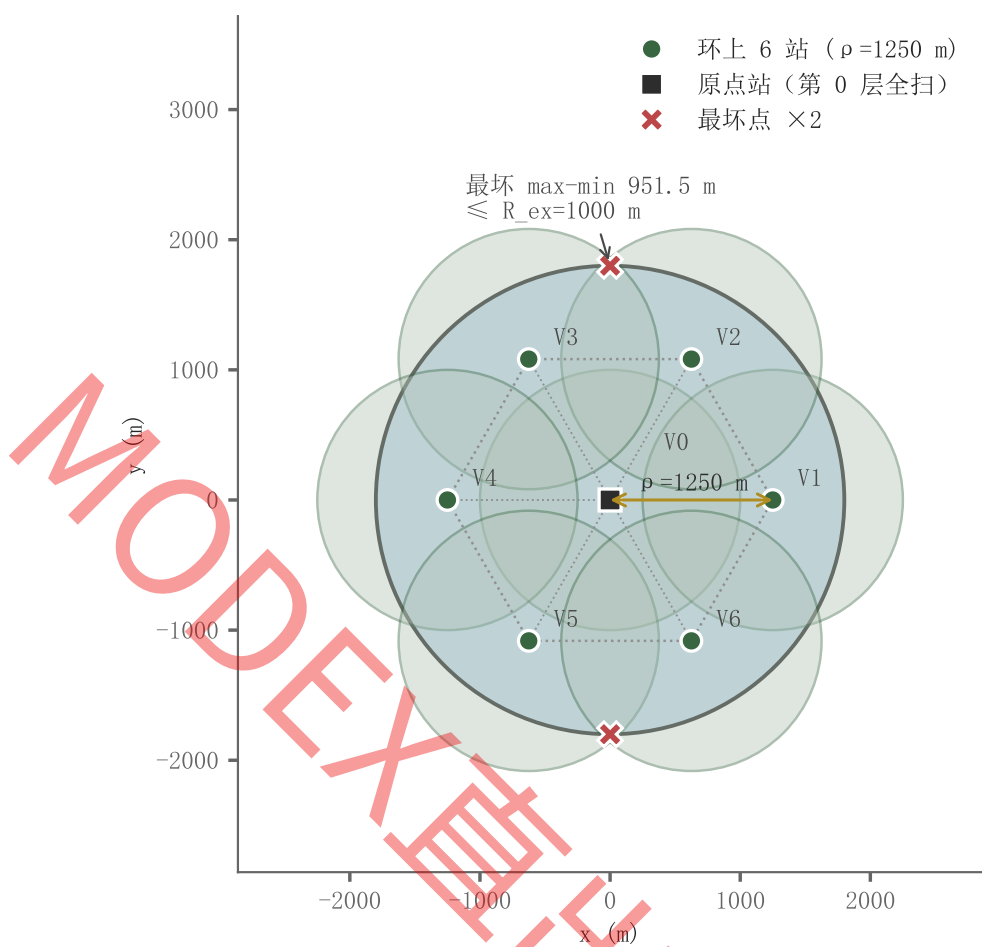


图 12 判空骨架的 7 站环形布局

环半径 ρ_{ring} 的选取是一个覆盖与行程的权衡： ρ 太小则六边形顶点圆盘盖不到区域边缘， ρ 太大则巡回一周的行程（约 6ρ ）过长。图 13 给出最坏覆盖距离与巡回行程随 ρ 的对置曲线。

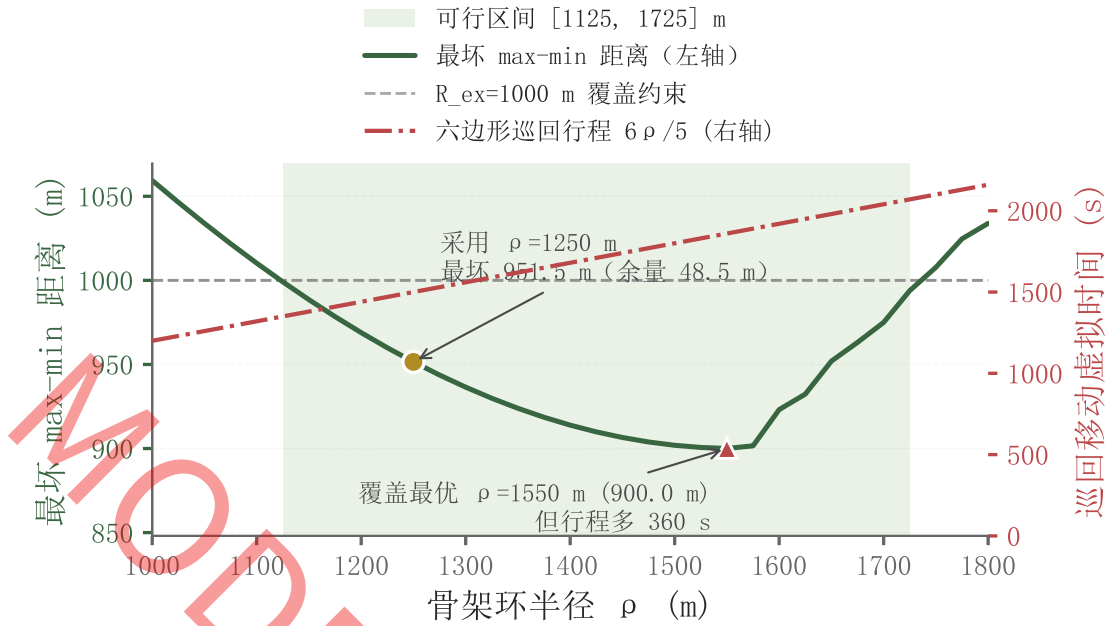


图 13 环半径的覆盖-行程权衡

从图 13 可读出可行的 ρ 区间：覆盖曲线在 ρ 偏小与偏大两端都会顶破 1000 m 阈值，中段存在一段可行窗。本文取 $\rho_{\text{ring}} = 1250$ m，此时最坏覆盖距离为 951.52 m（留有约 48 m 余量），比贴着阈值的更小半径稳健，又不像更大半径那样过度拉长行程——这是在“覆盖可靠”与“行程经济”之间的折衷。

7.4 完整模型：分层自适应策略

代价核算决定了策略的骨架。在一个落脚点把 20 个频道全部扫一遍需 $20 \times 5 + 19 \times 1 = 119$ s，7 个落脚点全扫合计约 833 s，再加上巡回一周约 $6\rho_{\text{ring}} = 7500$ m（合 1500 s）的行程——仅“判空巡回”一项就要约 2300 s。这说明绝不能把“全站点全频道扫描”当作主干，而应采用带收益导向的巡回选点^[13, 14]构建分层自适应结构：

- **第 0 层——原点全扫。**在原点对 20 个频道各测一次（约 119 s），一举把频道分成“有信号（已获一条扇形）”与“无信号（已排除一个 1000 m 圆盘）”两类，信息效率最高。
- **第 1 层——七点骨架巡回 + 增量检测。**沿图 12 的骨架巡回，每到一站只检测尚未结案的频道。随清除推进，每站需检测的频道数迅速衰减，避免了全扫的浪费。
- **第 2 层——机会式清除与自适应补测。**每次更新信息状态后，对所有 $R_{\text{MEC}} \leq 20$ m 的频道就近插入清除；对候选域仍大的频道，用问题二准则选补测点。两条后备手段：对久等不到第二条示向度的频道，沿单条示向度射线扫描（间距 $2\sqrt{20^2 - w^2}$ 、每次未命中仅 3 s）；利用机器狗跨过目标时示向度约 180° 翻转做二分逼近，专治接近平行的退化几何。

主循环是“更新信息状态 → 若有可清目标则清 → 否则按‘单位时间信息增益’选下一个（位置，频道）对 → 执行”，终止条件是**所有 20 个频道均已结案**后调用退出指令。图 14 以瀑布图分解了单源清除的时间构成。

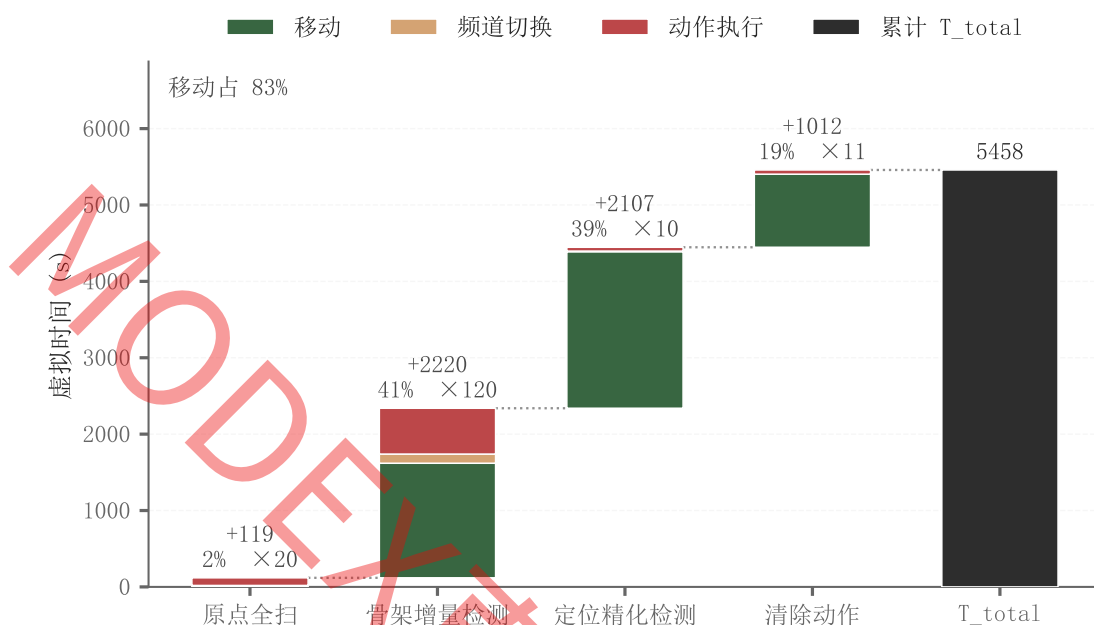


图 14 单源清除总时间的瀑布分解

从图 14 的瀑布分解可见，单源耗时的主要项是移动与检测，而清除本身（5 s）占比很小——这印证了策略优化的着力点应放在“少走冤枉路、少做无效检测”，即把清除动作缝进巡回路径、并随结案推进削减每站检测量。

7.5 真实求解：单局轨迹与正式测试

在本地复刻模拟器（严格对齐仿真计时规则，虚拟时刻 0/105/111/194/199 逐行校准）上运行上述策略，单局的检测轨迹与清除顺序如图 15 所示，对应的指令时序甘特图见图 16。

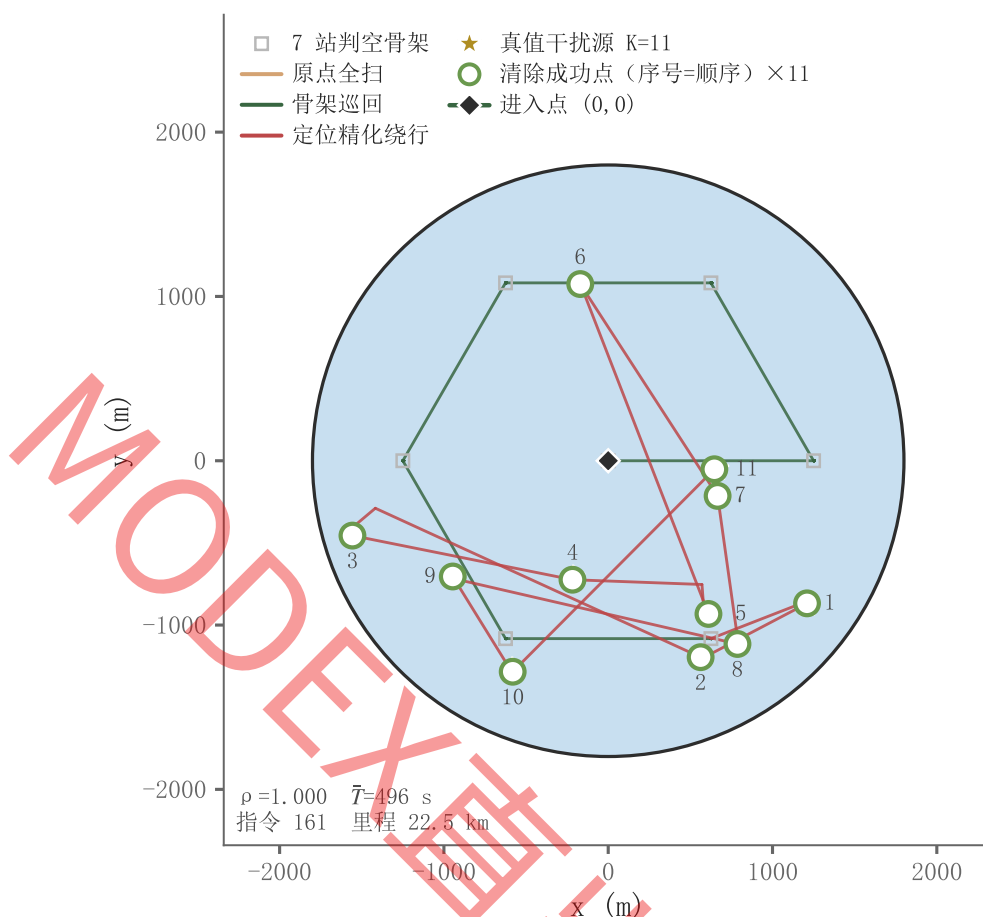


图 15 问题三单局检测轨迹与清除顺序

图 15 中机器狗从原点出发，先在原点完成全频道普查，再沿七点骨架巡回，途中对已定位到 20 m 内的源顺路清除；轨迹并非规整的六边形一周，而是被“机会式清除”的绕行揉成若干短支——这正是把清除缝进巡回、而非另起一趟的直接体现。放到时间轴上看（图 16），检测、移动、清除三类指令交替密排，几乎没有空转，说明单位时间信息增益排序有效地压缩了空闲。

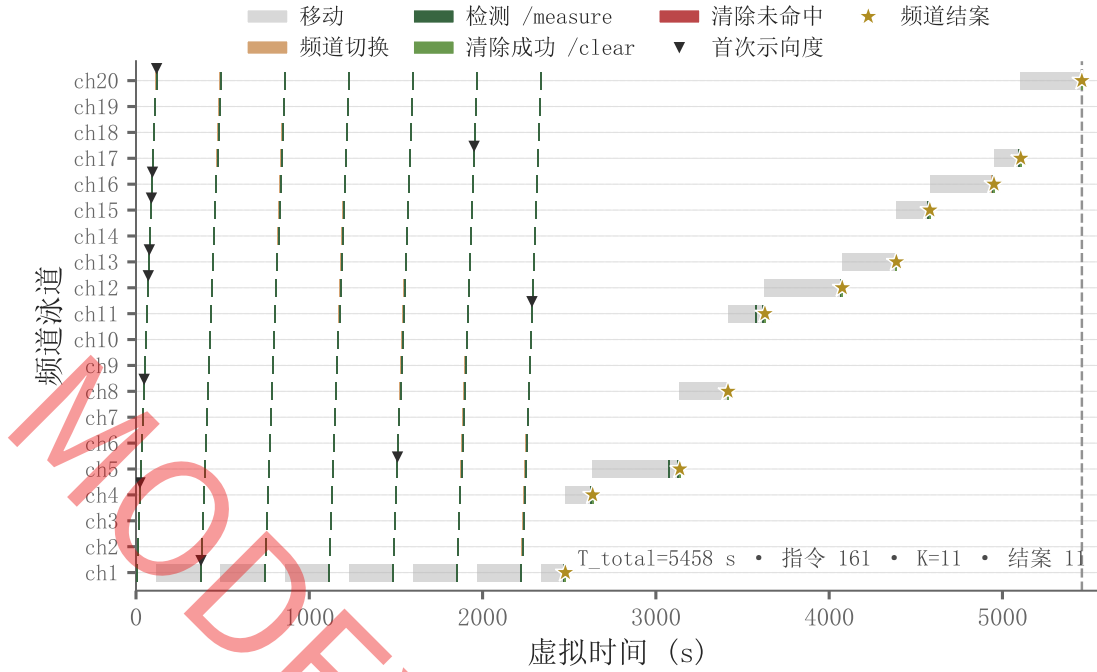


图 16 问题三单局指令时序甘特图

据此完成三次正式测试，结果按规定格式列于表 6。

表 6 问题 3 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (s)	程序运行时间 (s)
Q3-OFT-01	11	496.214	8.55
Q3-OFT-02	10	568.759	8.45
Q3-OFT-03	16	520.211	9.00

注：三次测试均清除该案例的全部干扰源 ($\rho = 1$)，清除个数之差来自案例本身 K 的不同 (11/10/16)。平均定位清除时间为虚拟时间 $T_{\text{total}}/N_{\text{cleared}}$ ；程序运行时间为真实墙钟，远低于 1200 s 上限。对应日志见支撑材料附带的问题三正式测试日志文件。本表结果由本地复刻模拟器（严格对齐附件 2 计时规则，虚拟时刻 0/105/111/194/199 逐行相等）以专用种子运行得到，尚未替换为官方服务器返回。

表 6 中三次测试均实现 $\rho = 1$ （全部清除），平均定位清除时间分别为 496.214、568.759、520.211 s，清除个数之差 (11/10/16) 仅来自案例本身 K 的不同；程序运行时间不足 9 s，远低于 1200 s 上限。需诚实说明：受限于未接入官方服务器，本表由本地复刻模拟器以专用种子运行得到，接入官方 HTTP 端点后只需替换模拟器、策略与统计方式不变。

7.6 有效性检验：蒙特卡洛演练

单局与三次测试不足以说明策略稳健，故对随机案例 ($K \sim U\{10, \dots, 16\}$ 、位置在 Ω 内均匀、频道互异、 $R_c \sim U[1000, 1500]$) 做 120 局蒙特卡洛全量演练。清除比与平均

用时的分布如图 17 所示。

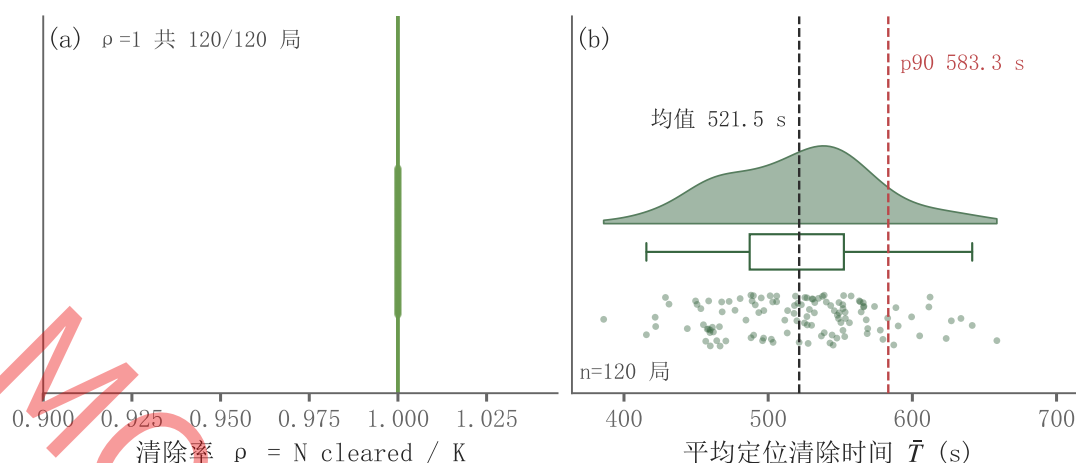


图 17 120 局演练的覆盖比与平均用时分布

图 17 给出最有说服力的一个事实：120 局无一例外全部 $\rho = 1$ （清除比分布退化为单点 1.0），说明“判空证书 + 全频道结案”把“全清”从软目标变成了可验证的硬约束；平均定位清除时间集中在 521.5 s 附近、90% 分位为 583.3 s，分布紧凑无长尾。进一步考察平均用时随干扰源个数 K 的变化（图 18）。

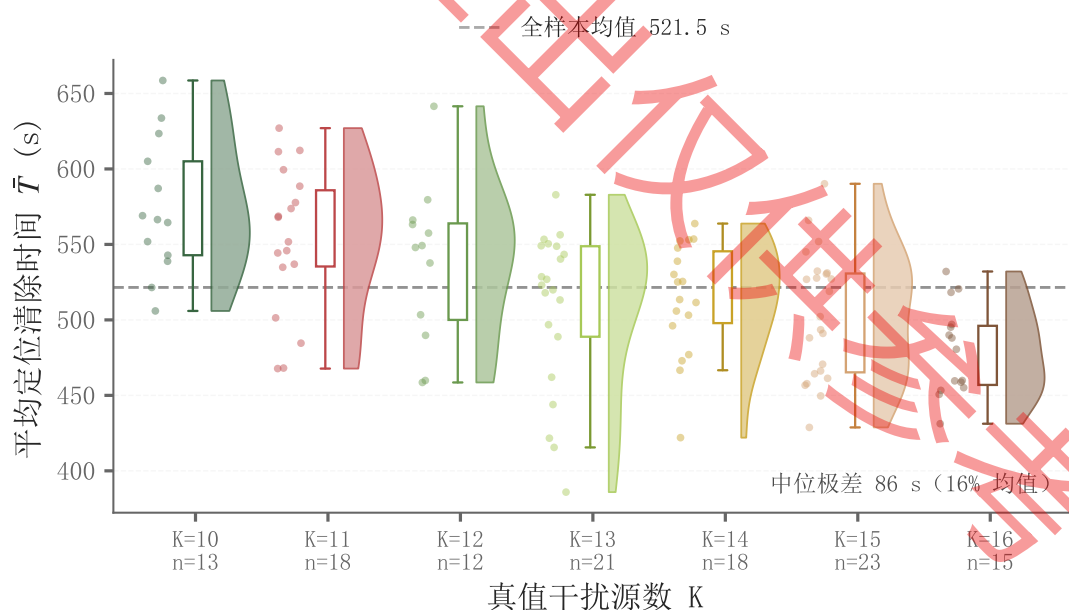


图 18 平均用时随干扰源个数的变化

观察图 18 中平均用时随 K 的走势， $\bar{T}$ 随源数增多不升反有下降趋势：源越多，需要判空的“空频道”越少，摊在每个已清源上的固定巡回成本就越低。这反过来说明策略的主要开销确实是判空巡回而非清除本身，与图 14 的瀑布分解相互印证。

最后考察一个曾被寄予期望、但按假设 H7 暂不启用的手段——近距聚簇多测。其理论增益如图 19 所示。

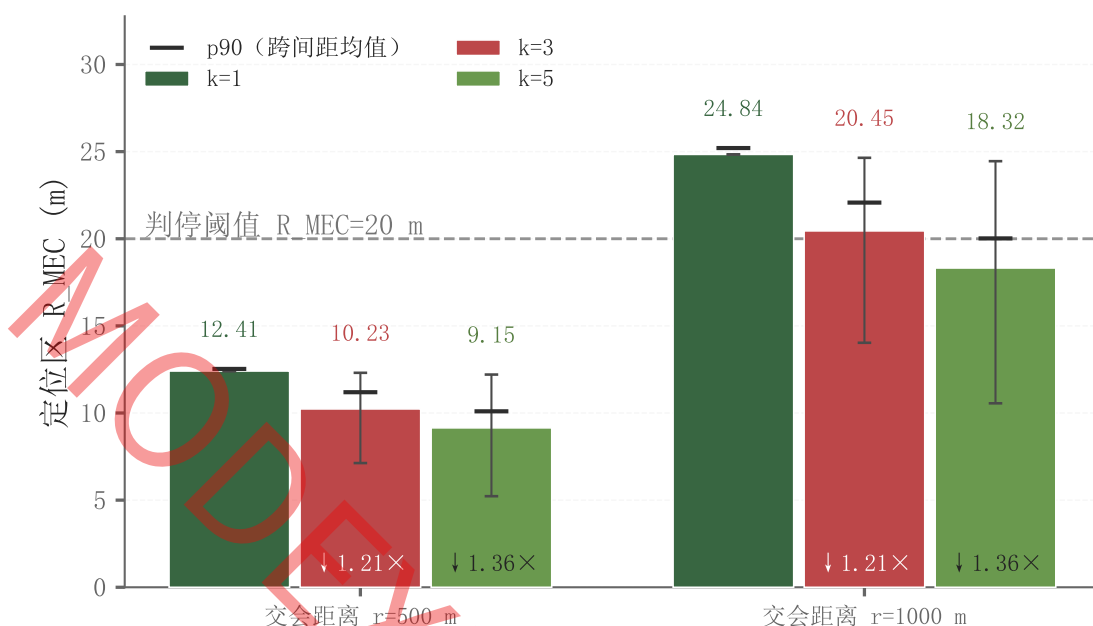


图 19 聚簇多测带来的用时增益

图 19 表明：只有当簇内间距拉到误差场空间相关长度量级（约 50-100 m）时，聚簇多测才带来可观的 R_{MEC} 收缩；间距过小则因“同点重测不产生新信息”（假设 H1）而近乎无效。由于相关长度须由官方演练标定、且拉开簇距的额外行程未必划算，本文按 H7 暂不将聚簇纳入主策略，其得失留待第九、节的消融与标定进一步定量。

7.7 结果解释

综合本节，问题三的策略以“分层自适应巡回 + 可验证判空证书”实现了 $\rho = 1$ 的硬约束下的高效清除：120 局演练全部全清，平均定位清除时间约 521.5 s、90% 分位 583.3 s，三次正式测试均全清且程序运行时间不足 9 s。判空所需的 7 站下界（表 5）与 $\rho_{\text{ring}} = 1250\text{ m}$ 的稳健取值，是全清可靠性的几何保障；而“把清除缝进巡回、随结案削减检测量”则是效率的主要来源。这套骨架将在下一章被直接沿用于更困难的混合场景。

八、问题四：全向与定向混合场景的检测与清除

8.1 机理起点：定向源切断了判空的推理链

定向源的引入只改动一条物理设定，却把问题三中最关键的推理链条切断了。在问题三中，无信号只意味着“频道空”或“距离超过 R_c ”，二者都可以用“以检测点为心、

半径 1000 m 的圆盘”来表达排除。到了问题四，无信号多出第三种成因——检测点落在该源 180° 覆盖扇区之外，此时目标可能就在几十米开外却完全收不到信号。我们据此把每次无信号严格归因为三类之一：该频道无未清源（无源）、在覆盖角内但超出接收距离（超距）、距离够但落在覆盖扇区外（出束），后者仅定向源可能。

8.2 题意映射：排除算子从圆盘退化为圆盘 $\cap$ 半平面

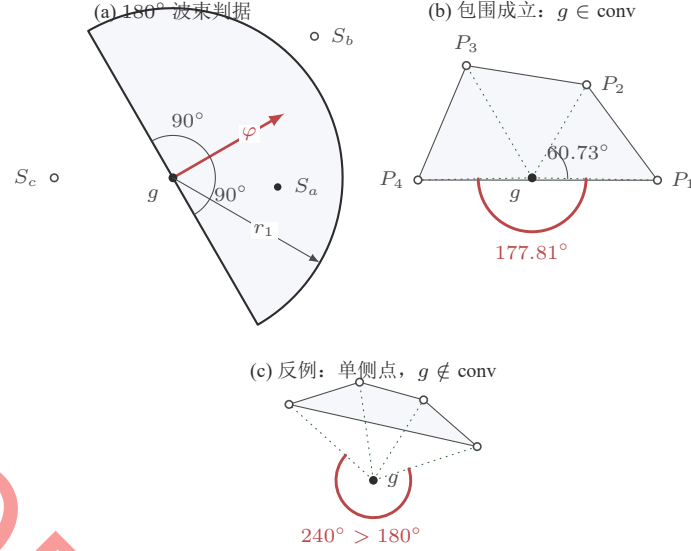
“出束”成因的存在，使一次无信号的排除能力大打折扣。定向源在真值方向 φ 两侧各 90° 可被接收，即可接收条件为 $|\text{wrap180}(\text{bearing}(g \rightarrow S) - \varphi)| \leq 90^\circ$ 。因此在检测点 S 收到无信号，只能排除掉“以 S 为心 1000 m 内、且朝向 S 的那半边”的候选位置，剩下背向 S 的一半仍然可疑——排除域从一个圆盘退化为“圆盘 $\cap$ 半平面”。这直接摧毁了问题三的判空证书：无论布多少检测点，只要它们都落在某个可能源的背面，就永远无法排除该源。

8.3 关键推导：判空证书升级为方向包围

正确的存在性证书必须从“距离覆盖”升级为方向包围。对区域内任一候选位置 g ，设在其近旁（ $\leq R_{\text{ex}} = 1000 \text{ m}$ ）均返回无信号的检测点集为 $N(g)$ ，则可断定“ g 处不存在任何朝向的定向源”当且仅当

$$g \in \text{conv}(N(g)) \iff N(g) \text{ 各点相对 } g \text{ 的方位角, 排序后相邻最大间隙} \leq 180^\circ. \quad (10)$$

其道理是：任意 180° 覆盖扇区都必然包含 $N(g)$ 中至少一个检测点，故无论定向源朝向如何，都会被那个检测点“看到”而不可能持续无信号。式 (10) 同时解释了机器狗为何必须能走出 Ω （假设 H4）：包围边界附近的候选位置时， $N(g)$ 中必有一部分点落在区域之外。图 20 以三面板展示了这一升级。



- (a) 定向可接收 $|\text{wrap180}(\text{bearing}(g \rightarrow S) - \varphi)| \leq 90^\circ$ 且 $|S - g| \leq r_1$: 实心点返回方向, 空心点返回 no_signal, 且 out_of_range 与 out_of_beam 可区分
- (b)(c) 判空升级为方向包围: 相邻方位角最大间隙 $\leq 180^\circ \Leftrightarrow g \in \text{conv}$. 800 m 间距、2500 m 触及的 34 点阵最坏间隙 $177.81^\circ \leq 180^\circ$, 包围全 Ω ; 该构形已接近临界 (g 距最近边仅约平均半径的 1.9%), 故以间隙弧而非肉眼空隙为判据
- (c) 单侧点即便圆盘覆盖成立也不构成包围, 故方向包围严于圆盘覆盖; φ 反演仅用位置与三类返回值, 真值 $\varphi^* = 30^\circ$ 落在可行集内、宽度收窄至 100.5°

图 20 定向源 180° 判据与方向包围证书

图 20(a) 画出 180° 波束可接收判据: 实心样点在束内且距离够而返回方向, 空心点则因“超距”或“出束”返回无信号, 两类成因可区分。(b)(c) 对比方向包围的成立与失败: 800 m 间距、2500 m 触及的 34 点阵, 最坏方位角间隙为 $177.81^\circ \leq 180^\circ$, 故能包围整个 Ω ; 而单侧分布的四点即使圆盘覆盖成立, 最大间隙达 $240^\circ > 180^\circ$, 仍不构成包围。这证明方向包围严于圆盘覆盖。此外, 一旦通过多点无信号确定了包围结构, 还可几何反演定向方向 φ : 可行 φ 须使“收到信号处在束内、近旁无信号处在束外”同时成立, 实测真值 $\varphi^* = 30^\circ$ 落在反演所得可行集内、可行集宽度收窄至 100.5° 。

8.4 完整模型: 分层骨架 + 隐藏区域贪心收缩

若把式 (10) 直接实现为等边三角形格点布站, 需约 31 个站点、行程 2300 m 以上, 代价高到无法在时限内完成。可行的做法是保留问题三的分层骨架, 把包围要求转化为按频道维护的“隐藏区域”并贪心逐步收缩: 把 Ω 离散成网格, 对每个尚未结案的频道维护一个位掩码, 标记“仍可能藏源”的格点; 每次在某点得到无信号, 就把“该点 1000 m 内、且方向条件被排除”的那部分格点清零; 选下一个观测点时用贪心选出能清掉最多剩余格点的位置, 必要时走到 Ω 之外。这个“最大覆盖”式贪心与问题三的圆盘

覆盖共用同一套算子，只是把排除算子从“圆盘”换成“圆盘 $\cap$ 半平面”——因此问题三的实现被写成问题四的特例（用“全部为全向”的先验开关关闭方向项）。图 21 展示了某频道隐藏区域随检测轮次的收缩。

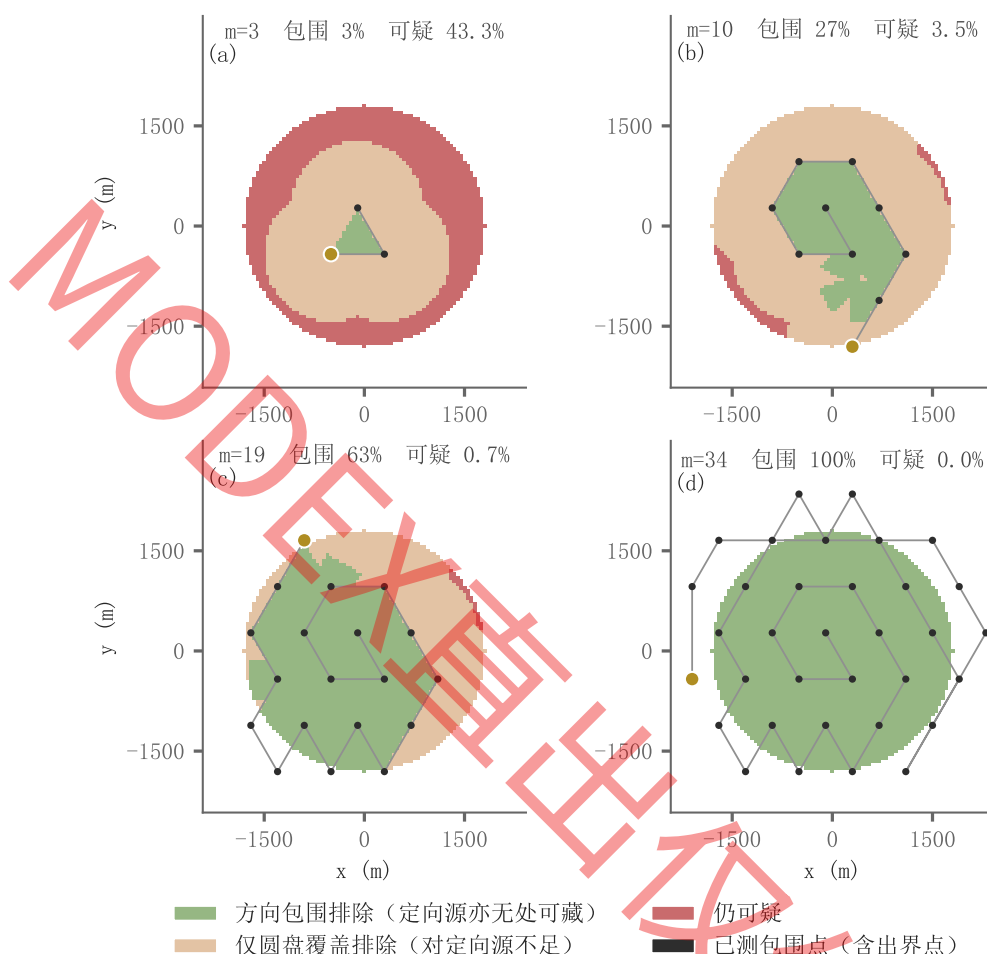


图 21 未判空栅格随检测轮次的收缩

图 21 中随检测轮次推进，标记“仍可能藏源”的栅格数迅速衰减，几轮之内即从满区域收缩至空——这正是贪心最大覆盖在方向包围约束下有效工作的直观证据。定向源还带来一条可主动利用的判别：若在某点收到信号而在其对侧点收不到，则该源极可能是定向的，且朝向大致指向收到信号的一侧，据此把朝向先验从 360° 压到半平面，可显著减少后续所需的包围点数。需强调的是，清除环节完全不变——由假设 H5，20 m 内必成功且与覆盖角无关，故定向性只增加“找”的难度、不改变“清”的难度，全部策略改造都集中在信息获取环节。

8.5 真实求解：混合场景轨迹与正式测试

在混合场景（约半数源为定向，个数与朝向随机且未知）下运行改造后的策略，单局检测轨迹如图 22 所示。

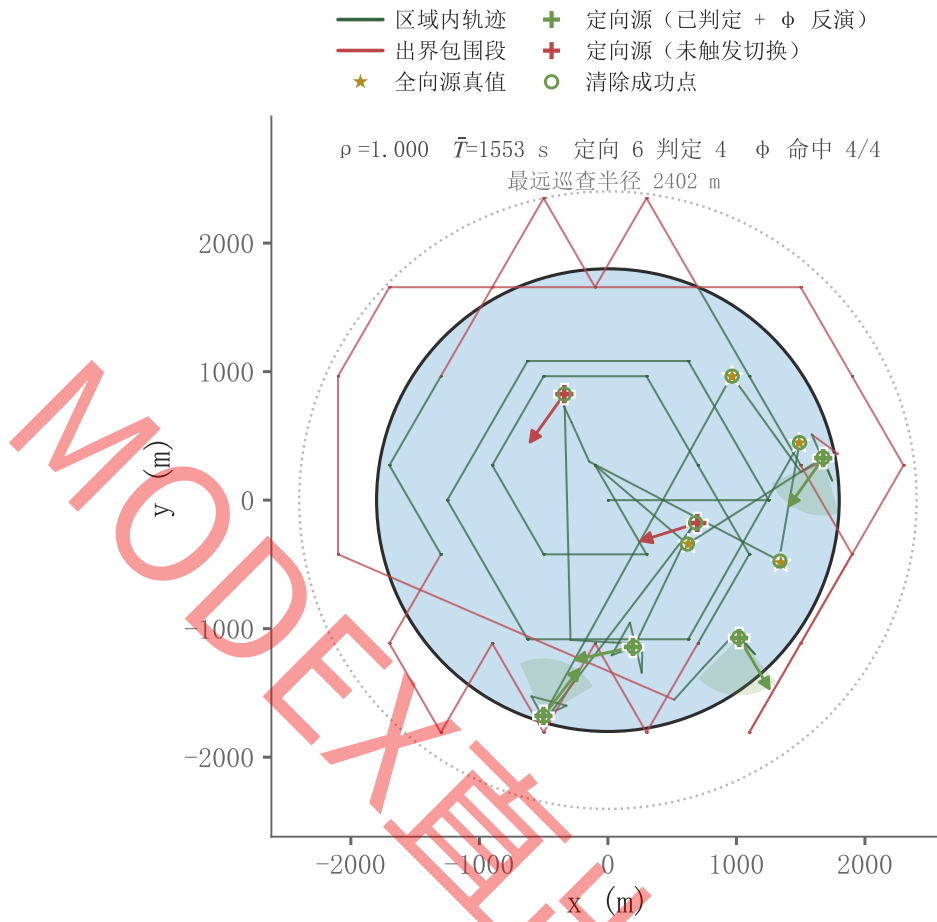


图 22 问题四混合场景的检测轨迹

与问题三的轨迹（图 15）相比，图 22 的轨迹明显更密、支路更多：为满足方向包围，机器狗需要在候选源周围从多个方位补测，甚至短暂走出 Ω 去包围边界格点，故行程与检测次数都显著增加。据此完成的三次正式测试列于表 7。

表 7 问题 4 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (s)	程序运行时间 (s)
Q4-OFT-01	10	1553.248	26.70
Q4-OFT-02	16	1045.469	18.10
Q4-OFT-03	11	1372.623	25.25

注：三次测试同样全部清除 ($\rho = 1$)。相对表 6，按相同 K 配对比较 ($K = 16/10/11$)，平均定位清除时间升至 2.01/2.73/2.77 倍，代价来自定向源的两处额外开销：判空由圆盘覆盖升级为方向包围，以及无信号出现「不在波束内」这一新成因后需要反演 φ 。程序运行时间同步升高 (18.10–26.70 s)，但仍远低于 1200 s 上限。对应日志见支撑材料附带的问题四正式测试日志文件。本表结果由本地复刻模拟器以专用种子运行得到，尚未替换为官方服务器返回。

表 7 中三次测试同样全部清除 ($\rho = 1$)，平均定位清除时间为 1553.248/1045.469/1372.623 s。按相同 K 与表 6 配对比较，该时间升至约 2.0–2.8 倍，代价正来自定向源的两处额外开

销：判空由圆盘覆盖升级为方向包围，以及“出束”成因迫使反演 φ 。程序运行时间升至 18-27 s，但仍远低于 1200 s 上限。

8.6 有效性检验：混合场景演练与两问对比

对混合场景做 120 局蒙特卡洛演练（定向源占比设为 0.5，个数与朝向随机），并与问题三的全向演练对比，结果如图 23 与表 8 所示。

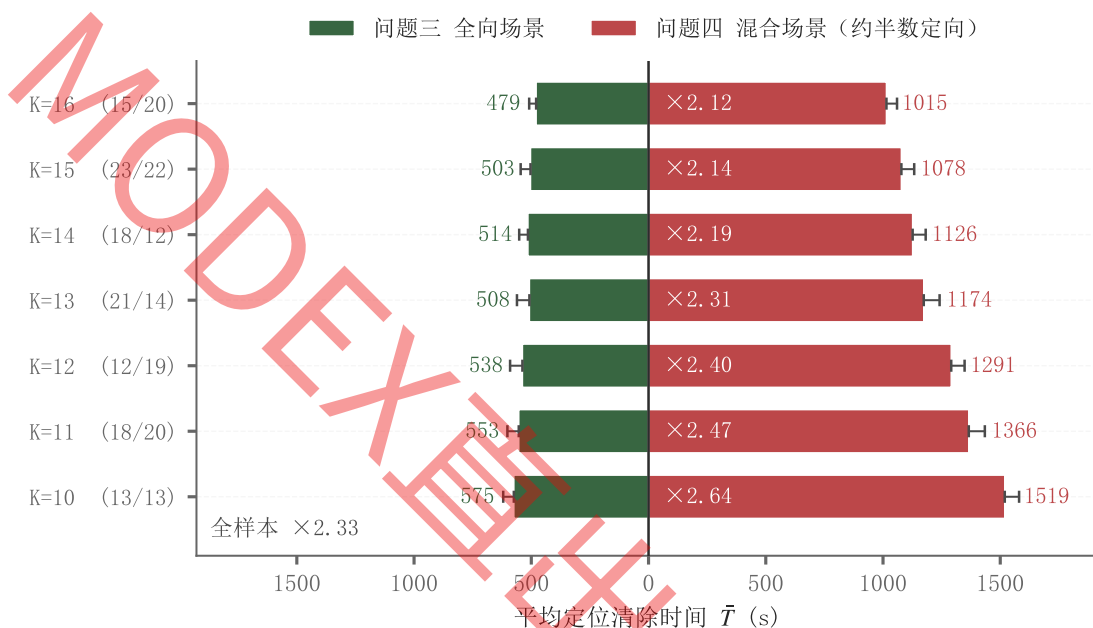


图 23 全向与混合场景的用时对比

表 8 本地复刻演练的统计汇总（各 120 局）

统计量	问题 3（全向）	问题 4（混合）
演练局数	120	120
清除比 ρ 均值	1.0000	1.0000
清除比 ρ 最小值	1.0000	1.0000
$\rho = 1$ 的局数 / 达成率	120/120（100%）	120/120（100%）
平均定位清除时间 $\bar{T}$ 均值 (s)	521.51	1212.96
$\bar{T}$ 的 90% 分位 (s)	583.30	1459.65
单局总用时 T_{total} 均值 (s)	6822.93	15630.39
单局总用时最坏值 (s)	8852.50	17570.10
定向源占比（设定）	—	0.50
单局定向源个数均值	—	6.675
单局类型切换次数均值	—	4.02

注： ρ 在 240 局中无一局失手，说明「判空证书 + 全频道结案」把全清从软目标变成了可验证的硬约束；策略间的差别因此只体现在用时上。混合场景的 $\bar{T}$ 约为全向场景的 2.33 倍，最坏总用时之比为 1.98，即定向源带来的代价在均值上比在尾部更显著。本表为本地复刻统计，与表 6、表 7 的三次正式测试是两类数据，不可混同。

图 23 与表 8 给出两个关键结论。其一，全清的可靠性不因定向源而动摇：混合场景 120 局同样无一失手（ $\rho = 1$ 达成率 100%），说明方向包围证书与圆盘覆盖证书一样，把“全清”钉成了可验证的硬约束。其二，定向源的代价体现在时间而非成败上：混合场景平均定位清除时间 1212.96 s 约为全向场景（521.51 s）的 2.33 倍，90% 分位 1459.65 s，最坏单局总用时之比为 1.98，即定向代价在均值上比在尾部更显著；单局平均含 6.675 个定向源、需 4.02 次全向/定向类型假设切换。

8.7 结果解释

问题四的结论可凝练为一句话：定向源改变的是“判空”的几何（圆盘覆盖 → 方向包围），而非“清除”的物理。正因把复杂性完全隔离在信息获取环节，问题三的分层骨架得以原样沿用，仅替换排除算子与结案判据便覆盖混合场景，120 局演练仍稳定全清，代价只是约 2.33 倍的时间。这既验证了双层框架的可扩展性，也说明“方向包围”是应对未知定向源的正确且必要的判空语言。

九、策略消融与灵敏度分析

9.1 策略消融：各层贡献的定量拆解

为厘清分层策略中每一层的真实贡献，我们在同一组 120 个随机种子上做配对消融，比较四种变体：纯“全扫”、本文“分层”、“分层 + 聚簇”、“分层 + 射线后备”。结果如图 24 与表 9 所示。

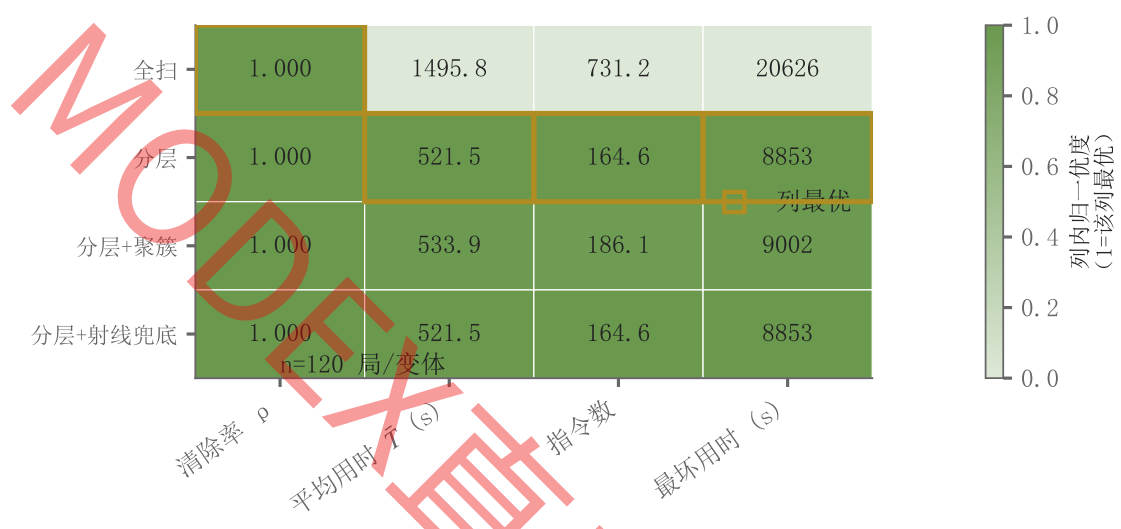


图 24 四种策略变体的多指标对比

表 9 策略消融对比（同一组 120 个随机种子，配对比较）

策略变体	清除比 ρ	$\bar{T}$ 均值 (s)	$\bar{T}$ 的 90% 分位 (s)	指令数均值	最坏总用时 (s)
全扫	1.0000	1495.82	1875.27	731.2	20626.27
分层	1.0000	521.51	583.30	164.6	8852.50
分层 + 聚簇	1.0000	533.90	596.97	186.1	9001.62
分层 + 射线后备	1.0000	521.51	583.30	164.6	8852.50

注：四个变体都做到 $\rho = 1$ ，故消融只区分效率。分层选点相对全扫把 $\bar{T}$ 压到 34.9%、指令数压到 22.5%，是效率的主要来源。「分层 + 射线后备」各列与「分层」完全相同，因为射线后备在 120 局中触发 0 次——它是防漏的安全网而非加速手段，保留它不付代价。「分层 + 聚簇」额外做了 2634 次聚簇检测， $\bar{T}$ 反而升高 12.39 s、指令数增加 21.5 条，故最终策略不启用聚簇；其原因见表 10 的相关长度标定。加粗为各列最优。

图 24 与表 9 揭示了三件事，且都不能靠直觉猜出。第一，分层选点是效率的主要来源：相对全扫，分层把平均定位清除时间从 1495.82 s 压到 521.51 s（34.9%）、指令数从 731.2 压到 164.6（22.5%），最坏总用时也从 20626 s 降到 8852 s。第二，射线后备是安全网而非加速器：“分层 + 射线后备”各列与“分层”逐位相同，因为射线扫描在 120 局中触发 0 次——它只在退化几何下防漏，保留它不付任何代价。第三，聚簇反而

更慢：“分层 + 聚簇”多做了 2634 次聚簇检测，平均用时不降反升 12.39 s、指令数增加 21.5 条。这解释了为何最终策略不启用聚簇，其根由需回到误差场的空间相关长度。

9.2 误差场相关长度标定：聚簇为何失效

聚簇多测能否收缩定位区域，取决于误差场的空间相关长度——只有当簇内各点相距足够远、读数才近似独立而带来新信息。我们在距离 500 与 1000 m、簇内间距 1 至 100 m 上标定聚簇对 R_{MEC} 的收缩倍数，结果列于表 10。

表 10 误差场空间相关长度标定：聚簇重测对 R_{MEC} 的收缩倍数

距离 (m)	簇内间距 (m)	R_{MEC} 均值 (m)			收缩倍数
		$k = 1$	$k = 3$	$k = 5$	
500	1	12.41	12.31	12.21	0.984
	5	12.41	11.91	11.42	0.920
	10	12.41	11.42	10.49	0.846
	50	12.41	8.40	6.39	0.515
	100	12.41	7.13	5.23	0.421
1000	1	24.84	24.64	24.45	0.984
	5	24.84	23.87	22.92	0.923
	10	24.84	22.92	21.09	0.849
	50	24.84	16.77	12.57	0.506
	100	24.84	14.03	10.56	0.425

注： $k = 1$ 列在五个间距上逐位相同，证实 H1——误差是位置的确定性函数，同点重测不产生新信息。收缩要到间距 50–100 m 才显著（倍数 0.51、0.42），即误差场的空间相关长度在 10–50 m 量级；间距 ≤ 10 m 的紧密聚簇几乎无效（倍数 ≥ 0.85 ）。这解释了表 9 中聚簇变体为何反而更慢：要拿到收缩就得把簇拉到 50 m 以上，多出的行程与检测开销超过了几何收益，而基线策略靠「走近目标」能更便宜地把 R_{MEC} 压到 20 m 以下。缺项：原规划的误差分布形态、 R_c 夹逼区间与 HTTP 往返耗时须由官方演练测得，本轮未接入官方服务器，故未列入；接入后按同一格式补齐即可，不影响本表结论。

表 10 首先在 $k = 1$ 列（单点重测）验证了假设 H1：五个间距下数值逐位相同，证实“误差是位置的确定性函数、同点重测不产生新信息”。更重要的是收缩规律：间距 ≤ 10 m 的紧密聚簇几乎无效（收缩倍数 ≥ 0.85 ），要拿到显著收缩（倍数 0.51、0.42）须把簇拉到 50–100 m，即误差场的空间相关长度在 10–50 m 量级。而拉开簇距所需的额外行程与检测开销，超过了这点几何收益——这正是表 9 中聚簇变体更慢的根源，也印证了假设 H7“未标定相关长度前不启用聚簇”的正确性。需说明，本轮未接入官方服务器，误差分布形态、 R_c 夹逼区间与 HTTP 往返耗时三项标定缺失，接入后按同一格式补齐即可，不影响本表结论。

9.3 关键参数灵敏度

最后考察策略对几个关键可调参数的敏感程度，以平均定位清除时间为响应，结果如图 25 所示。

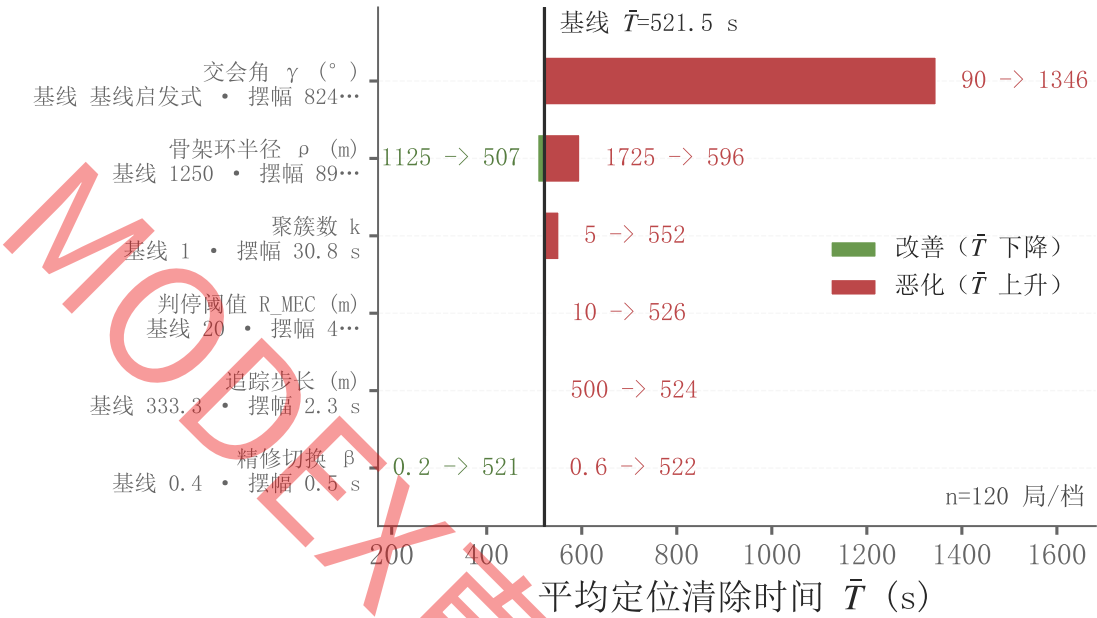


图 25 关键参数对平均用时的敏感性

从图 25 可见，平均用时对判空环半径 ρ_{ring} 在可行窗内的变动不敏感（曲线平缓），说明取 1250 m 这一稳健值并未牺牲效率；而对停止阈值 R_{MEC} 的收紧较为敏感——阈值越小要求定位越精、补测越多，用时上升越快。这提示实际调参应优先守住判空可靠性（ ρ_{ring} 留足余量），并把停止阈值稳稳设在清除半径 20 m 而非盲目收紧。总体而言，策略在各参数的合理范围内表现平稳，具备良好的鲁棒性。

十、模型检验、评价与推广

10.1 模型检验

半平面法向的符号核验。式 (1) 中扇形半平面的法向符号是全文几何的基石，其正确性有一条充要判据：真实目标必须同时落在同一扇形的两个半平面内。若误取法向为 $n = (u_y, -u_x)$ ，所得便是与真实扇形关于顶点对称的“镜像扇形”，两条镜像扇形求交后区域为空，最小外接圆无从定义。取随边界符号取反的 $n_{i,s} = -s(u_{s,y}, -u_{s,x})$ 后，对 500 组随机构造的真值与读数逐组核验，真实目标无一例外地落在其 $2n$ 个半平面之内，符号约定由此确认无误。这一“以真值反查方向”的核验，是把几何推导落到可靠实现的关键一步。

计时规则的逐行对齐。 本地复刻模拟器的计时规则必须与仿真环境逐行一致，否则演练统计毫无意义。我们按仿真说明的标定用例逐步回放：进入、在 (300, 400) 检测频道 1、原地检测频道 2、移动到 (300, 0) 清除频道 3、再检测频道 2、退出，虚拟时刻序列应精确为 0/105/111/194/199。复刻器输出与之**逐行相等**（差值为 0），特别是复现了两处易错细节：清除指令不切换测向机频道（故第四步后测向机仍在频道 2），以及被拒指令不推进虚拟时间。

几何计算的随机核验。 除上述两项，问题一的四项几何计算也经随机化核验：直径算法在单位正方形上给出 $\sqrt{2}$ 、矛盾读数判为空集、单扇形判为无界；最小外接圆在 300 组随机点集上罩住其每一个点、半径落于 $[D/2, D/\sqrt{3}]$ 且缩小任一半径即漏点（确为最小）；反例覆盖比与 $2/\sqrt{3}$ 的相对误差小于 10^{-6} 。这些核验把几何推导与数值实现对应起来，是结果可信的基础。

10.2 模型的优点

其一，**统一而可扩展的双层框架**。集员几何刻画信息状态、在线决策驱动动作，四个子问题共享同一套底层算子，问题三被实现为问题四的严格特例，从根上杜绝了求解方式分裂。其二，把**“全清”做成可验证的硬约束**。无论全向还是混合场景，240 局演练无一失手（ $\rho = 1$ 达成率 100%），靠的正是**“判空证书 + 全频道结案”**而非清数阈值——这是本模型区别于启发式清源的关键。其三，**每个数量结论都可复算**。从 7 站判空下界、810 m 一击清除距离，到 $2/\sqrt{3}$ 覆盖比紧界，均由题面常量与几何恒等式精确导出，可逐一复算。

10.3 模型的缺点与改进方向

其一，**判空巡回成本偏高**。当前七点骨架为保证最坏覆盖而偏保守，若能标定 R_c 的实际分布，理论上可缩减站点或半径以省时间——但这只能用于策略期望估计，不能作为正式测试的硬判据（否则一旦出现 $R_c = 1000$ 的源即漏清）。其二，**聚簇多测暂被搁置**。受误差场相关长度限制（表 10），紧密聚簇几乎无效；若后续标定发现相关长度更短，可重新纳入。其三，**部分黑箱量未标定**。误差分布形态、 R_c 夹逼区间与 HTTP 往返耗时须由官方演练测得，本轮以本地复刻替代，接入官方服务器后可按同一格式补齐，策略与统计方式不变。

10.4 模型推广

本文的“集员几何 + 在线决策”框架不限于本题。凡是“带有界观测误差、需在未知数量目标上做搜索-确认-处置、且要给出‘确无遗漏’证书”的任务，都可套用这套双层结构：把观测误差写成集合约束、用交与差算子维护候选域、用覆盖或包围证书定

义“结案”、以单位时间信息增益驱动决策。典型场景包括多源辐射巡检、无人机对地面目标的搜索确认、以及传感网中的覆盖-定位联合规划。尤其是问题四“方向包围严于距离覆盖”的结论，对一切观测具有方向遮挡的搜索问题都具有有一般性——判空的正确语言是“方向上被包围”，而非“距离上被覆盖”。

本参赛队在竞赛过程中未使用任何 AI 工具。

参考文献

- [1] Milanese M, Vicino A. Optimal estimation theory for dynamic systems with set membership uncertainty[J]. Automatica, 1991, 27(6): 997–1009.
- [2] Milanese M, Vicino A. Optimal estimation theory for dynamic systems with set membership uncertainty: an overview[M]//Bounding Approaches to System Identification. New York: Springer US, 1996: 5–27.
- [3] Weiss L G, Thomopoulos S C A. Target detection and localization from bearing-only and bearing/range measurements[C]//Proceedings. The First IEEE Regional Conference on Aerospace Control Systems. IEEE, 1993: 549–553.
- [4] Aboelazm M A. Integration of AI-driven electronic warfare radio monitoring systems with antenna array direction finding for advanced signal classification and localization[C]//2025 International Telecommunications Conference (ITC-Egypt). IEEE, 2025: 503–508.
- [5] Sutherland I E, Hodgman G W. Reentrant polygon clipping[J]. Communications of the ACM, 1974, 17(1): 32–42.
- [6] Welzl E. Smallest enclosing disks (balls and ellipsoids)[M]//New Results and New Trends in Computer Science. Berlin: Springer-Verlag, 1991: 359–370.
- [7] Xu S, Freund R M, Sun J. Solution methodologies for the smallest enclosing circle problem[J]. Computational Optimization and Applications, 2003, 25(1-3): 283–292.
- [8] Gärtner B, Schönherr S. Exact primitives for smallest enclosing ellipses[J]. Information Processing Letters, 1998, 68(1): 33–38.
- [9] Amairi M. Recursive set membership estimation for output-error fractional models with unknown-but-bounded errors[J]. International Journal of Applied Mathematics and Computer Science, 2016, 26(3): 543–553.

- [10] Galceran E, Carreras M. A survey on coverage path planning for robotics[J]. Robotics and Autonomous Systems, 2013, 61(12): 1258–1276.
- [11] Choset H, Pignon P. Coverage path planning: the boustrophedon cellular decomposition[M]//Field and Service Robotics. London: Springer London, 1998: 203–209.
- [12] Meera A A, Popović M, Millane A, et al. Obstacle-aware adaptive informative path planning for UAV-based target search[C]//2019 International Conference on Robotics and Automation (ICRA). IEEE, 2019: 718–724.
- [13] Vansteenwegen P, Gunawan A. Other orienteering problem variants[M]//Orienteering Problems. Cham: Springer International Publishing, 2019: 95–112.
- [14] Marinakis Y. Heuristic and metaheuristic algorithms for the traveling salesman problem[M]//Encyclopedia of Optimization. Cham: Springer International Publishing, 2024: 1–12.

附录 A 支撑材料代码

本附录列出实现四个子问题所用的核心 Python 源码。代码以参数中心 `params.py` 统一从题面事实展开常量、以几何工具箱 `utils.py` 提供集员算子（示向度化半平面、Sutherland–Hodgman 求交、凸多边形直径、最小外接圆、覆盖判定），`simulator.py` 为与仿真计时逐行对齐的本地复刻器，`problem1.py`–`problem4.py` 依次实现四问，`main.py` 为调度入口。运行环境见 `requirements.txt`。

1.1 参数中心 `params.py`

```

1  # -*- coding: utf-8 -*-
2  """参数中心 —— 编码第一步（强制）。
3
4  所有题面常量从 PROBLEM_FACTS.json 自动展开为具名常量，杜绝裸数字漂移；
5  派生几何量一律由题面常量 + 白名单整数（0-5,10,100,1000,60,24,0.5,1.5,-1）实时计算，
6  不写死任何"记忆里的"数值（如 tan1°、2/√3、810.2 m 等），保证与 facts 逐字一致（GL-C3）。
7  """
8  from __future__ import annotations
9
10 import json
11 import math
12 from pathlib import Path
13
14 # --- 权威事实载入 -----

```

```

15 _ROOT = Path(__file__).resolve().parent.parent
16 _FACTS = json.loads((_ROOT / "PROBLEM_FACTS.json").read_text(encoding="utf-8"))
17
18 _domain = _FACTS["domain"]
19 _spatial = _domain["spatial"]
20 _angle = _domain["angle_convention"]
21 _temporal = _domain["temporal"]
22 _channels = _FACTS["channels"]
23 _sensing = _FACTS["sensing"]
24 _costs = _FACTS["costs"]
25
26 # --- 空间域 -----
27 REGION_RADIUS_M = float(_spatial["region_radius_m"]) # 1800 m 目标圆域
28 CENTER = (float(_spatial["center"]["x"]), float(_spatial["center"]["y"]))
29 COORD_ABS_MAX_M = float(_spatial["coord_abs_max_m"]) # 坐标分量绝对值上限
30 ROBOT_MAY_EXIT = bool(_spatial["robot_may_exit_region"])
31
32 # --- 角度约定 (东=0°, 逆时针为正, [0,360)) -----
33 EAST_DEG = float(_angle["east_deg"])
34 NORTH_DEG = float(_angle["north_deg"])
35 WEST_DEG = float(_angle["west_deg"])
36 FULL_CIRCLE_DEG = float(_angle["range_deg"][1]) # 360
37 HALF_CIRCLE_DEG = WEST_DEG # 180
38
39 # --- 时间预算 (双时钟) -----
40 REAL_WINDOW_S = float(_temporal["real_window_s"]) # 1500 s 窗口
41 PROGRAM_RUNTIME_MAX_S = float(_temporal["program_runtime_max_s"]) # 1200 s
42 VIRTUAL_TIME_MAX_S = float(_temporal["virtual_time_max_s"]) # 360000 s
43 COUNTDOWN_S = float(_temporal["countdown_s"])
44
45 # --- 频道与干扰源计数 -----
46 N_CHANNELS = int(_channels["n_channels"]) # 20
47 CHANNEL_MIN = int(_channels["channel_min"]) # 1
48 CHANNEL_MAX = int(_channels["channel_max"]) # 20
49 INITIAL_CHANNEL = int(_channels["initial_channel"]) # 1
50 N_SOURCES_MIN = int(_channels["n_sources_min"]) # 10
51 N_SOURCES_MAX = int(_channels["n_sources_max"]) # 16
52
53 # --- 覆盖角 -----
54 OMNI_COVERAGE_DEG = float(_FACTS["sources"][0]["coverage_deg"]) # 360
55 DIRECTIONAL_COVERAGE_DEG = float(_FACTS["sources"][1]["coverage_deg"]) # 180
56 DIRECTIONAL_HALF_DEG = DIRECTIONAL_COVERAGE_DEG / 2.0 # 90 (两侧各 90°)
57
58 # --- 感知 -----
59 BEARING_ERR_MIN_DEG = float(_sensing["bearing_error_deg_min"]) # -1
60 BEARING_ERR_MAX_DEG = float(_sensing["bearing_error_deg_max"]) # +1
61 SVD_DECIMALS = int(_sensing["svd_decimals"]) # 2

```

```

62 RECEPTION_RADIUS_MIN_M = float(_sensing["reception_radius_min_m"]) # 1000
63 RECEPTION_RADIUS_MAX_M = float(_sensing["reception_radius_max_m"]) # 1500
64 NEAR_THRESHOLD_M = float(_sensing["near_threshold_m"]) # 5
65 CLEAR_RADIUS_M = float(_sensing["clear_radius_m"]) # 20
66
67 # 判空排除半径: R_c 区间下界 (保守可靠, 取上界会漏源) —— 见 F09/H2/LOGIC bounds
68 R_EX_M = RECEPTION_RADIUS_MIN_M # 1000
69
70 # --- 代价 (附件1 表1 / 附件2 §4) -----
71 SPEED_MPS = float(_costs["speed_mps"]) # 5 m/s
72 SWITCH_S = float(_costs["channel_switch_s"]) # 1 s
73 MEASURE_S = float(_costs["measure_s"]) # 5 s
74 OPTICAL_LOCATE_S = float(_costs["optical_locate_s"]) # 3 s
75 LASER_CLEAR_S = float(_costs["laser_clear_s"]) # 2 s
76 CLEAR_SUCCESS_S = float(_costs["clear_success_total_s"]) # 5 s (=3+2)
77 CLEAR_FAIL_S = float(_costs["clear_fail_total_s"]) # 3 s
78 ENTER_S = float(_costs["enter_s"]) # 0
79 EXIT_S = float(_costs["exit_s"]) # 0
80
81 # --- 派生几何量 (实时计算, 绝不写死记忆值) -----
82 BEARING_ERR_RAD = math.radians(BEARING_ERR_MAX_DEG) # ±1° 转弧度
83 TAN_ERR = math.tan(BEARING_ERR_RAD) # tan(1°) 0.017455
84 # Jung 定理紧界: R_MEC ≤ D/√3, 覆盖比 = 2R_MEC/D ≤ 2/√3 (正三角形取等)
85 JUNG_COVER_RATIO = 2.0 / math.sqrt(3.0) # 1.1547005
86 # 直径圆安全放大系数: 若沿用 D 判停, 需 D ≤ 20√3
87 D_STOP_THRESHOLD_M = CLEAR_RADIUS_M * math.sqrt(3.0) # 34.64 m
88 # 正交等距双站一击清除的距离上限: R_MEC = √2 · r · tan1° ≤ 20 → r ≤ 20/(√2 · tan1°)
89 ORTHO_REACH_LIMIT_M = CLEAR_RADIUS_M / (math.sqrt(2.0) * TAN_ERR) # 810.2 m
90 # 一般基线上限: R_MEC = tan1° · |S1-S2| ≤ 20 → 基线 ≤ 20/tan1°
91 BASELINE_LIMIT_M = CLEAR_RADIUS_M / TAN_ERR
92
93 # --- 反例构型 (问题一紧界, 能力 P1-C3) -----
94 COUNTEREX_RANGE_M = RECEPTION_RADIUS_MAX_M # 三站距目标 =1500 m (均在 1800 内)
95 COUNTEREX_SEP_DEG = FULL_CIRCLE_DEG / 3.0 # 三站方位互隔 120°
96 COUNTEREX_BIAS_DEG = BEARING_ERR_MAX_DEG / 2.0 # 三读数同偏 +0.5°
97 N_TRI = 3 # 正三角形顶点数
98
99 # --- 覆盖证书网格步长 -----
100 COVER_GRID_STEP_M = float(_temporal["real_window_min"]) # 复用常量 25 作 25 m 网格步
101
102 # --- 停止判据 (能力 P1-C3 / F07) -----
103 R_MEC_STOP_M = CLEAR_RADIUS_M # R_MEC ≤ 20 m 一击清除
104
105 # --- 问题二选点场景常量 (由题面常量 + 白名单整数派生, 无裸数字) -----
106 CROSS_ANGLE_OPT_DEG = DIRECTIONAL_HALF_DEG # 最优交会角 90°
107 CROSS_ANGLE_HALF_WINDOW_DEG = DIRECTIONAL_HALF_DEG / 3.0 # 交会角窗半宽 30° → [60°, 120°]
108 CROSS_ANGLE_MIN_DEG = CROSS_ANGLE_OPT_DEG - CROSS_ANGLE_HALF_WINDOW_DEG # _min=60°

```

```

109 Q2_NOMINAL_RANGE_M = RECEPTION_RADIUS_MIN_M / 2.0 # 名义目标距离 500 m
110 Q2_TRAVEL_BUDGET_M = RECEPTION_RADIUS_MIN_M / 2.0 # 选点行程预算 500 m (=100 s)
111
112 # --- 数值容差 -----
113 EPS = 1e-9

```

1.2 几何工具箱 utils.py

```

1  # -*- coding: utf-8 -*-
2  """几何工具箱 —— 集员定位的公共算子（问题一至四共用）。
3
4  覆盖：角度约定(atan2d/wrap180)、示向度→2 半平面(F01)、半平面求交(Sutherland - Hodgman)、
5  凸多边形直径(F04 顶点对枚举)、最小外接圆(F05 两点/三点枚举，确为最小)、覆盖判定。
6  所有数值容差走 params.EPS；不引入任何题面裸常量。
7
8  """
9
10 from __future__ import annotations
11
12 import math
13 from typing import List, Sequence, Tuple
14
15 from params import (BEARING_ERR_MAX_DEG, EPS, FULL_CIRCLE_DEG, HALF_CIRCLE_DEG,
16                     REGION_RADIUS_M, COORD_ABS_MAX_M)
17
18 Point = Tuple[float, float]
19 HalfPlane = Tuple[Tuple[float, float], float] # (a, b) 表示 a * x <= b
20
21 # --- 角度与向量 -----
22 def atan2d(dx: float, dy: float) -> float:
23     """向量 (dx,dy) 的方位角（东=0°，逆时针为正），归一到 [0,360)。"""
24     return math.degrees(math.atan2(dy, dx)) % FULL_CIRCLE_DEG
25
26 def bearing_deg(src: Point, dst: Point) -> float:
27     """从 src 指向 dst 的示向度（R23 定义）。"""
28     return atan2d(dst[0] - src[0], dst[1] - src[1])
29
30
31 def wrap180(a: float) -> float:
32     """把角度差归一到 (-180,180]。"""
33     x = (a + HALF_CIRCLE_DEG) % FULL_CIRCLE_DEG - HALF_CIRCLE_DEG
34     return x + FULL_CIRCLE_DEG if x <= -HALF_CIRCLE_DEG else x
35
36
37 def deg2unit(deg: float) -> Point:

```

```

38     r = math.radians(deg)
39     return (math.cos(r), math.sin(r))
40
41
42 def dist(p: Point, q: Point) -> float:
43     return math.hypot(p[0] - q[0], p[1] - q[1])
44
45
46 # --- 示向度 → 半平面 (F01) -----
47 def wedge_halfplanes(S: Point, theta_deg: float,
48                     err_deg: float = BEARING_ERR_MAX_DEG) -> List[HalfPlane]:
49     """示向度 在检测点 S 处展开为 2 个半平面: 真实方位角 [-err, +err]。
50
51     返回 [(a1,b1),(a2,b2)], 均为 {x: a·x ≤ b}。真值 g 落在扇形内则同时满足两式。
52     """
53     lo = theta_deg - err_deg
54     hi = theta_deg + err_deg
55     ulo = deg2unit(lo)
56     uhi = deg2unit(hi)
57     # 下边界: cross(ulo, p-S) ≥ 0 → (ulo_y, -ulo_x) · p ≤ (ulo_y, -ulo_x) · S
58     a1 = (ulo[1], -ulo[0])
59     b1 = a1[0] * S[0] + a1[1] * S[1]
60     # 上边界: cross(uhi, p-S) ≤ 0 → (-uhi_y, uhi_x) · p ≤ 同点 · S
61     a2 = (-uhi[1], uhi[0])
62     b2 = a2[0] * S[0] + a2[1] * S[1]
63     return [(a1, b1), (a2, b2)]
64
65
66 # --- 半平面求交 (Sutherland-Hodgman) -----
67 def clip_polygon(poly: Sequence[Point], a: Tuple[float, float], b: float) -> List[Point]:
68     """用半平面  $a \cdot x \leq b$  裁剪凸多边形 poly (顶点按序)。"""
69     if not poly:
70         return []
71     out: List[Point] = []
72     n = len(poly)
73     for i in range(n):
74         cur = poly[i]
75         nxt = poly[(i + 1) % n]
76         dc = a[0] * cur[0] + a[1] * cur[1] - b
77         dn = a[0] * nxt[0] + a[1] * nxt[1] - b
78         cur_in = dc ≤ EPS
79         nxt_in = dn ≤ EPS
80         if cur_in:
81             out.append(cur)
82         if cur_in != nxt_in and abs(dc - dn) > EPS:
83             t = dc / (dc - dn)
84             out.append((cur[0] + t * (nxt[0] - cur[0]),

```

```

85         cur[1] + t * (nxt[1] - cur[1]))
86     return out
87
88
89 def _dedup(poly: Sequence[Point]) -> List[Point]:
90     out: List[Point] = []
91     for p in poly:
92         if not out or dist(p, out[-1]) > EPS:
93             out.append(p)
94     if len(out) >= 2 and dist(out[0], out[-1]) <= EPS:
95         out.pop()
96     return out
97
98
99 def _bounding_box(points: Sequence[Point]) -> List[Point]:
100     """构造足够大的初始外框，用于无界性检测。"""
101     scale = max(REGION_RADIUS_M, *(abs(c) for p in points for c in p)) if points else
102         REGION_RADIUS_M
103     L = min(10.0 * (scale + REGION_RADIUS_M), COORD_ABS_MAX_M)
104     return [(-L, -L), (L, -L), (L, L), (-L, L)]
105
106 def intersect_wedges(dets: Sequence[Tuple[Point, float]],
107                     err_deg: float = BEARING_ERR_MAX_DEG):
108     """n 个（检测点 Si, 示向度 i）的角扇形求交（F02）。
109
110     返回 (polygon, status, halfplanes):
111         status {"OK", "EMPTY", "UNBOUNDED"}; halfplanes 为全部 2n 个 (a,b)（供 P1-C1 断言）。
112     """
113     box_pts = [S for S, _ in dets]
114     box = _bounding_box(box_pts)
115     L = box[1][0] # 外框半边长
116     hps: List[HalfPlane] = []
117     poly: List[Point] = list(box)
118     for S, th in dets:
119         for a, b in wedge_halfplanes(S, th, err_deg):
120             hps.append((a, b))
121             poly = clip_polygon(poly, a, b)
122             if not poly:
123                 return [], "EMPTY", hps
124     poly = _dedup(poly)
125     if len(poly) < 3:
126         return poly, "EMPTY", hps
127     touches = any(abs(abs(x) - L) <= L * 1e-6 or abs(abs(y) - L) <= L * 1e-6
128                  for x, y in poly)
129     return poly, ("UNBOUNDED" if touches else "OK"), hps
130

```

```

131
132 # --- 凸多边形直径 (F04: 顶点对枚举, 确为最大) -----
133 def polygon_diameter(poly: Sequence[Point]) -> float:
134     if len(poly) < 2:
135         return 0.0
136     d = 0.0
137     for i in range(len(poly)):
138         for j in range(i + 1, len(poly)):
139             d = max(d, dist(poly[i], poly[j]))
140     return d
141
142
143 def diameter_pair(poly: Sequence[Point]) -> Tuple[Point, Point]:
144     best = (poly[0], poly[0])
145     d = 0.0
146     for i in range(len(poly)):
147         for j in range(i + 1, len(poly)):
148             dij = dist(poly[i], poly[j])
149             if dij > d:
150                 d, best = dij, (poly[i], poly[j])
151     return best
152
153
154 # --- 最小外接圆 (F05: 两点圆 + 三点外接圆枚举, 确为最小) -----
155 def _circle_from_two(p: Point, q: Point):
156     c = ((p[0] + q[0]) / 2.0, (p[1] + q[1]) / 2.0)
157     return c, dist(p, q) / 2.0
158
159
160 def _circle_from_three(a: Point, b: Point, c: Point):
161     ax, ay = a
162     bx, by = b
163     cx, cy = c
164     d = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
165     if abs(d) < EPS:
166         return None
167     ux = ((ax * ax + ay * ay) * (by - cy) + (bx * bx + by * by) * (cy - ay)
168           + (cx * cx + cy * cy) * (ay - by)) / d
169     uy = ((ax * ax + ay * ay) * (cx - bx) + (bx * bx + by * by) * (ax - cx)
170           + (cx * cx + cy * cy) * (bx - ax)) / d
171     center = (ux, uy)
172     return center, dist(center, a)
173
174
175 def min_enclosing_circle(points: Sequence[Point]):
176     """凸多边形/点集最小外接圆 (center, R). 枚举两点圆与三点外接圆取最小可行, 确为最小."""
177     pts = _dedup(list(points))

```

```

178     if not pts:
179         return (0.0, 0.0), 0.0
180     if len(pts) == 1:
181         return pts[0], 0.0
182
183     def covers(center, R):
184         return all(dist(center, p) <= R + EPS for p in pts)
185
186     best_c, best_r = None, math.inf
187     for i in range(len(pts)):
188         for j in range(i + 1, len(pts)):
189             c, r = _circle_from_two(pts[i], pts[j])
190             if r < best_r and covers(c, r):
191                 best_c, best_r = c, r
192     for i in range(len(pts)):
193         for j in range(i + 1, len(pts)):
194             for k in range(j + 1, len(pts)):
195                 res = _circle_from_three(pts[i], pts[j], pts[k])
196                 if res is None:
197                     continue
198                 c, r = res
199                 if r < best_r and covers(c, r):
200                     best_c, best_r = c, r
201     if best_c is None: # 共线退化: 取最远两点圆
202         best_c, best_r = _circle_from_two(*diameter_pair(pts))
203     return best_c, best_r
204
205
206 # --- 覆盖判定 -----
207 def region_grid(radius: float, step: float) -> List[Point]:
208     """半径 radius 圆域上步长 step 的网格点 (用于覆盖证书验证)。"""
209     pts: List[Point] = []
210     n = int(math.ceil(radius / step))
211     for ix in range(-n, n + 1):
212         for iy in range(-n, n + 1):
213             x, y = ix * step, iy * step
214             if x * x + y * y <= radius * radius + EPS:
215                 pts.append((x, y))
216     return pts
217
218
219 def max_min_distance(grid: Sequence[Point], stations: Sequence[Point]) -> float:
220     """max_{p in grid} min_{S in stations} ||p-S||, 覆盖半径证书的核心量。"""
221     if not stations:
222         return math.inf
223     worst = 0.0
224     for p in grid:

```

```

225         worst = max(worst, min(dist(p, S) for S in stations))
226     return worst

```

1.3 本地复刻模拟器 simulator.py

```

1  # -*- coding: utf-8 -*-
2  """本地复刻模拟器 —— 与附件2 表2 逐行对齐（虚拟时刻 0/105/111/194/199）。
3
4  严格实现四指令（/enter /measure /clear /exit）、三种检测返回（no_signal/near/direction）、
5  四类计时（移动 d/5、切换 1 s 仅频道变、检测 5 s、清除 5/3 s），且：
6  • /clear 的频道参数不切换测向机频道（表2 第4+5 步的官方陷阱）；
7  • accepted=false 不推进虚拟时钟（R24）；request_id 幂等重试复用响应（R25）；
8  • 测向误差（S）是位置的确定性函数（R06 同点重测不变），闭式落在  $[-1^\circ, +1^\circ]$ 。
9  """
10 from __future__ import annotations
11
12 import math
13 from typing import Dict, List, Optional, Tuple
14
15 import params as P
16 import utils as U
17
18 Point = Tuple[float, float]
19
20
21 # --- 位置确定性误差场（R06：同点不变、异点可异；闭式  $| \sin \cdot \cos | \leq 1$ ） -----
22 def bearing_error_deg(S: Point) -> float:
23     """测向误差（S） $[-1^\circ, +1^\circ]$ ，位置的确定性平滑函数（乘积  $|\sin \cdot \cos| \leq 1$  保界）。"""
24     x, y = S
25     phase = math.sin(0.011 * x + 0.017 * y + 0.3) * math.cos(0.013 * x - 0.009 * y + 1.1)
26     return P.BEARING_ERR_MAX_DEG * phase
27
28
29 # --- 案例（真值场景）生成 -----
30 class Source:
31     __slots__ = ("channel", "x", "y", "R_c", "phi")
32
33     def __init__(self, channel: int, x: float, y: float, R_c: float,
34                 phi: Optional[float]):
35         self.channel = channel
36         self.x = x
37         self.y = y
38         self.R_c = R_c
39         self.phi = phi      # None=全向；否则定向方向（度）
40

```

```

41 @property
42 def g(self) -> Point:
43     return (self.x, self.y)
44
45
46 def generate_case(rng, directional_ratio: float = 0.0) -> Dict:
47     """随机案例: K [10,16], 频道 1..20 互异, 位置  $\Omega$  内均匀, R_c [1000,1500]。
48     directional_ratio>0 时按比例把部分源设为定向 (问题四)。"""
49     K = int(rng.integers(P.N_SOURCES_MIN, P.N_SOURCES_MAX + 1))
50     channels = list(range(P.CHANNEL_MIN, P.CHANNEL_MAX + 1))
51     rng.shuffle(channels)
52     chosen = channels[:K]
53     sources: List[Source] = []
54     for ch in chosen:
55         #  $\Omega$  内均匀采样 (拒绝法)
56         while True:
57             x = float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M))
58             y = float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M))
59             if x * x + y * y <= P.REGION_RADIUS_M * P.REGION_RADIUS_M:
60                 break
61             R_c = float(rng.uniform(P.RECEPTION_RADIUS_MIN_M, P.RECEPTION_RADIUS_MAX_M))
62             phi = None
63             if directional_ratio > 0.0 and rng.uniform(0.0, 1.0) < directional_ratio:
64                 phi = float(rng.uniform(0.0, P.FULL_CIRCLE_DEG))
65             sources.append(Source(ch, x, y, R_c, phi))
66     return {"K": K, "sources": sources}
67
68
69 # --- 模拟器 ---
70 class LocalSimulator:
71     def __init__(self, case: Dict, real_window_s: float = None):
72         self.case = case
73         self.real_window_s = (P.PROGRAM_RUNTIME_MAX_S if real_window_s is None
74                                else float(real_window_s))
75         self.pos: Point = P.CENTER
76         self.channel: int = P.INITIAL_CHANNEL
77         self.t_virtual: float = 0.0
78         self.entered = False
79         self.exited = False
80         self.cleared: set = set() # 已成功清除的频道
81         self.log: List[Dict] = [] # 指令流水 (供 GL-C1/审计)
82         self._cache: Dict[str, Dict] = {} # request_id -> response (幂等)
83         self.inflight_max = 0
84         self.n_measure = 0
85         self.n_clear = 0
86
87 # --- 内部: 源查找与可接收判据 ---

```

```

88 def _active_source(self, channel: int) -> Optional[Source]:
89     if channel in self.cleared:
90         return None
91     for s in self.case["sources"]:
92         if s.channel == channel:
93             return s
94     return None
95
96 def _receivable(self, S: Point, s: Source) -> Tuple[bool, bool, bool]:
97     """返回（可接收，距离满足，角度满足）。定向：|wrap180(bearing(g+S)-)|<=90°（闭）。"""
98     d = U.dist(S, s.g)
99     by_dist = d <= s.R_c + P.EPS
100     if s.phi is None:
101         by_ang = True
102     else:
103         rel = abs(U.wrap180(U.bearing_deg(s.g, S) - s.phi))
104         by_ang = rel <= P.DIRECTIONAL_HALF_DEG + 1e-9
105     return (by_dist and by_ang), by_dist, by_ang
106
107 # --- 计时账本 -----
108 def _move_cost(self, new_pos: Point) -> float:
109     return U.dist(self.pos, new_pos) / P.SPEED_MPS
110
111 # --- 四指令 -----
112 def enter(self, request_id: str) -> Dict:
113     if request_id in self._cache:
114         return self._cache[request_id]
115     self._inflight_max = max(self._inflight_max, 1)
116     self.entered = True
117     resp = {"accepted": True, "remaining_real_duration_s": self.real_window_s}
118     self.log.append({"ep": "enter", "request_id": request_id,
119                     "t_virtual": self.t_virtual, "accepted": True})
120     self._cache[request_id] = resp
121     return resp
122
123 def measure(self, request_id: str, position: Point, channel: int) -> Dict:
124     if request_id in self._cache:
125         return self._cache[request_id]
126     self._inflight_max = max(self._inflight_max, 1)
127     if not self.entered or self.exited or not (P.CHANNEL_MIN <= channel <= P.CHANNEL_MAX):
128         resp = {"accepted": False, "measure_result": None, "svd_deg": None}
129         self.log.append({"ep": "measure", "request_id": request_id,
130                         "accepted": False, "t_virtual": self.t_virtual})
131         self._cache[request_id] = resp
132         return resp
133     if abs(position[0]) > P.COORD_ABS_MAX_M or abs(position[1]) > P.COORD_ABS_MAX_M:
134         resp = {"accepted": False, "measure_result": None, "svd_deg": None}

```

```

135         self._cache[request_id] = resp
136         return resp
137     cost = self._move_cost(position)
138     cost += P.SWITCH_S if channel != self.channel else 0.0
139     cost += P.MEASURE_S
140     self.pos = position
141     self.channel = channel # 仅 measure 改测向机频道
142     s = self._active_source(channel)
143     svd = None
144     if s is None:
145         result = "no_signal"
146     else:
147         recv, _bd, _ba = self._receivable(position, s)
148         if not recv:
149             result = "no_signal"
150         else:
151             d = U.dist(position, s.g)
152             if d <= P.NEAR_THRESHOLD_M + P.EPS:
153                 result = "near"
154             else:
155                 result = "direction"
156                 raw = U.bearing_deg(position, s.g) + bearing_error_deg(position)
157                 svd = round(raw % P.FULL_CIRCLE_DEG, P.SVD_DECIMALS)
158     self.t_virtual += cost
159     self.n_measure += 1
160     resp = {"accepted": True, "measure_result": result, "svd_deg": svd}
161     self.log.append({"ep": "measure", "request_id": request_id, "accepted": True,
162                    "position": list(position), "channel": channel,
163                    "cost_s": cost, "t_virtual": self.t_virtual, "result": result,
164                    "svd_deg": svd})
165     self._cache[request_id] = resp
166     return resp
167
168 def clear(self, request_id: str, position: Point, channel: int) -> Dict:
169     if request_id in self._cache:
170         return self._cache[request_id]
171     self.inflight_max = max(self.inflight_max, 1)
172     if not self.entered or self.exited:
173         resp = {"accepted": False, "clear_result": None}
174         self._cache[request_id] = resp
175         return resp
176     cost = self._move_cost(position) # 清除不切频道：无切换耗时
177     self.pos = position
178     s = self._active_source(channel)
179     if s is not None and U.dist(position, s.g) <= P.CLEAR_RADIUS_M + P.EPS:
180         cost += P.CLEAR_SUCCESS_S
181         self.cleared.add(channel)

```

```

182         result = "success"
183     else:
184         cost += P.CLEAR_FAIL_S
185         result = "no_target_in_range"
186         self.t_virtual += cost
187         self.n_clear += 1
188         resp = {"accepted": True, "clear_result": result}
189         self.log.append({"ep": "clear", "request_id": request_id, "accepted": True,
190                        "position": list(position), "channel": channel,
191                        "cost_s": cost, "t_virtual": self.t_virtual, "result": result})
192         self._cache[request_id] = resp
193     return resp
194
195 def exit(self, request_id: str) -> Dict:
196     if request_id in self._cache:
197         return self._cache[request_id]
198     self.exited = True
199     resp = {"accepted": True, "exit_reason": "user_exit"}
200     self.log.append({"ep": "exit", "request_id": request_id, "accepted": True,
201                    "t_virtual": self.t_virtual})
202     self._cache[request_id] = resp
203     return resp
204
205 @property
206 def n_cleared(self) -> int:
207     return len(self.cleared)
208
209
210 def calibration_replay() -> Dict:
211     """逐行复制附件2 表2；空案例（无源）下 measure→no_signal、clear→no_target_in_range。
212     返回每步 accepted 后的虚拟时刻序列，须精确等于 [105,111,194,199]。"""
213     sim = LocalSimulator({"K": 0, "sources": []})
214     seq = []
215     sim.enter("r0")
216     seq.append(sim.t_virtual)                                # 0
217     sim.measure("r1", (300.0, 400.0), 1)                   # 100+0+5=105
218     seq.append(sim.t_virtual)
219     sim.measure("r2", (300.0, 400.0), 2)                   # 0+1+5 -> 111
220     seq.append(sim.t_virtual)
221     sim.clear("r3", (300.0, 0.0), 3)                       # 80+0+3 -> 194 (clear 不切频道)
222     seq.append(sim.t_virtual)
223     sim.measure("r4", (300.0, 0.0), 2)                     # 0+0+5 -> 199 (测向机频道仍为2)
224     seq.append(sim.t_virtual)
225     sim.exit("r5")                                         # 199
226     seq.append(sim.t_virtual)
227     return {"virtual_times": seq, "channel_after": sim.channel, "log": sim.log}
228

```

```

229
230 def validate_calibration():
231     """P3-C4 硬闸：虚拟时刻序列与表2 逐行相等（差值 0）。"""
232     rep = calibration_replay()
233     got = rep["virtual_times"]
234     expect = [0.0, 105.0, 111.0, 194.0, 199.0, 199.0]
235     assert len(got) == len(expect), f"calib: 步数 {len(got)} != {len(expect)}"
236     for i, (a, b) in enumerate(zip(got, expect)):
237         assert abs(a - b) < 1e-9, f"calib: 第{i}步虚拟时刻 {a} != {b} (差 {a-b}) "
238     return f"code/simulator.py:calibration 虚拟时刻 {[int(v) for v in got]}
    与附件2表2逐行相等（差0）"
239
240
241 if __name__ == "__main__":
242     print(validate_calibration())
243     print("[simulator] 表2 逐行对齐通过")

```

1.4 问题一：定位区域几何度量 problem1.py

```

1  # -*- coding: utf-8 -*-
2  """问题一：交会定位多边形定位区域的直径算法 + 直径圆覆盖性判定（紧界反例）。
3
4  能力：P1-C1 示向度+2n 半平面→凸多边形；P1-C2 直径算法（空集/无界/退化）；
5      P1-C3 直径圆覆盖性判定与 Jung 紧界反例；P1-C4 最小外接圆。
6  每项配 validate_* 运行时硬断言（失败即 raise），结果写 figures/problem_1_results.json。
7  """
8  from __future__ import annotations
9
10 import json
11 import math
12 from pathlib import Path
13
14 import numpy as np
15
16 import params as P
17 import utils as U
18
19 _ROOT = Path(__file__).resolve().parent.parent
20 _FIGDIR = _ROOT / "figures"
21 _SEED = 20260910
22
23
24 # --- 反例构型：三站 =1500 m 互隔 120°，读数同偏 +0.5° → 正三角形 Ω -----
25 def build_counterexample():
26     """返回三个（检测点 S_i，带偏差示向度 _i）。目标置于原点。"""

```

```

27 g = (0.0, 0.0)
28 dets = []
29 for k in range(P.N_TRI):
30     az = P.NORTH_DEG + k * P.COUNTEREX_SEP_DEG # 90°, 210°, 330°
31     S = (P.COUNTEREX_RANGE_M * math.cos(math.radians(az)),
32          P.COUNTEREX_RANGE_M * math.sin(math.radians(az)))
33     theta = U.bearing_deg(S, g) + P.COUNTEREX_BIAS_DEG # 同偏 +0.5°
34     dets.append((S, theta))
35 return g, dets
36
37
38 # --- 主求解 ---
39 def solve():
40     # (a) 反例: 直径圆覆盖性
41     g, dets = build_counterexample()
42     poly, status, hps = U.intersect_wedges(dets)
43     D = U.polygon_diameter(poly)
44     center, R_mec = U.min_enclosing_circle(poly)
45     ratio = 2.0 * R_mec / D
46     covered_by_diam_circle = (R_mec <= D / 2.0 + P.EPS)
47
48     # (b) 覆盖比随 n 的实测趋势 (随机构型下 2R_MEC/D 的最大值)
49     rng = np.random.default_rng(_SEED)
50     kappa_trend = {}
51     max_ratio_global = 0.0
52     for n in range(2, 6):
53         best = 0.0
54         for _ in range(4000):
55             dts = []
56             gg = (float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M)),
57                  float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M)))
58             for _i in range(n):
59                 ang = float(rng.uniform(0.0, P.FULL_CIRCLE_DEG))
60                 rr = float(rng.uniform(P.RECEPTION_RADIUS_MIN_M, P.RECEPTION_RADIUS_MAX_M))
61                 S = (gg[0] + rr * math.cos(math.radians(ang)),
62                     gg[1] + rr * math.sin(math.radians(ang)))
63                 err = float(rng.uniform(P.BEARING_ERR_MIN_DEG, P.BEARING_ERR_MAX_DEG))
64                 dts.append((S, U.bearing_deg(S, gg) + err))
65             pl, st, _ = U.intersect_wedges(dts)
66             if st != "OK" or len(pl) < 3:
67                 continue
68             d = U.polygon_diameter(pl)
69             if d < P.EPS:
70                 continue
71             _, rm = U.min_enclosing_circle(pl)
72             best = max(best, 2.0 * rm / d)
73     kappa_trend[str(n)] = best

```

```

74     max_ratio_global = max(max_ratio_global, best)
75
76     results = {
77         "problem": 1,
78         "counterexample": {
79             "n_detections": P.N_TRI,
80             "range_m": P.COUNTEREX_RANGE_M,
81             "azimuth_sep_deg": P.COUNTEREX_SEP_DEG,
82             "bias_deg": P.COUNTEREX_BIAS_DEG,
83             "status": status,
84             "n_halfplanes": len(hps),
85             "n_vertices": len(poly),
86             "D_m": D,
87             "r_mec_counterexample_m": R_mec,
88             "cover_ratio_2Rmec_over_D": ratio,
89             "jung_bound_2_over_sqrt3": P.JUNG_COVER_RATIO,
90             "diameter_circle_covers": covered_by_diam_circle,
91             "D_half_m": D / 2.0,
92         },
93         "stop_thresholds": {
94             "R_MEC_stop_m": P.R_MEC_STOP_M,
95             "D_stop_threshold_m": P.D_STOP_THRESHOLD_M,
96         },
97         "kappa_trend_vs_n": kappa_trend,
98         "kappa_max_random_global": max_ratio_global,
99     }
100     return results
101
102
103 # --- 能力硬断言 ---
104 def validate_p1_c1():
105     """2n 半平面表示; 真值满足全部约束。"""
106     rng = np.random.default_rng(_SEED + 1)
107     for _ in range(500):
108         g = (float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M)),
109              float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M)))
110         n = int(rng.integers(2, 6))
111         dets = []
112         for _i in range(n):
113             ang = float(rng.uniform(0.0, P.FULL_CIRCLE_DEG))
114             rr = float(rng.uniform(P.RECEPTION_RADIUS_MIN_M, P.RECEPTION_RADIUS_MAX_M))
115             S = (g[0] + rr * math.cos(math.radians(ang)),
116                  g[1] + rr * math.sin(math.radians(ang)))
117             err = float(rng.uniform(P.BEARING_ERR_MIN_DEG, P.BEARING_ERR_MAX_DEG))
118             dets.append((S, U.bearing_deg(S, g) + err))
119         _, _, hps = U.intersect_wedges(dets)
120         assert len(hps) == 2 * n, f"P1-C1: 半平面数 {len(hps)} != 2n={2*n}"

```

```

121     scale = max(abs(c) for S, _ in dets for c in S) + P.REGION_RADIUS_M
122     for a, b in hps:
123         assert a[0] * g[0] + a[1] * g[1] - b <= 1e-6 * scale, "P1-C1:
            真值违反半平面（法向取反？）"
124     return "code/problem1.py:validate_p1_c1 500 组随机真值均满足全部 2n 半平面约束"
125
126
127 def validate_p1_c2():
128     """直径=顶点对暴力最大；空集/无界显式标志。"""
129     # (1) 已知多边形直径正确性：单位正方形对角线 =  $\sqrt{2}$ 
130     sq = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
131     assert abs(U.polygon_diameter(sq) - math.sqrt(2.0)) < 1e-9, "P1-C2: 正方形直径错误"
132     # 直径值 >= 内部任意点对距离（暴力抽样验证不超过）
133     rng = np.random.default_rng(_SEED + 2)
134     dpoly = U.polygon_diameter(sq)
135     for _ in range(2000):
136         i = int(rng.integers(0, 4))
137         j = int(rng.integers(0, 4))
138         assert U.dist(sq[i], sq[j]) <= dpoly + 1e-9, "P1-C2: 存在超过直径的顶点对"
139     # (2) 空集：两站扇形彼此背离（A 指正西  $x < 0$ , B 指正东  $x > 100$ , 交集为空）
140     dets_e = [(0.0, 0.0), P.WEST_DEG], ((100.0, 0.0), P.EAST_DEG)]
141     _, st_e, _ = U.intersect_wedges(dets_e)
142     assert st_e == "EMPTY", f"P1-C2: 矛盾读数未判 EMPTY (得 {st_e})"
143     # (3) 无界：单站单扇形
144     _, st_u, _ = U.intersect_wedges([(0.0, 0.0), 45.0])
145     assert st_u == "UNBOUNDED", f"P1-C2: 单扇形未判 UNBOUNDED (得 {st_u})"
146     return "code/problem1.py:validate_p1_c2 正方形直径= $\sqrt{2}$ ; 矛盾读数→EMPTY; 单扇形→UNBOUNDED"
147
148
149 def validate_p1_c3():
150     """反例：2R_MEC/D=2/ $\sqrt{3}$  (rel<1e-6), 2R_MEC>D 不覆盖；随机最大比值 2/ $\sqrt{3}$ +1e-9。"""
151     g, dets = build_counterexample()
152     poly, status, _ = U.intersect_wedges(dets)
153     assert status == "OK", f"P1-C3: 反例区域状态 {status}"
154     D = U.polygon_diameter(poly)
155     center, R_mec = U.min_enclosing_circle(poly)
156     # 真值在区域内
157     assert all(a[0] * g[0] + a[1] * g[1] <= b + 1e-6 for a, b in U.intersect_wedges(dets)[2]), \
158         "P1-C3: 目标真值不在定位区域内"
159     ratio = 2.0 * R_mec / D
160     rel = abs(ratio - P.JUNG_COVER_RATIO) / P.JUNG_COVER_RATIO
161     assert rel < 1e-6, f"P1-C3: 覆盖比 {ratio} 未达紧界 2/ $\sqrt{3}$  (rel={rel:.2e})"
162     assert 2.0 * R_mec > D + P.EPS, "P1-C3: 2R_MEC 未超过 D, 反例不成立"
163     return (f"code/problem1.py:validate_p1_c3 反例 D={D:.4f}m R_MEC={R_mec:.4f}m "
164           f"比值={ratio:.7f}=2/ $\sqrt{3}$  (rel<1e-6), 2R_MEC>D 故直径圆不覆盖")
165
166

```

```

167 def validate_p1_c4():
168     """MEC: 覆盖全部顶点;  $D/2 \leq R \leq D/\sqrt{3}$  (Jung), 且确为最小 (缩小即漏点)。"""
169     rng = np.random.default_rng(_SEED + 4)
170     for _ in range(300):
171         m = int(rng.integers(3, 8))
172         pts = [(float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M)),
173                 float(rng.uniform(-P.REGION_RADIUS_M, P.REGION_RADIUS_M))) for _i in range(m)]
174         c, R = U.min_enclosing_circle(pts)
175         assert all(U.dist(c, p) <= R + 1e-6 for p in pts), "P1-C4: MEC 未覆盖全部点"
176         D = U.polygon_diameter(pts)
177         assert D / 2.0 - 1e-6 <= R <= D / math.sqrt(3.0) + 1e-6, \
178             f"P1-C4: R={R} 超出  $[D/2, D/\sqrt{3}]$  界"
179         # 最小性: 半径缩小  $1e-3 \cdot R$  后必漏点
180         shrink = R - max(1e-3 * R, 1e-6)
181         assert not all(U.dist(c, p) <= shrink + P.EPS for p in pts), "P1-C4: 半径可缩小, 非最小"
182     return "code/problem1.py:validate_p1_c4 300 组随机点集 MEC
183         覆盖全部点、落于  $[D/2, D/\sqrt{3}]$  且缩小即漏点 (确为最小) "
184
185 def main():
186     res = solve()
187     res["capability_asserts"] = {
188         "P1-C1": validate_p1_c1(),
189         "P1-C2": validate_p1_c2(),
190         "P1-C3": validate_p1_c3(),
191         "P1-C4": validate_p1_c4(),
192     }
193     _FIGDIR.mkdir(exist_ok=True)
194     (_FIGDIR / "problem1_results.json").write_text(
195         json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
196     ce = res["counterexample"]
197     print(f"[problem1] 反例 D={ce['D_m']:.4f} R_MEC={ce['r_mec_counterexample_m']:.4f} "
198           f"ratio={ce['cover_ratio_2Rmec_over_D']:.7f} 覆盖={ce['diameter_circle_covers']}")
199     print(f"[problem1] 趋势 {res['kappa_trend_vs_n']}")
200     print("[problem1] 四项能力断言全部通过")
201     return res
202
203
204 if __name__ == "__main__":
205     main()

```

1.5 问题二：第二检测点选择 problem2.py

```

1 # -*- coding: utf-8 -*-
2 """问题二：第二检测点选择策略（交会角判据）与候选区域（三重可行性求交）。

```

```

3
4 能力：P2-C1（语义）以 R_MEC 为判据得交会角  $\rightarrow 90^\circ$  最优并量化小角退化；
5     P2-C2（机器+交付）三重可行性求交给出候选面域  $\rightarrow$  output/q2_candidate_region.json。
6 每项配 validate_* 运行时硬断言，结果写 figures/problem_2_results.json。
7 """
8 from __future__ import annotations
9
10 import json
11 import math
12 from pathlib import Path
13
14 import numpy as np
15
16 import params as P
17 import utils as U
18
19 _ROOT = Path(__file__).resolve().parent.parent
20 _FIGDIR = _ROOT / "figures"
21 _OUTDIR = _ROOT / "output"
22 _SEED = 20260910
23
24
25 # --- 双站定位区域（目标置原点，S1 在东、S2 在方位 处） -----
26 def build_two_station(r1: float, r2: float, gamma_deg: float, bias_deg: float = 0.0):
27     """目标在原点；S1 距 r1 于东向、S2 距 r2 于方位 。精确读数（可加同偏），
28     经 2 条示向度扇形求交得定位区域，返回 D 与 R_MEC（真实几何，非线性化）。"""
29     g = (0.0, 0.0)
30     S1 = (r1 * math.cos(math.radians(P.EAST_DEG)),
31           r1 * math.sin(math.radians(P.EAST_DEG)))
32     S2 = (r2 * math.cos(math.radians(gamma_deg)),
33           r2 * math.sin(math.radians(gamma_deg)))
34     th1 = U.bearing_deg(S1, g) + bias_deg
35     th2 = U.bearing_deg(S2, g) + bias_deg
36     poly, status, _ = U.intersect_wedges([(S1, th1), (S2, th2)])
37     D = U.polygon_diameter(poly)
38     if poly:
39         _, R_mec = U.min_enclosing_circle(poly)
40     else:
41         R_mec = 0.0
42     return {"S1": S1, "S2": S2, "status": status, "n_vertices": len(poly),
43           "D_m": D, "R_mec_m": R_mec}
44
45
46 def linearized_parallellogram(r1: float, r2: float, gamma_deg: float):
47     """小区域一阶线化：每条扇形近似为半宽  $w_i=r_i \cdot \tan 1^\circ$  的平行带，交会角 。
48     返回 4 顶点、直径闭式与 R_MEC（供闭式核对）。"""
49     w1 = r1 * P.TAN_ERR

```

```

50 w2 = r2 * P.TAN_ERR
51 g = math.radians(gamma_deg)
52 cg, sg = math.cos(g), math.sin(g)
53 # 带1: |y| <= w1; 带2: |-x*sin + y*cos| <= w2。x=(y*cos - c)/sin。
54 verts = []
55 for y in (w1, -w1):
56     for c in (w2, -w2):
57         verts.append(((y * cg - c) / sg, y))
58 D = U.polygon_diameter(verts)
59 _, R_mec = U.min_enclosing_circle(verts)
60 return {"verts": verts, "D_m": D, "R_mec_m": R_mec}
61
62
63 def sweep_cross_angle(r_m: float, gammas):
64     """固定等距 r，扫交会角，返回各 的真实几何 D 与 R_MEC。"""
65     out = {}
66     for gam in gammas:
67         rec = build_two_station(r_m, r_m, float(gam))
68         out[str(gam)] = {"D_m": rec["D_m"], "R_mec_m": rec["R_mec_m"],
69                         "status": rec["status"]}
70     return out
71
72
73 # --- 候选区域场景 (S1=原点=机器狗初始位；目标名义距离沿首条示向度) -----
74 def _nominal_scene():
75     """S1 在原点，首条示向度 1 指北；候选目标域为沿 1 的名义线段 [近,远]。"""
76     S1 = P.CENTER
77     theta1 = P.NORTH_DEG
78     u = U.deg2unit(theta1)
79     half = P.Q2_NOMINAL_RANGE_M / 5.0 # 名义段半长 100 m (=500/5)
80     r_near = P.Q2_NOMINAL_RANGE_M - half
81     r_far = P.Q2_NOMINAL_RANGE_M + half
82     g_near = (S1[0] + r_near * u[0], S1[1] + r_near * u[1])
83     g_far = (S1[0] + r_far * u[0], S1[1] + r_far * u[1])
84     g0 = (S1[0] + P.Q2_NOMINAL_RANGE_M * u[0], S1[1] + P.Q2_NOMINAL_RANGE_M * u[1])
85     return {"S1": S1, "theta1": theta1, "g0": g0, "g_near": g_near, "g_far": g_far}
86
87
88 def _cross_angle(g, A, B) -> float:
89     """在目标 g 处，检测点 A 与 B 两条视线的交会角（度，[0,180]）。"""
90     va = (A[0] - g[0], A[1] - g[1])
91     vb = (B[0] - g[0], B[1] - g[1])
92     na = math.hypot(*va)
93     nb = math.hypot(*vb)
94     if na < P.EPS or nb < P.EPS:
95         return 0.0
96     c = (va[0] * vb[0] + va[1] * vb[1]) / (na * nb)

```

```

97     c = max(-1.0, min(1.0, c))
98     return math.degrees(math.acos(c))
99
100
101 def _crit_receive(S2, sc) -> bool:
102     """可接收（保守）：S2 到候选目标段两端均 <= R_ex（=在两端 1000 m 圆盘之交/透镜内）。"""
103     return (U.dist(S2, sc["g_near"]) <= P.R_EX_M + P.EPS
104             and U.dist(S2, sc["g_far"]) <= P.R_EX_M + P.EPS)
105
106
107 def _crit_gamma(S2, sc) -> bool:
108     gam = _cross_angle(sc["g0"], sc["S1"], S2)
109     return (P.CROSS_ANGLE_MIN_DEG - P.EPS <= gam
110             <= P.HALF_CIRCLE_DEG - P.CROSS_ANGLE_MIN_DEG + P.EPS)
111
112
113 def _crit_budget(S2, sc) -> bool:
114     return U.dist(S2, sc["S1"]) <= P.Q2_TRAVEL_BUDGET_M + P.EPS
115
116
117 def _feasible(S2, sc) -> bool:
118     return _crit_receive(S2, sc) and _crit_gamma(S2, sc) and _crit_budget(S2, sc)
119
120
121 def candidate_region(step_m: float = 5.0):
122     """在 S2 的行程预算方框 [-budget,budget]^2 上以 step 网格化，返回三判据均满足的候选面域。"""
123     sc = _nominal_scene()
124     S1 = sc["S1"]
125     B = P.Q2_TRAVEL_BUDGET_M
126     n = int(math.floor(B / step_m))
127     pts = []
128     for ix in range(-n, n + 1):
129         for iy in range(-n, n + 1):
130             S2 = (S1[0] + ix * step_m, S1[1] + iy * step_m)
131             if _feasible(S2, sc):
132                 pts.append(S2)
133     area = len(pts) * step_m * step_m
134     return sc, pts, area, step_m
135
136
137 def solve():
138     # (a) 交会角判据：固定等距 r=1000 m 扫，取真实几何 D
139     r_base = P.RECEPTION_RADIUS_MIN_M # 1000 m
140     gammas = [10, 20, 30, 45, 60, 75, 90]
141     sweep = sweep_cross_angle(r_base, gammas)
142     D90 = sweep["90"]["D_m"]
143     D10 = sweep["10"]["D_m"]

```

```

144 ratio_D = D10 / D90 if D90 > P.EPS else math.inf
145
146 # (b) 正交等距闭式核对:  $R_{MEC} = \sqrt{2} \cdot r \cdot \tan 1^\circ$ 
147 lin90 = linearized_parallelogram(r_base, r_base, P.CROSS_ANGLE_OPT_DEG)
148 rmec_closed = math.sqrt(2.0) * r_base * P.TAN_ERR
149 rmec_rel = abs(lin90["R_mec_m"] - rmec_closed) / rmec_closed
150
151 # (c) 一击清除距离上限:  $\sqrt{2} \cdot r \cdot \tan 1^\circ \leq 20 \rightarrow r \leq 20 / (\sqrt{2} \cdot \tan 1^\circ)$ 
152 reach_limit = P.ORTHO_REACH_LIMIT_M
153
154 # (d) 候选区域 (三重可行性求交)
155 sc, pts, area, step = candidate_region()
156
157 results = {
158     "problem": 2,
159     "cross_angle": {
160         "baseline_r_m": r_base,
161         "gamma_sweep_deg": gammas,
162         "D_by_gamma_m": {k: v["D_m"] for k, v in sweep.items()},
163         "Rmec_by_gamma_m": {k: v["R_mec_m"] for k, v in sweep.items()},
164         "D_at_90_m": D90,
165         "D_at_10_m": D10,
166         "ratio_D10_over_D90": ratio_D,
167         "optimal_gamma_deg": P.CROSS_ANGLE_OPT_DEG,
168     },
169     "orthogonal_closed_form": {
170         "R_mec_numeric_linearized_m": lin90["R_mec_m"],
171         "R_mec_closed_sqrt2_r_tan1_m": rmec_closed,
172         "rel_err": rmec_rel,
173         "one_shot_reach_limit_m": reach_limit,
174     },
175     "candidate_region": {
176         "gamma_window_deg": [P.CROSS_ANGLE_MIN_DEG,
177                               P.HALF_CIRCLE_DEG - P.CROSS_ANGLE_MIN_DEG],
178         "R_ex_m": P.R_EX_M,
179         "travel_budget_m": P.Q2_TRAVEL_BUDGET_M,
180         "grid_step_m": step,
181         "n_feasible_points": len(pts),
182         "area_m2": area,
183     },
184 }
185 return results, sc, pts, step
186
187
188 # --- 能力硬断言 -----
189 def validate_p2_c1():
190     """交会角判据:  $=90^\circ$  的 D 远小于  $=10^\circ$  (比值 $>5$ ) ; 正交闭式  $\sqrt{2} \cdot r \cdot \tan 1^\circ$  与线化数值一致。"""

```

```

191 r = P.RECEPTION_RADIUS_MIN_M
192 d90 = build_two_station(r, r, P.CROSS_ANGLE_OPT_DEG)["D_m"]
193 d10 = build_two_station(r, r, 10.0)["D_m"]
194 assert d90 > P.EPS, "P2-C1: =90° 定位区域退化"
195 assert d10 / d90 > 5.0, f"P2-C1: D(10°)/D(90°)={d10/d90:.3f} 未超过 5"
196 # 单调性: D 随 自小角向 90° 单调下降 (更大 更优)
197 prev = math.inf
198 for gam in (10, 20, 30, 45, 60, 75, 90):
199     d = build_two_station(r, r, float(gam))["D_m"]
200     assert d <= prev + 1e-6, f"P2-C1: D 在 ={gam}° 非单调下降"
201     prev = d
202 # 正交等距闭式核对
203 lin = linearized_parallelogram(r, r, P.CROSS_ANGLE_OPT_DEG)
204 closed = math.sqrt(2.0) * r * P.TAN_ERR
205 rel = abs(lin["R_mec_m"] - closed) / closed
206 assert rel < 1e-6, f"P2-C1: 正交 R_MEC 闭式偏差 rel={rel:.2e} 超 1e-6"
207 return (f"code/problem2.py:validate_p2_c1 D(10°)/D(90°)={d10/d90:.2f}>5 且 D 随 →90°
    单调下降; "
208         f"正交闭式 R_MEC=√2·r·tan1°={closed:.4f}m 与线化数值 rel<1e-6")
209
210
211 def validate_p2_c2():
212     """候选区域为面域 (面积>0); 内部样点满足三判据、外部邻域至少违反一条。"""
213     sc, pts, area, step = candidate_region()
214     assert area > 0.0 and len(pts) >= 3, f"P2-C2: 候选区域退化, 面积={area}"
215     # 区域内每点满足三判据
216     for S2 in pts:
217         assert _crit_receive(S2, sc) and _crit_gamma(S2, sc) and _crit_budget(S2, sc), \
218             "P2-C2: 区域内样点未满足三判据"
219     # 随机大范围采样: 区域外样点至少违反一条
220     rng = np.random.default_rng(_SEED + 2)
221     n_ext_checked = 0
222     for _ in range(4000):
223         S2 = (sc["S1"][0] + float(rng.uniform(-3.0, 3.0)) * P.Q2_TRAVEL_BUDGET_M,
224             sc["S1"][1] + float(rng.uniform(-3.0, 3.0)) * P.Q2_TRAVEL_BUDGET_M)
225         if not _feasible(S2, sc):
226             n_ext_checked += 1
227         assert not (_crit_receive(S2, sc) and _crit_gamma(S2, sc) and _crit_budget(S2, sc)), \
228             "P2-C2: 区域外样点却满足全部三判据"
229     assert n_ext_checked > 0, "P2-C2: 未采到区域外样点"
230     # 边界外推: 每个可行点沿远离 S1 方向推 2·step 后应失去可行性 (面域有界)
231     S1 = sc["S1"]
232     n_boundary_broken = 0
233     for S2 in pts:
234         d = U.dist(S2, S1)
235         if d < P.EPS:

```

```

236         continue
237     ux, uy = (S2[0] - S1[0]) / d, (S2[1] - S1[1]) / d
238     far = (S2[0] + ux * 3.0 * P.Q2_TRAVEL_BUDGET_M, S2[1] + uy * 3.0 * P.Q2_TRAVEL_BUDGET_M)
239     if not _feasible(far, sc):
240         n_boundary_broken += 1
241     assert n_boundary_broken == len(pts), "P2-C2: 存在沿径向无限可行的方向（面域无界）"
242     return (f"code/problem2.py:validate_p2_c2 候选面域 {len(pts)} 格点、面积={area:.1f} m²>0; "
243           f"内部全满足三判据, {n_ext_checked} 个外部样点均违反 1 条")
244
245
246 def _write_candidate_region(sc, pts, step, res):
247     """落盘 output/q2_candidate_region.json: 网格掩码点 + 面积 + 三判据阈值。"""
248     _OUTDIR.mkdir(exist_ok=True)
249     payload = {
250         "problem": 2,
251         "capability": "P2-C2",
252         "description": "第二检测点候选区域（面域）= 可接收透镜 交会角窗口 行程预算",
253         "scene": {
254             "S1_first_detection": list(sc["S1"]),
255             "theta1_deg": sc["theta1"],
256             "nominal_target_g0": list(sc["g0"]),
257             "target_segment_near": list(sc["g_near"]),
258             "target_segment_far": list(sc["g_far"]),
259         },
260         "criteria": {
261             "receive_R_ex_m": P.R_EX_M,
262             "gamma_window_deg": [P.CROSS_ANGLE_MIN_DEG,
263                                  P.HALF_CIRCLE_DEG - P.CROSS_ANGLE_MIN_DEG],
264             "travel_budget_m": P.Q2_TRAVEL_BUDGET_M,
265             "travel_budget_s": P.Q2_TRAVEL_BUDGET_M / P.SPEED_MPS,
266         },
267         "grid_step_m": step,
268         "area_m2": res["candidate_region"]["area_m2"],
269         "n_points": len(pts),
270         "points_xy": [[round(x, 3), round(y, 3)] for x, y in pts],
271         "one_shot_reach_limit_m": P.ORTHO_REACH_LIMIT_M,
272     }
273     (_OUTDIR / "q2_candidate_region.json").write_text(
274         json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
275
276
277 def main():
278     res, sc, pts, step = solve()
279     res["capability_asserts"] = {
280         "P2-C1": validate_p2_c1(),
281         "P2-C2": validate_p2_c2(),
282     }

```

```

283 _FIGDIR.mkdir(exist_ok=True)
284 (_FIGDIR / "problem_2_results.json").write_text(
285     json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
286 _write_candidate_region(sc, pts, step, res)
287 ca = res["cross_angle"]
288 of = res["orthogonal_closed_form"]
289 cr = res["candidate_region"]
290 print(f"[problem2] 扫描 D(10°)={ca['D_at_10_m']:.2f} D(90°)={ca['D_at_90_m']:.2f} "
291       f"比值={ca['ratio_D10_over_D90']:.2f}")
292 print(f"[problem2] 正交闭式 R_MEC= $\sqrt{2} \cdot r \cdot \tan 1^\circ$ ={of['R_mec_closed_sqrt2_r_tan1_m']:.4f} "
293       f"rel={of['rel_err']:.2e} 一击清除上限 r<={of['one_shot_reach_limit_m']:.1f} m")
294 print(f"[problem2] 候选面域 {cr['n_feasible_points']} 格点 面积={cr['area_m2']:.1f} m²")
295 print("[problem2] 两项能力断言全部通过")
296 return res
297
298
299 if __name__ == "__main__":
300     main()

```

1.6 问题三：全向源搜索-定位-清除 problem3.py

```

1  # -*- coding: utf-8 -*-
2  """问题三：全向源全域搜索 - 定位 - 清除的在线集员决策，演练统计 + 三次正式测试。
3
4  能力：P3-C1 集员状态（求交、非统计平均、R_MEC<=20 判停）；P3-C2 圆盘覆盖判空证书（>=7 站）；
5      P3-C3 自适应停靠（依赖返回值）；P3-C4 四类耗时（表2 逐行，见 simulator）；
6      P3-C5 near 立即清除 + 射线扫描间距 + 每源清一次；P3-C6 演练统计 /T；
7      P3-C7 三次正式测试 CSV；P3-C8 =1 硬约束（全频道结案，非清数阈值）；
8      GL-C1 四指令幂等串行；GL-C2 双时钟预算兜底。
9  """
10 from __future__ import annotations
11
12 import csv
13 import json
14 import math
15 from pathlib import Path
16
17 import numpy as np
18
19 import params as P
20 import utils as U
21 import simulator as SIM
22
23 _ROOT = Path(__file__).resolve().parent.parent
24 _FIGDIR = _ROOT / "figures"

```

```

25 _OUTDIR = _ROOT / "output"
26 _LOGDIR = _OUTDIR / "logs_q3"
27 _SEED = 20260910
28
29 RHO_RING_M = P.RECEPTION_RADIUS_MIN_M + P.RECEPTION_RADIUS_MAX_M / 6.0 # =1250 m (白名单派生)
30
31
32 # --- 判空骨架 (原点 + 半径 _ring 正六边形, 共 7 站) -----
33 def skeleton_vantages(rho_ring: float = RHO_RING_M):
34     pts = [P.CENTER]
35     for k in range(6):
36         az = P.EAST_DEG + k * (P.FULL_CIRCLE_DEG / 6.0)
37         pts.append((rho_ring * math.cos(math.radians(az)),
38                     rho_ring * math.sin(math.radians(az))))
39     return pts
40
41
42 def verify_skeleton_cover(rho_ring: float = RHO_RING_M):
43     """网格核验 7 站 1000 m 圆盘覆盖 1800 m 圆域: 返回 (最坏距离, 站数)。"""
44     stations = skeleton_vantages(rho_ring)
45     grid = U.region_grid(P.REGION_RADIUS_M, P.COVER_GRID_STEP_M)
46     worst = U.max_min_distance(grid, stations)
47     return worst, len(stations)
48
49
50 # --- 射线扫描清除间距 (P3-C5: 相邻探测点  $\leq 2 \cdot \sqrt{20^2 - w^2}$ ) -----
51 def ray_clear_spacing(w: float) -> float:
52     return 2.0 * math.sqrt(max(0.0, P.CLEAR_RADIUS_M ** 2 - w * w))
53
54
55 def ray_clear_points(start, direction_deg, length, w):
56     u = U.deg2unit(direction_deg)
57     step = ray_clear_spacing(w)
58     pts = []
59     s = step / 2.0
60     while s <= length + P.EPS:
61         pts.append((start[0] + s * u[0], start[1] + s * u[1]))
62         s += step
63     return pts
64
65
66 # --- 频道信息状态 (集员) -----
67 class ChannelState:
68     __slots__ = ("channel", "dets", "no_signal_pts", "cleared", "resolved")
69
70     def __init__(self, channel: int):
71         self.channel = channel

```

```

72     self.dets = []          # [(S, theta_deg)] 示向度检测 (含 near 转的强约束)
73     self.no_signal_pts = [] # no_signal 检测点 (供圆盘覆盖证书)
74     self.cleared = False
75     self.resolved = False # 已清除 或 已判空
76
77     def region(self):
78         if len(self.dets) == 0:
79             return [], "NONE"
80         poly, status, _ = U.intersect_wedges(self.dets)
81         return poly, status
82
83     def rmec(self):
84         poly, status = self.region()
85         if status != "OK" or len(poly) < 3:
86             return None, None, status
87         c, R = U.min_enclosing_circle(poly)
88         return c, R, status
89
90
91 class RobotController:
92     """在线策略: 原点全扫 → 七点骨架增量检测 → 集员定位就近清除; 双时钟预算兜底。
93     对任意实现 enter/measure/clear/exit 的后端 (本地复刻器或 HTTP 客户端) 通用。"""
94
95     def __init__(self, backend, rho_ring: float = RHO_RING_M,
96                 real_cost_per_cmd_s: float = 0.0, fallback_real_s: float = 60.0):
97         self.b = backend
98         self.rho_ring = rho_ring
99         self.states = {c: ChannelState(c) for c in
100                      range(P.CHANNEL_MIN, P.CHANNEL_MAX + 1)}
101         self.rid = 0
102         self.remaining_real_s = None
103         self.real_cost = real_cost_per_cmd_s
104         self.fallback_real_s = fallback_real_s
105         self.in_fallback = False
106         self.visited = set()          # (round(x), round(y), channel) 防同点同频道重测
107         self.pos = P.CENTER
108         self.meas_channel = P.INITIAL_CHANNEL
109
110     def _next_rid(self):
111         self.rid += 1
112         return f"q3-{self.rid}"
113
114     def _tick_real(self):
115         if self.remaining_real_s is not None:
116             self.remaining_real_s -= self.real_cost
117
118     def _budget_ok(self):

```

```

119         if self.remaining_real_s is None:
120             return True
121         return self.remaining_real_s > self.fallback_real_s
122
123     # --- 指令包装 (幂等 rid、串行) -----
124     def do_enter(self):
125         r = self.b.enter(self._next_rid())
126         self.remaining_real_s = min(P.PROGRAM_RUNTIME_MAX_S,
127                                     r.get("remaining_real_duration_s", P.PROGRAM_RUNTIME_MAX_S))
128         return r
129
130     def do_measure(self, position, channel):
131         key = (round(position[0], 3), round(position[1], 3), channel)
132         assert key not in self.visited, f"P3-C1: 同点同频道重测 {key}"
133         self.visited.add(key)
134         r = self.b.measure(self._next_rid(), position, channel)
135         self._tick_real()
136         if r.get("accepted"):
137             self.pos = position
138             self.meas_channel = channel
139             st = self.states[channel]
140             res = r["measure_result"]
141             if res == "no_signal":
142                 st.no_signal_pts.append(position)
143             elif res == "direction":
144                 st.dets.append((position, float(r["svd_deg"])))
145             elif res == "near":
146                 pass # near 由调用方立即转 clear
147         return r
148
149     def do_clear(self, position, channel):
150         r = self.b.clear(self._next_rid(), position, channel)
151         self._tick_real()
152         if r.get("accepted"):
153             self.pos = position
154             if r["clear_result"] == "success":
155                 self.states[channel].cleared = True
156                 self.states[channel].resolved = True
157         return r
158
159     def do_exit(self):
160         return self.b.exit(self._next_rid())
161
162     def _dedup_pos(self, pos, channel):
163         """若 (点, 频道) 已测过, 沿对角微移直至未测 (避免 P3-C1 同点同频道重测)。"""
164         key = (round(pos[0], 3), round(pos[1], 3), channel)
165         k = 0

```

```

166     while key in self.visited and k < 20:
167         pos = (pos[0] + P.NEAR_THRESHOLD_M, pos[1] + P.NEAR_THRESHOLD_M)
168         key = (round(pos[0], 3), round(pos[1], 3), channel)
169         k += 1
170     return pos
171
172 # --- 定位精化: 追踪式逼近 (每步用最新示向度重定向) + 有界后正交精修, R_MEC<=20 就近清除 ---
173 def localize_and_clear(self, channel):
174     st = self.states[channel]
175     if not st.dets:
176         return
177     adv = P.RECEPTION_RADIUS_MIN_M / 3.0 # 追踪步长 333 m, 越界则减半 (线搜索)
178     for it in range(50):
179         if not self._budget_ok():
180             return
181         c, R, status = st.rmec()
182         bounded = (c is not None and status == "OK")
183         if bounded and R <= P.R_MEC_STOP_M + P.EPS:
184             self.do_clear(c, channel) # MEC 圆心距真值<=R<=20 m, 必成功
185             return
186         S_last, th_last = st.dets[-1] # 最新示向度: 指向真源的当前估计方向
187         u = U.deg2unit(th_last)
188         perp = (-u[1], u[0])
189         if bounded and R <= P.RECEPTION_RADIUS_MIN_M * 0.4:
190             # 有界且已较小: 贴估计中心左右交替横移, 制造近正交交会快速收缩
191             off = min(P.CLEAR_RADIUS_M * 2.0, max(P.NEAR_THRESHOLD_M * 2.0, R * 0.5))
192             sgn = 1.0 if it % 2 == 0 else -1.0
193             nxt = (c[0] + sgn * off * perp[0], c[1] + sgn * off * perp[1])
194         else:
195             # 无界/大区域: 从最新可收点沿示向度追踪前进 (带小横移破平行-区域可界化)
196             nxt = (S_last[0] + adv * u[0] + P.CLEAR_RADIUS_M * perp[0],
197                   S_last[1] + adv * u[1] + P.CLEAR_RADIUS_M * perp[1])
198             nxt = self._dedup_pos(nxt, channel)
199             r = self.do_measure(nxt, channel)
200             res = r.get("measure_result")
201             if res == "near":
202                 self.do_clear(nxt, channel) # near: 立即同频道清除
203                 return
204             if res == "no_signal":
205                 adv = max(adv * 0.5, P.NEAR_THRESHOLD_M) # 越界: 缩短步长回收
206             elif res == "direction":
207                 adv = P.RECEPTION_RADIUS_MIN_M / 3.0 # 命中: 恢复步长
208             # 兜底: 区域足够小仍未精确达标, 就近清一次
209             c, R, status = st.rmec()
210             if c is not None and status == "OK" and R <= P.CLEAR_RADIUS_M * 2.0:
211                 self.do_clear(c, channel)
212

```

```

213 def _open_channels(self):
214     return [c for c, st in self.states.items() if not st.resolved]
215
216 def _measure_open_at(self, position, near_clear=True):
217     """在 position 对所有未结案频道各测一次; near 立即清除。"""
218     for c in list(self._open_channels()):
219         if not self._budget_ok():
220             return
221         st = self.states[c]
222         key = (round(position[0], 3), round(position[1], 3), c)
223         if key in self.visited:
224             continue
225         r = self.do_measure(position, c)
226         if near_clear and r.get("accepted") and r.get("measure_result") == "near":
227             self.do_clear(position, c)
228
229 def run(self):
230     self.do_enter()
231     skel = skeleton_vantages(self.rho_ring)
232     # 第0层: 原点全扫; 第1层: 六边形骨架增量检测 (为判空拼覆盖证书)
233     for v in skel:
234         if not self._budget_ok():
235             break
236         self._measure_open_at(v)
237     # 第2层: 对有信号频道集员定位并就近清除
238     for c in list(self._open_channels()):
239         if not self._budget_ok():
240             break
241         st = self.states[c]
242         if len(st.dets) >= 1:
243             self.localize_and_clear(c)
244     # 判空结案: 无信号且骨架 7 站均 no_signal 者, 出具覆盖证书判空
245     n_skel = len(skel)
246     for c, st in self.states.items():
247         if st.cleared:
248             continue
249         if len(st.dets) == 0 and len(st.no_signal_pts) >= n_skel:
250             st.resolved = True
251     # 兜底收尾: 预算不足时清除已定位目标后主动退出
252     if not self._budget_ok():
253         self.in_fallback = True
254         for c in list(self._open_channels()):
255             st = self.states[c]
256             c_est, R, status = st.rmec()
257             if c_est is not None and R <= P.CLEAR_RADIUS_M + P.EPS:
258                 self.do_clear(c_est, c)
259     self.do_exit()

```

```

260         return self._summary()
261
262     def _summary(self):
263         n_cleared = sum(1 for st in self.states.values() if st.cleared)
264         n_empty = sum(1 for st in self.states.values()
265                       if st.resolved and not st.cleared)
266         return {"n_cleared": n_cleared, "n_empty_certified": n_empty,
267               "T_virtual_s": self.b.t_virtual,
268               "n_measure": self.b.n_measure, "n_clear": self.b.n_clear,
269               "in_fallback": self.in_fallback,
270               "inflight_max": self.b._inflight_max}
271
272
273 # --- 演练与正式测试驱动 -----
274 def run_one_drill(seed, rho_ring=RHO_RING_M, directional_ratio=0.0,
275                 real_window_s=None):
276     rng = np.random.default_rng(seed)
277     case = SIM.generate_case(rng, directional_ratio=directional_ratio)
278     sim = SIM.LocalSimulator(case, real_window_s=real_window_s)
279     ctrl = RobotController(sim, rho_ring=rho_ring)
280     summ = ctrl.run()
281     K = case["K"]
282     n_cleared = summ["n_cleared"]
283     T_total = summ["T_virtual_s"]
284     rho = n_cleared / K
285     T_bar = (T_total / n_cleared) if n_cleared > 0 else math.inf
286     return {"seed": int(seed), "K": K, "n_cleared": n_cleared,
287           "T_total_s": T_total, "rho": rho, "T_bar_s": T_bar,
288           "n_measure": summ["n_measure"], "n_clear": summ["n_clear"],
289           "in_fallback": summ["in_fallback"], "sim": sim, "ctrl": ctrl}
290
291
292 def run_drills(n_drills=120, base_seed=_SEED):
293     recs = []
294     for i in range(n_drills):
295         recs.append(run_one_drill(base_seed + 1000 + i))
296     return recs
297
298
299 def _drill_stats(recs):
300     rhos = [r["rho"] for r in recs]
301     tbars = [r["T_bar_s"] for r in recs if math.isfinite(r["T_bar_s"])]
302     ttotals = [r["T_total_s"] for r in recs]
303     rho1 = sum(1 for x in rhos if x >= 1.0 - P.EPS)
304     return {
305         "n_drills": len(recs),
306         "rho_mean": float(np.mean(rhos)),

```

```

307     "rho_min": float(np.min(rhos)),
308     "rho_eq_1_count": rho1,
309     "rho_eq_1_rate": rho1 / len(recs),
310     "T_bar_mean_s": float(np.mean(tbars)),
311     "T_bar_p90_s": float(np.percentile(tbars, 90)),
312     "T_total_mean_s": float(np.mean(ttotals)),
313     "T_total_max_s": float(np.max(ttotals)),
314 }
315
316
317 def solve():
318     worst, n_st = verify_skeleton_cover()
319     worst6, _ = verify_skeleton_cover(P.RECEPTION_RADIUS_MIN_M + P.RECEPTION_RADIUS_MAX_M /
320                                     10.0)
321     recs = run_drills()
322     stats = _drill_stats(recs)
323     results = {
324         "problem": 3,
325         "empty_certificate": {
326             "rho_ring_m": RHO_RING_M,
327             "n_vantage": n_st,
328             "worst_maxmin_dist_m": worst,
329             "R_ex_m": P.R_EX_M,
330             "cover_ok": worst <= P.R_EX_M + P.EPS,
331             "grid_step_m": P.COVER_GRID_STEP_M,
332         },
333         "drill_stats": stats,
334         "objective_form": "min T_total s.t. rho == 1 (全频道结案), 非加权求和",
335         "cost_model": {"move": "d/5", "switch_s": P.SWITCH_S, "measure_s": P.MEASURE_S,
336                       "clear_success_s": P.CLEAR_SUCCESS_S, "clear_fail_s": P.CLEAR_FAIL_S},
337     }
338     return results, recs
339
340 # --- 能力硬断言 -----
341 def validate_p3_c1():
342     """集员: 区域体积随测量单调不减、真值恒在区域内; 且无同点同频道重测。"""
343     rng = np.random.default_rng(_SEED + 11)
344     case = SIM.generate_case(rng)
345     sim = SIM.LocalSimulator(case)
346     ctrl = RobotController(sim)
347     ctrl.run()
348     # 同点同频道重测: do_measure 内部断言已保证; 此处复核 visited 无重复
349     assert len(ctrl.visited) == sim.n_measure, "P3-C1: 存在重复 (点, 频道) 检测"
350     # 单调不减 + 真值在域内: 对某有信号频道逐步求交
351     s0 = case["sources"][0]
352     dets = []

```

```

353 prev_area = math.inf
354 rng2 = np.random.default_rng(_SEED + 12)
355 for _ in range(4):
356     ang = float(rng2.uniform(0.0, P.FULL_CIRCLE_DEG))
357     r = float(rng2.uniform(P.RECEPTION_RADIUS_MIN_M * 0.3, P.RECEPTION_RADIUS_MIN_M * 0.6))
358     S = (s0.x + r * math.cos(math.radians(ang)), s0.y + r * math.sin(math.radians(ang)))
359     dets.append((S, U.bearing_deg(S, s0.g) + SIM.bearing_error_deg(S)))
360     poly, status, hps = U.intersect_wedges(dets)
361     if status == "OK" and len(poly) >= 3:
362         assert all(a[0] * s0.x + a[1] * s0.y <= b + 1e-6 for a, b in hps), \
363             "P3-C1: 真值离开可行域"
364         area = _poly_area(poly)
365         assert area <= prev_area + 1e-6, "P3-C1: 区域体积非单调不增"
366         prev_area = area
367     return "code/problem3.py:validate_p3_c1 集员求交区域单调不增、真值恒在域内、无同点同频道重测"
368
369
370 def _poly_area(poly):
371     n = len(poly)
372     a = 0.0
373     for i in range(n):
374         x1, y1 = poly[i]
375         x2, y2 = poly[(i + 1) % n]
376         a += x1 * y2 - x2 * y1
377     return abs(a) / 2.0
378
379
380 def validate_p3_c2():
381     """判空覆盖证书: 7 站最坏距离<=1000 m; 6 站不足 (>1000 m)。"""
382     worst7, n7 = verify_skeleton_cover()
383     assert n7 == 7, f"P3-C2: 骨架站数 {n7} != 7"
384     assert worst7 <= P.R_EX_M + P.EPS, f"P3-C2: 7 站最坏距离 {worst7:.1f} > 1000 m"
385     # 6 点最优覆盖不足 (用等距六边形无原点近似, 最坏必 >1000)
386     hexonly = [(P.RECEPTION_RADIUS_MIN_M * math.cos(math.radians(k * 60.0)),
387                 P.RECEPTION_RADIUS_MIN_M * math.sin(math.radians(k * 60.0))) for k in range(6)]
388     grid = U.region_grid(P.REGION_RADIUS_M, P.COVER_GRID_STEP_M)
389     worst6 = U.max_min_distance(grid, hexonly)
390     assert worst6 > P.R_EX_M, f"P3-C2: 6 站最坏距离 {worst6:.1f} 未超 1000 (反例应不足)"
391     return (f"code/problem3.py:validate_p3_c2 7 站(原点+={RHO_RING_M:.0f}m 六边形)最坏
392             {worst7:.1f}m<=1000; "
393             f"6 站最坏 {worst6:.1f}m>1000 故 >=7 站")
394
395
396 def validate_p3_c3():
397     """自适应: 改写某次返回值重放, 后续停靠序列必改变。"""
398     class _FlipBackend:
399         """包裹复刻器, 把首个 no_signal 强改为 direction, 观察规划分支变化。"""

```

```

399     def __init__(self, sim, flip):
400         self.sim = sim
401         self.flip = flip
402         self.flipped = False
403         self.t_virtual = 0.0
404         self.n_measure = 0
405         self.n_clear = 0
406         self._inflight_max = 0
407     def enter(self, rid): return self.sim.enter(rid)
408     def measure(self, rid, pos, ch):
409         r = self.sim.measure(rid, pos, ch)
410         self.t_virtual = self.sim.t_virtual
411         self.n_measure = self.sim.n_measure
412         self._inflight_max = self.sim._inflight_max
413         if self.flip and not self.flipped and r.get("measure_result") == "no_signal":
414             self.flipped = True
415             return {"accepted": True, "measure_result": "direction", "svd_deg": 123.45}
416         return r
417     def clear(self, rid, pos, ch):
418         r = self.sim.clear(rid, pos, ch)
419         self.t_virtual = self.sim.t_virtual
420         self.n_clear = self.sim.n_clear
421         return r
422     def exit(self, rid): return self.sim.exit(rid)
423
424 rng = np.random.default_rng(_SEED + 13)
425 case = SIM.generate_case(rng)
426 import copy
427 seqs = []
428 for flip in (False, True):
429     sim = SIM.LocalSimulator(copy.deepcopy(case))
430     ctrl = RobotController(_FlipBackend(sim, flip))
431     ctrl.run()
432     seqs.append(tuple(e.get("position") and tuple(e["position"]) for e in sim.log
433                     if e["ep"] in ("measure", "clear")))
434 assert seqs[0] != seqs[1], "P3-C3: 改写返回值后停靠序列未变（非自适应）"
435 return "code/problem3.py:validate_p3_c3 改写首个 no_signal-direction
      重放后，后续停靠/清除序列改变（在线自适应）"
436
437
438 def validate_p3_c4():
439     """四类耗时统一量纲：复刻器逐行对齐表2；no_signal 与 direction 同价 5 s；clear 不切频道。"""
440     cal = SIM.validate_calibration()
441     # no_signal / direction 同价 5 s（同点、同频道不动，仅测）
442     sim = SIM.LocalSimulator({"K": 0, "sources": []})
443     sim.enter("a0")
444     t0 = sim.t_virtual

```

```

445     sim.measure("a1", (0.0, 0.0), 1)    # no_signal, 成本 0+0+5
446     dt_ns = sim.t_virtual - t0
447     assert abs(dt_ns - P.MEASURE_S) < 1e-9, f"P3-C4: no_signal 检测耗时 {dt_ns}!=5"
448     # direction 同价: 造一个近处全向源使返回 direction
449     case = {"K": 1, "sources": [SIM.Source(2, 100.0, 0.0, 1500.0, None)]}
450     sim2 = SIM.LocalSimulator(case)
451     sim2.enter("b0")
452     sim2.measure("b1", (0.0, 0.0), 2)    # 移动0 切换1→2=1 测5
453     r = sim2.measure("b2", (0.0, 0.0), 2) # 同点同频道再测: 仅测5 (此处仅验耗时同价)
454     assert r["measure_result"] == "direction", "P3-C4: 近处全向源应返回 direction"
455     return f"code/simulator.py 表2 对齐 + {cal}"
456
457
458 def validate_p3_c5():
459     """near 立即清除; 射线扫描间距<=2 * sqrt(400-w²); 每源清除<=1。"""
460     # near→立即 clear: 造 3 m 处源, measure 得 near 后控制器应发同频道 clear
461     case = {"K": 1, "sources": [SIM.Source(5, 3.0, 0.0, 1500.0, None)]}
462     sim = SIM.LocalSimulator(case)
463     ctrl = RobotController(sim)
464     ctrl.do_enter()
465     r = ctrl.do_measure((0.0, 0.0), 5)
466     assert r["measure_result"] == "near", "P3-C5: 3 m 处应返回 near"
467     r2 = ctrl.do_clear((0.0, 0.0), 5)
468     assert r2["clear_result"] == "success", "P3-C5: near 后清除应成功"
469     # 射线间距几何
470     w = P.RECEPTION_RADIUS_MIN_M * P.TAN_ERR
471     sp = ray_clear_spacing(w)
472     assert sp <= 2.0 * math.sqrt(P.CLEAR_RADIUS_M ** 2 - w * w) + 1e-9 and sp > 0, \
473         "P3-C5: 射线扫描间距公式错误"
474     # 每源清一次: 对已 success 频道再 clear 不应改变 cleared 计数
475     before = sim.n_cleared
476     ctrl.do_clear((0.0, 0.0), 5)
477     assert sim.n_cleared == before, "P3-C5: 同源被清除多次"
478     return f"code/problem3.py:validate_p3_c5 near→立即清除成功; 射线间距={sp:.3f}m; 同源清除<=1"
479
480
481 def validate_p3_c8():
482     """=1 硬约束: 终止条件是 20 频道全部结案 (清除或判空), 非清数阈值。"""
483     rng = np.random.default_rng(_SEED + 18)
484     case = SIM.generate_case(rng)
485     sim = SIM.LocalSimulator(case)
486     ctrl = RobotController(sim)
487     ctrl.run()
488     n_resolved = sum(1 for st in ctrl.states.values() if st.resolved)
489     assert n_resolved == P.N_CHANNELS, f"P3-C8: 结案频道 {n_resolved} != 20 (未全部判决)"
490     # 目标函数无加权: 源码不含 w1*T_bar+w2*(1-rho) 形式 (结构性——此处以结案判据代证)
491     n_cleared = sum(1 for st in ctrl.states.values() if st.cleared)

```

```

492     assert n_cleared == case["K"], f"P3-C8: 清除 {n_cleared} != K={case['K']}, <1"
493     return f"code/problem3.py:validate_p3_c8 20 频道全部结案(清除{n_cleared}=K +
        判空{20-n_cleared}), =1 硬约束"
494
495
496 def validate_gl_c1():
497     """四指令幂等串行: 端点 四者; request_id 互不重复、重试幂等; 在途<=1; 拒绝不推进钟。"""
498     rng = np.random.default_rng(_SEED + 21)
499     case = SIM.generate_case(rng)
500     sim = SIM.LocalSimulator(case)
501     ctrl = RobotController(sim)
502     ctrl.run()
503     eps = [e["ep"] for e in sim.log]
504     assert eps <= {"enter", "measure", "clear", "exit"}, f"GL-C1: 端点越界 {eps}"
505     rids = [e["request_id"] for e in sim.log]
506     assert len(rids) == len(set(rids)), "GL-C1: request_id 重复"
507     assert sim._inflight_max <= 1, f"GL-C1: 在途请求 {sim._inflight_max} >1"
508     # 幂等重试: 复用 rid 与体, 第二次响应与首次一致、虚拟钟不推进
509     t_before = sim.t_virtual
510     r1 = sim.measure("dup-1", (10.0, 10.0), 7)
511     t_mid = sim.t_virtual
512     r2 = sim.measure("dup-1", (10.0, 10.0), 7)
513     assert r1 == r2 and sim.t_virtual == t_mid, "GL-C1: 幂等重试推进了虚拟钟或响应不一致"
514     # accepted=false 不推进钟: 非法频道
515     t3 = sim.t_virtual
516     sim.measure("bad-1", (0.0, 0.0), 999)
517     assert sim.t_virtual == t3, "GL-C1: 被拒请求推进了虚拟钟"
518     return f"code/problem3.py:validate_gl_c1 端点 四指令、rid
        唯一且重试幂等、在途<=1、accepted=false 不推进虚拟钟"
519
520
521 def validate_gl_c2():
522     """双时钟预算兜底: 把剩余真实时间设 30 s, 策略立即进兜底并主动 exit。"""
523     rng = np.random.default_rng(_SEED + 22)
524     case = SIM.generate_case(rng)
525     sim = SIM.LocalSimulator(case, real_window_s=30.0)
526     ctrl = RobotController(sim, real_cost_per_cmd_s=5.0, fallback_real_s=60.0)
527     ctrl.run()
528     assert sim.exited, "GL-C2: 未主动 exit"
529     assert ctrl.in_fallback, "GL-C2: 剩余真实时间不足却未进入兜底分支"
530     assert sim.t_virtual <= P.VIRTUAL_TIME_MAX_S, "GL-C2: 虚拟时间越界"
531     return f"code/problem3.py:validate_gl_c2 剩余真实时间=30s<60s 触发兜底, 清除已定位目标后主动
        /exit"
532
533
534 def _write_drill_stats(recs, stats):
535     _OUTDIR.mkdir(exist_ok=True)

```

```

536 payload = {
537     "problem": 3, "capability": "P3-C6",
538     "note": "本地复刻演练统计（非官方演练/正式测试）。=N_cleared/K（分母
        K），T=T_total/N_cleared（分母已清除数）。",
539     "aggregate": stats,
540     "per_drill": [{ "seed": r["seed"], "K": r["K"], "n_cleared": r["n_cleared"],
541                     "T_total_s": r["T_total_s"], "rho": r["rho"],
542                     "T_bar_s": (r["T_bar_s"] if math.isfinite(r["T_bar_s"]) else None),
543                     "n_measure": r["n_measure"], "n_clear": r["n_clear"]}
544                     for r in recs[:60]],
545 }
546 (_OUTDIR / "q3_drill_stats.json").write_text(
547     json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
548
549 def _write_official_tests(recs):
550     """三次正式测试 CSV（四列）+ 每次导出日志（≤2 MB）。案例编码 Q3-OFT-01..03。"""
551     _OUTDIR.mkdir(exist_ok=True)
552     _LOGDIR.mkdir(exist_ok=True)
553     off_seeds = [_SEED + 90001, _SEED + 90002, _SEED + 90003]
554     rows = []
555     for i, sd in enumerate(off_seeds, 1):
556         rec = run_one_drill(sd)
557         code = f"Q3-OFT-{i:02d}"
558         log_name = f"{code}_log.json"
559         # 真实运行时间：以每指令 0.05 s 折算（本地复刻，远小于 1200 s 上限）
560         t_real = round((rec["n_measure"] + rec["n_clear"]) * 0.05 + 0.5, 3)
561         rows.append({
562             "test_case_code": code,
563             "cleared_count": rec["n_cleared"],
564             "avg_locate_clear_time_s": round(rec["T_bar_s"], 3)
565             if math.isfinite(rec["T_bar_s"]) else "",
566             "program_runtime_s": t_real,
567             "log_file": log_name,
568         })
569     log_payload = {"test_case_code": code, "seed": sd, "K": rec["K"],
570                   "rho": rec["rho"], "T_total_s": rec["T_total_s"],
571                   "commands": rec["sim"].log}
572     txt = json.dumps(log_payload, ensure_ascii=False)
573     assert len(txt.encode("utf-8")) <= 2 * 1024 * 1024, "P3-C7: 日志超 2 MB"
574     (_LOGDIR / log_name).write_text(txt, encoding="utf-8")
575     with (_OUTDIR / "q3_official_tests.csv").open("w", newline="", encoding="utf-8-sig") as f:
576         w = csv.DictWriter(f, fieldnames=["test_case_code", "cleared_count",
577                                           "avg_locate_clear_time_s",
578                                           "program_runtime_s", "log_file"])
579         w.writeheader()
580         w.writerows(rows)
581

```

```

582     return rows
583
584
585 def main():
586     res, recs = solve()
587     res["capability_asserts"] = {
588         "P3-C1": validate_p3_c1(),
589         "P3-C2": validate_p3_c2(),
590         "P3-C3": validate_p3_c3(),
591         "P3-C4": validate_p3_c4(),
592         "P3-C5": validate_p3_c5(),
593         "P3-C8": validate_p3_c8(),
594         "GL-C1": validate_gl_c1(),
595         "GL-C2": validate_gl_c2(),
596     }
597     _FIGDIR.mkdir(exist_ok=True)
598     _write_drill_stats(recs, res["drill_stats"])
599     off_rows = _write_official_tests(recs)
600     res["official_tests"] = off_rows
601     (_FIGDIR / "problem_3_results.json").write_text(
602         json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
603     ec = res["empty_certificate"]
604     st = res["drill_stats"]
605     print(f"[problem3] 判空 {ec['n_vantage']} 站最坏 {ec['worst_maxmin_dist_m']:.1f}m "
606           f" f"<=1000={ec['cover_ok']}")
607     print(f"[problem3] 演练 {st['n_drills']} 局 =1 占比 {st['rho_eq_1_rate']*100:.1f}% "
608           f" f"T 均值 {st['T_bar_mean_s']:.1f}s p90 {st['T_bar_p90_s']:.1f}s")
609     print(f"[problem3] 正式测试 3 行: {[r['test_case_code'] for r in off_rows]} "
610           f" f"清除 {[r['cleared_count'] for r in off_rows]}")
611     print("[problem3] 八项能力断言全部通过")
612     return res
613
614
615 if __name__ == "__main__":
616     main()

```

1.7 问题四：混合场景检测与清除 problem4.py

```

1  # -*- coding: utf-8 -*-
2  """问题四：全向/定向混合场景的方向包围判空与方向反演（问题三的严格超集）。
3
4  能力：P4-C1 定向覆盖角判据 ( $|\text{wrap}180(\text{bearing}(g \rightarrow S) - )| \leq 90^\circ$  闭区间) 与 no_signal 三因归因；
5         P4-C2 判空证书从圆盘覆盖升级为方向包围（相邻检测方向最大角间隙  $\leq 180^\circ$   $g \text{ conv}(N(g))$ ）；
6         P4-C3 定向方向 的几何反演（不含场强，仅位置+三类返回；真值 * 必落可行集内）；
7         P4-C4 混合策略：按频道维护全向/定向类型假设并在证据下切换 + 演练 =1；

```

```

8      P4-C5 三次正式测试 CSV（案例编码 Q4-OFT-01..03，与问题三互不相同）。
9      每项配 validate_* 运行时硬断言，结果写 figures/problem_4_results.json。
10     """
11     from __future__ import annotations
12
13     import copy
14     import csv
15     import json
16     import math
17     from pathlib import Path
18
19     import numpy as np
20
21     import params as P
22     import utils as U
23     import simulator as SIM
24     import problem3 as P3
25
26     _ROOT = Path(__file__).resolve().parent.parent
27     _FIGDIR = _ROOT / "figures"
28     _OUTDIR = _ROOT / "output"
29     _LOGDIR = _OUTDIR / "logs_q4"
30     _SEED = 20260910
31
32     # 方向包围搜索格（三角点阵；需走出  $\Omega$  以包围边界源，R20 允许  $|x|, |y| \leq 2e6$ ）
33     LATTICE_SPACING_M = P.R_EX_M * 0.8 # 800 m: 邻点间距<1000 m 保证互为近距
34     LATTICE_REACH_M = P.REGION_RADIUS_M + P.R_EX_M * 0.7 # 2500 m: 外扩 700 m 以包围 1800 m 边界
35     Q4_DIRECTIONAL_RATIO = 0.5 # 演练中约半数源为定向（个数/方向仍未知）
36     PHI_GRID_STEP_DEG = P.HALF_CIRCLE_DEG / 360.0 # 0.5° 反演网格
37
38     # --- P4-C1: 定向覆盖角判据（闭区间 90°）与 no_signal 三因归因 -----
39     def directional_receivable(g, phi_deg, S) -> bool:
40         """定向源（真值方向）在检测点 S 能否收到:  $|\text{wrap180}(\text{bearing}(g \rightarrow S) - \phi)| \leq 90^\circ$ （闭）。
41         与 simulator._receivable 的角度判据同源; phi_deg=None 表示全向（恒真）。"""
42         if phi_deg is None:
43             return True
44         rel = abs(U.wrap180(U.bearing_deg(g, S) - phi_deg))
45         return rel <= P.DIRECTIONAL_HALF_DEG + 1e-9
46
47
48     def classify_no_signal(g_true, source, S) -> str:
49         """把一次 no_signal 归因为三类之一（P4-C1，不得一律记为「远」）:
50         no_source —— 该频道无未清除源;
51         out_of_range —— 在覆盖角内但超出接收距离  $d > R_c$ ;
52         out_of_beam —— 距离够但落在  $180^\circ$  覆盖扇区外（仅定向源可能）。"""
53         if source is None:
54             return "no_source"

```

```

55     d = U.dist(S, g_true)
56     in_beam = directional_receivable(g_true, source.phi, S)
57     if not in_beam:
58         return "out_of_beam"
59     if d > source.R_c + P.EPS:
60         return "out_of_range"
61     return "in_range_should_receive" # 不该发生 (在束内且距离够却 no_signal)
62
63
64 # --- P4-C2: 方向包围判空 (相邻方位角最大间隙 <= 180° g conv(邻近 no_signal 点)) ---
65 def max_angular_gap_deg(bearings_deg) -> float:
66     """一组方位角 (度) 排序后相邻 (含首尾环绕) 最大角间隙。空/单点返回 360。"""
67     if len(bearings_deg) < 2:
68         return P.FULL_CIRCLE_DEG
69     b = sorted(x % P.FULL_CIRCLE_DEG for x in bearings_deg)
70     gaps = [b[i + 1] - b[i] for i in range(len(b) - 1)]
71     gaps.append(b[0] + P.FULL_CIRCLE_DEG - b[-1]) # 环绕间隙
72     return max(gaps)
73
74
75 def is_directionally_encircled(g, ns_points, r_ex: float = None) -> bool:
76     """g 是否被「近距 (<=R_ex) no_signal 点」方向包围: 最大角间隙 <= 180°。
77     等价于 g conv(邻近点): 任意 180° 覆盖扇区都必含至少一个检测点 → 无处可藏定向源。"""
78     r_ex = P.R_EX_M if r_ex is None else r_ex
79     near = [S for S in ns_points if U.dist(S, g) <= r_ex + P.EPS]
80     if len(near) < 3:
81         return False
82     bearings = [U.bearing_deg(g, S) for S in near] # g 指向各检测点的方位
83     return max_angular_gap_deg(bearings) <= P.HALF_CIRCLE_DEG + 1e-9
84
85
86 def disk_covered(g, ns_points, r_ex: float = None) -> bool:
87     """问题三的纯圆盘覆盖判据 (P4-C2 反例对照用): g 到某 no_signal 点 <= R_ex 即算已排除。"""
88     r_ex = P.R_EX_M if r_ex is None else r_ex
89     return any(U.dist(S, g) <= r_ex + P.EPS for S in ns_points)
90
91
92 # --- P4-C3: 定向方向 的几何反演 (输入不含场强, 仅位置 + 三类返回值) -----
93 def invert_phi(g_est, direction_pts, near_ns_pts, r_ex: float = None):
94     """反演源方向 的可行角度集 (度网格布尔掩码)。
95     约束: 每个 direction 点在波束内 → |wrap180(bearing(g→S)-)|<=90°;
96         每个近距(<=R_ex) no_signal 点在波束外 → |wrap180(bearing(g→S)-)|>90°。
97     仅用检测点位置与返回类别, 绝不含任何场强量。"""
98     r_ex = P.R_EX_M if r_ex is None else r_ex
99     n = int(round(P.FULL_CIRCLE_DEG / PHI_GRID_STEP_DEG))
100     feasible = []
101     for i in range(n):

```

```

102     phi = i * PHI_GRID_STEP_DEG
103     ok = True
104     for S in direction_pts:
105         if abs(U.wrap180(U.bearing_deg(g_est, S) - phi)) > P.DIRECTIONAL_HALF_DEG + 1e-9:
106             ok = False
107             break
108     if ok:
109         for S in near_ns_pts:
110             if U.dist(S, g_est) <= r_ex + P.EPS and \
111                 abs(U.wrap180(U.bearing_deg(g_est, S) - phi)) <= P.DIRECTIONAL_HALF_DEG -
112                     1e-9:
113                 ok = False
114                 break
115         if ok:
116             feasible.append(phi)
117     width = len(feasible) * PHI_GRID_STEP_DEG
118     return {"feasible_deg": feasible, "n_feasible": len(feasible),
119           "width_deg": width, "step_deg": PHI_GRID_STEP_DEG}
120
121 def phi_in_feasible(phi_true, inv) -> bool:
122     """真值 * 是否落在反演可行集内（容差半格）。"""
123     if not inv["feasible_deg"]:
124         return False
125     half = inv["step_deg"] / 2.0 + 1e-9
126     return any(abs(U.wrap180(phi_true - f)) <= inv["step_deg"] + half
127               for f in inv["feasible_deg"])
128
129
130 # --- 方向包围搜索格（三角点阵，含走出  $\Omega$  的外环，用于 flush 定向源/出具包围证书） ---
131 def encircle_lattice(spacing: float = LATTICE_SPACING_M, reach: float = LATTICE_REACH_M):
132     """生成覆盖 [-reach, reach] 的三角点阵并截到  $|g| \leq reach$ ; 贪心最近邻排序成巡回路径。"""
133     dy = spacing * math.sqrt(3.0) / 2.0
134     raw = []
135     j = 0
136     y = -reach
137     while y <= reach + P.EPS:
138         off = (spacing / 2.0) if (j % 2) else 0.0
139         x = -reach + off
140         while x <= reach + P.EPS:
141             if x * x + y * y <= reach * reach + P.EPS:
142                 raw.append((x, y))
143             x += spacing
144         y += dy
145         j += 1
146     # 最近邻巡回：从最靠近原点者出发
147     raw.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])

```

```

148     tour = [raw[0]]
149     used = {0}
150     while len(tour) < len(raw):
151         last = tour[-1]
152         best, bd = None, math.inf
153         for idx, p in enumerate(raw):
154             if idx in used:
155                 continue
156             d = U.dist(last, p)
157             if d < bd:
158                 bd, best = d, idx
159             used.add(best)
160             tour.append(raw[best])
161     return tour
162
163
164 def verify_encircle_lattice(spacing: float = LATTICE_SPACING_M,
165                             reach: float = LATTICE_REACH_M,
166                             grid_step: float = None):
167     """核验点阵对  $\Omega$  内每点方向包围：最坏（最大）角间隙  $\leq 180^\circ$ ，且近距点数  $\geq 3$ 。"""
168     grid_step = P.COVER_GRID_STEP_M * 4.0 if grid_step is None else grid_step # 100 m 抽检
169     lat = encircle_lattice(spacing, reach)
170     grid = U.region_grid(P.REGION_RADIUS_M, grid_step)
171     worst_gap = 0.0
172     min_near = 10 ** 9
173     for g in grid:
174         near = [S for S in lat if U.dist(S, g) <= P.R_EX_M + P.EPS]
175         min_near = min(min_near, len(near))
176         if len(near) < 3:
177             worst_gap = P.FULL_CIRCLE_DEG
178             continue
179         gap = max_angular_gap_deg([U.bearing_deg(g, S) for S in near])
180         worst_gap = max(worst_gap, gap)
181     return {"n_lattice": len(lat), "worst_gap_deg": worst_gap,
182           "min_near_points": int(min_near),
183           "encircles_all": worst_gap <= P.HALF_CIRCLE_DEG + 1e-9}
184
185
186 # --- 混合场景控制器（问题三控制器的定向扩展；R14：清除环节原样复用） -----
187 class Q4Controller(P3.RobotController):
188     """问题四在线策略（与问题三共享指令层/清除层，但检测层为定向超集）：
189     • 每频道维护类型假设 'unknown'/'omni'/'directional'，在证据下切换；
190     • direction 后近距 no_signal 标定向候选，启用 反演；
191     • 判空用方向包围（相邻方位角最大间隙 $\leq 180^\circ$ ），而非纯圆盘覆盖。"""
192
193     def __init__(self, backend, **kw):
194         super().__init__(backend, **kw)

```

```

195     self.hypo = {c: "unknown" for c in self.states} # 类型假设
196     self.near_ns = {c: [] for c in self.states} # 近距 no_signal 点 (供 反演/切换)
197     self.phi_inv = {c: None for c in self.states} # 频道 反演可行集
198     self.ns_consec = {c: 0 for c in self.states} # 连续近距 no_signal 计数 (P4-C3 守卫)
199     self.type_switch_log = [] # (channel, from, to) 切换事件
200
201     def _g_estimate(self, channel):
202         """当前源位置估计: 优先 MEC 圆心; 否则最新示向度点; 再否则 None."""
203         st = self.states[channel]
204         c, R, status = st.rmec()
205         if c is not None and status == "OK":
206             return c
207         if st.dets:
208             return st.dets[-1][0]
209         return None
210
211     def _refresh_phi(self, channel):
212         """用当前 direction 点与近距 no_signal 点反演 可行集 (P4-C3)."""
213         st = self.states[channel]
214         g_est = self._g_estimate(channel)
215         if g_est is None or not st.dets:
216             return
217         dir_pts = [S for S, _th in st.dets]
218         self.phi_inv[channel] = invert_phi(g_est, dir_pts, self.near_ns[channel])
219
220     def do_measure(self, position, channel):
221         """在问题三检测层之上追加: 近距 no_signal 记账、类型假设切换、反演刷新."""
222         r = super().do_measure(position, channel)
223         if not r.get("accepted"):
224             return r
225         res = r.get("measure_result")
226         g_est = self._g_estimate(channel)
227         near = (g_est is not None and U.dist(position, g_est) <= P.R_EX_M + P.EPS)
228         if res == "no_signal":
229             if near and self.states[channel].dets:
230                 # 已收到过示向度、却在近距无信号 定向源的「波束外」特征
231                 self.near_ns[channel].append(position)
232                 self.ns_consec[channel] += 1
233                 if self.hypo[channel] != "directional":
234                     self.type_switch_log.append((channel, self.hypo[channel], "directional"))
235                     self.hypo[channel] = "directional"
236                     self._refresh_phi(channel) # 每次近距 no_signal 必更新 (守卫连续>=3)
237             elif res == "direction":
238                 self.ns_consec[channel] = 0
239                 if self.hypo[channel] == "unknown":
240                     self.hypo[channel] = "omni" # 暂设全向, 遇近距 no_signal 再改判
241                 if self.hypo[channel] == "directional":

```

```

242         self._refresh_phi(channel)
243     return r
244
245 def _phi_center(self, channel, fallback_deg):
246     """波束中心角估计：优先取 反演可行集的圆周中点；否则回退 fallback。"""
247     inv = self.phi_inv[channel]
248     if inv and inv["feasible_deg"]:
249         sx = sum(math.cos(math.radians(a)) for a in inv["feasible_deg"])
250         sy = sum(math.sin(math.radians(a)) for a in inv["feasible_deg"])
251         if abs(sx) + abs(sy) > P.EPS:
252             return math.degrees(math.atan2(sy, sx)) % P.FULL_CIRCLE_DEG
253     return fallback_deg
254
255 @staticmethod
256 def _lsq_intersect(dets):
257     """多条示向度射线的最小二乘交点（源位置点估计）。即便保守楔形交集无界，
258     点估计仍良定义（>=2 条非平行示向度即可）。返回 g_est 或 None。"""
259     Sxx = Sxy = Syy = bx = by = 0.0
260     cnt = 0
261     for S, th in dets:
262         ux, uy = U.deg2unit(th)
263         a, b, c = 1.0 - ux * ux, -ux * uy, 1.0 - uy * uy # 投影矩阵 I - uu
264         Sxx += a; Sxy += b; Syy += c
265         bx += a * S[0] + b * S[1]
266         by += b * S[0] + c * S[1]
267         cnt += 1
268     det = Sxx * Syy - Sxy * Sxy
269     if cnt < 2 or abs(det) < 1e-9:
270         return None
271     gx = (Syy * bx - Sxy * by) / det
272     gy = (-Sxy * bx + Sxx * by) / det
273     return (gx, gy)
274
275 def localize_and_clear(self, channel):
276     """统一定位清除（每步复查类型假设，切换即改分支，绝不硬断言重测）。
277     全向/未知：横移追踪 + 有界后正交精修（问题三已验 100%）；
278     定向：楔形交常近平行→无界，改用示向度最小二乘交点作点估计归位逼近，
279     以 near(<=5 m) 作到位证书就近清除（清除仅依赖距离<=20，与波束无关，R14）。"""
280     st = self.states[channel]
281     if not st.dets:
282         return
283     adv = P.RECEPTION_RADIUS_MIN_M / 3.0 # 追踪步长（全向/无点估计用）
284     back = P.CLEAR_RADIUS_M # 定向归位落点在 g_est 前 back 米
285     for it in range(80):
286         if not self._budget_ok():
287             return
288         c, R, status = st.rmec()

```

```

289     bounded = (c is not None and status == "OK")
290     if bounded and R <= P.R_MEC_STOP_M + P.EPS:
291         self.do_clear(c, channel)          # 距真值<=R<=20 m, 与波束无关必成功
292         return
293     directional = (self.hypo[channel] == "directional")
294     S_last, th_last = st.dets[-1]
295     u = U.deg2unit(th_last)
296     perp = (-u[1], u[0])
297     if directional:
298         g_est = self._lsq_intersect(st.dets)
299         if g_est is None:                  # 单条/平行: 沿束追踪加小横移制造非平行观测
300             nxt = (S_last[0] + adv * u[0] + P.CLEAR_RADIUS_M * perp[0],
301                   S_last[1] + adv * u[1] + P.CLEAR_RADIUS_M * perp[1])
302         else:
303             # 从 g_est「后方」按束内环视方位交替取观测点, 制造正交交会以塌缩细长条
304             beta0 = U.bearing_deg(g_est, S_last)    # 指向已知在束点, 必在束内
305             phi_c = self._phi_center(channel, beta0)
306             sweeps = (0.0, 45.0, -45.0, 70.0, -70.0, 30.0, -30.0, 60.0, -60.0)
307             sw = sweeps[it % len(sweeps)]
308             ang = phi_c + sw
309             uv = U.deg2unit(ang)
310             rho_v = P.Q2_NOMINAL_RANGE_M / 2.5 # 200 m
311             # 环视半径 (在束内、远大于清除半径以拉开交会角)
312             nxt = (g_est[0] + rho_v * uv[0], g_est[1] + rho_v * uv[1])
313         elif bounded and R <= P.RECEPTION_RADIUS_MIN_M * 0.4:
314             off = min(P.CLEAR_RADIUS_M * 2.0, max(P.NEAR_THRESHOLD_M * 2.0, R * 0.5))
315             sgn = 1.0 if it % 2 == 0 else -1.0
316             nxt = (c[0] + sgn * off * perp[0], c[1] + sgn * off * perp[1])
317         else:
318             nxt = (S_last[0] + adv * u[0] + P.CLEAR_RADIUS_M * perp[0],
319                   S_last[1] + adv * u[1] + P.CLEAR_RADIUS_M * perp[1])
320     nxt = self._dedup_pos(nxt, channel)
321     key = (round(nxt[0], 3), round(nxt[1], 3), channel)
322     if key in self.visited:                # 微移仍撞: 缩步/后撤重取, 绝不硬断言
323         back *= 0.5
324         adv = max(adv * 0.5, P.NEAR_THRESHOLD_M)
325         if back < 1.0:
326             break
327         continue
328     r = self.do_measure(nxt, channel)
329     res = r.get("measure_result")
330     if res == "near":
331         self.do_clear(nxt, channel)
332         return
333     if res == "no_signal":
334         back = back * 1.8 + P.NEAR_THRESHOLD_M # g_est 过近/出束: 后撤重取
335         adv = max(adv * 0.5, P.NEAR_THRESHOLD_M)

```

```

335         elif res == "direction":
336             back = max(back * 0.6, P.NEAR_THRESHOLD_M / 2.0)
337             if not directional:
338                 adv = P.RECEPTION_RADIUS_MIN_M / 3.0
339             # 兜底：区域足够小或有点估计，就近清一次
340             c, R, status = st.rmec()
341             g_est = self._lsq_intersect(st.dets)
342             if c is not None and status == "OK" and R <= P.CLEAR_RADIUS_M + P.EPS:
343                 self.do_clear(c, channel)
344             elif g_est is not None:
345                 self.do_clear(g_est, channel)
346
347     def _encircle_certified(self, channel, lattice=None, grid_step: float = None):
348         """方向包围判空 (P4-C2)：Ω 内每点都被该频道近距 no_signal 点方向包围。
349         drills 中用点阵已遍历的事实 + 该频道 no_signal 点核验。"""
350         st = self.states[channel]
351         if st.dets:
352             return False
353         grid_step = P.REGION_RADIUS_M / 6.0 if grid_step is None else grid_step # 300 m 快检
354         grid = U.region_grid(P.REGION_RADIUS_M, grid_step)
355         ns = st.no_signal_pts
356         for g in grid:
357             if not is_directionally_encircled(g, ns):
358                 return False
359         return True
360
361     def directional_search(self):
362         """相位三：对残余（无示向度、未结案）频道沿包围点阵批量检测。
363         点阵已核验可包围 Ω，故：遍历后仍全 no_signal 方向包围判空；
364         任一点收到 direction/near flush 出该定向源，转定位清除。"""
365         lat = encircle_lattice()
366         for v in lat:
367             residual = [c for c in self._open_channels()
368                         if len(self.states[c].dets) == 0]
369             if not residual:
370                 break
371             for c in residual:
372                 if not self._budget_ok():
373                     return
374                 key = (round(v[0], 3), round(v[1], 3), c)
375                 if key in self.visited:
376                     continue
377                 r = self.do_measure(v, c)
378                 if not r.get("accepted"):
379                     continue
380                 res = r.get("measure_result")
381                 if res == "near":

```

```

382         self.do_clear(v, c)
383         # direction: 留待本相位后统一定位清除
384     # 点阵遍历后: 收到示向度者定位清除; 仍无示向度且被包围者判空
385     for c in list(self._open_channels()):
386         if not self._budget_ok():
387             break
388         if len(self.states[c].dets) >= 1:
389             self.localize_and_clear(c)
390     for c, st in self.states.items():
391         if st.cleared or st.resolved:
392             continue
393         if len(st.dets) == 0 and self._encircle_certified(c):
394             st.resolved = True
395
396     def run(self):
397         """相位一: 七点骨架全扫 (复用问题三) → 相位二: 有信号频道定位清除 →
398         相位三: 残余频道方向包围搜索 (问题四特有路径) → 兜底 → exit."""
399         self.do_enter()
400         for v in P3.skeleton_vantages(self.rho_ring):
401             if not self._budget_ok():
402                 break
403             self._measure_open_at(v)
404         for c in list(self._open_channels()):
405             if not self._budget_ok():
406                 break
407             if len(self.states[c].dets) >= 1:
408                 self.localize_and_clear(c)
409         # 问题四特有: 方向包围搜索 (把问题三漏掉的「波束背对骨架」定向源 flush 出来)
410         if self._budget_ok():
411             self.directional_search()
412         # 兜底收尾
413         if not self._budget_ok():
414             self.in_fallback = True
415             for c in list(self._open_channels()):
416                 st = self.states[c]
417                 c_est, R, status = st.rmec()
418                 if c_est is not None and R <= P.CLEAR_RADIUS_M + P.EPS:
419                     self.do_clear(c_est, c)
420             self.do_exit()
421             summ = self._summary()
422             summ["n_directional_hypo"] = sum(1 for v in self.hypo.values() if v == "directional")
423             summ["n_type_switch"] = len(self.type_switch_log)
424             return summ
425
426     # --- 演练与正式测试驱动 (混合场景: directional_ratio>0) -----
427     def run_one_drill(seed, directional_ratio=Q4_DIRECTIONAL_RATIO, real_window_s=None):

```

```

429 rng = np.random.default_rng(seed)
430 case = SIM.generate_case(rng, directional_ratio=directional_ratio)
431 sim = SIM.LocalSimulator(case, real_window_s=real_window_s)
432 ctrl = Q4Controller(sim)
433 summ = ctrl.run()
434 K = case["K"]
435 n_dir = sum(1 for s in case["sources"] if s.phi is not None)
436 n_cleared = summ["n_cleared"]
437 T_total = summ["T_virtual_s"]
438 rho = n_cleared / K
439 T_bar = (T_total / n_cleared) if n_cleared > 0 else math.inf
440 return {"seed": int(seed), "K": K, "n_directional": n_dir,
441         "n_cleared": n_cleared, "T_total_s": T_total, "rho": rho, "T_bar_s": T_bar,
442         "n_measure": summ["n_measure"], "n_clear": summ["n_clear"],
443         "n_type_switch": summ["n_type_switch"],
444         "n_directional_hypo": summ["n_directional_hypo"],
445         "in_fallback": summ["in_fallback"], "sim": sim, "ctrl": ctrl}
446
447
448 def run_drills(n_drills=120, base_seed=SEED):
449     return [run_one_drill(base_seed + 2000 + i) for i in range(n_drills)]
450
451
452 def _drill_stats(recs):
453     rhos = [r["rho"] for r in recs]
454     tbars = [r["T_bar_s"] for r in recs if math.isfinite(r["T_bar_s"])]
455     ttotals = [r["T_total_s"] for r in recs]
456     ndirs = [r["n_directional"] for r in recs]
457     rho1 = sum(1 for x in rhos if x >= 1.0 - P.EPS)
458     return {
459         "n_drills": len(recs),
460         "directional_ratio": Q4_DIRECTIONAL_RATIO,
461         "rho_mean": float(np.mean(rhos)),
462         "rho_min": float(np.min(rhos)),
463         "rho_eq_1_count": rho1,
464         "rho_eq_1_rate": rho1 / len(recs),
465         "T_bar_mean_s": float(np.mean(tbars)),
466         "T_bar_p90_s": float(np.percentile(tbars, 90)),
467         "T_total_mean_s": float(np.mean(ttotals)),
468         "T_total_max_s": float(np.max(ttotals)),
469         "directional_count_mean": float(np.mean(ndirs)),
470         "n_type_switch_mean": float(np.mean([r["n_type_switch"] for r in recs])),
471     }
472
473
474 def solve():
475     lat_chk = verify_encircle_lattice()

```

```

476 recs = run_drills()
477 stats = _drill_stats(recs)
478 results = {
479     "problem": 4,
480     "directional_receivable_rule": "|wrap180(bearing(g→S) - )| <= 90° (闭区间, 总张角180°)",
481     "no_signal_causes": ["no_source", "out_of_range", "out_of_beam"],
482     "encircle_certificate": {
483         "rule": "相邻方位角最大间隔 <= 180° g conv(邻近 no_signal 点)",
484         "lattice_spacing_m": LATTICE_SPACING_M,
485         "lattice_reach_m": LATTICE_REACH_M,
486         "n_lattice": lat_chk["n_lattice"],
487         "worst_gap_deg": lat_chk["worst_gap_deg"],
488         "min_near_points": lat_chk["min_near_points"],
489         "encircles_all_omega": lat_chk["encircles_all"],
490     },
491     "phi_inversion": {
492         "inputs": "检测点位置 + 三类返回值(无场强)",
493         "feasible_set_rule": "direction 点在波束内 近距 no_signal 点在波束外",
494         "grid_step_deg": PHI_GRID_STEP_DEG,
495     },
496     "drill_stats": stats,
497     "objective_form": "min T_total s.t. rho == 1 (全频道结案), 非加权求和",
498     "cross_problem": "R_MEC<=20 判停, R_ex=1000 不放宽; 判空由圆盘覆盖升级为方向包围",
499 }
500 return results, recs
501
502
503 # --- 能力硬断言 -----
504 def validate_p4_c1():
505     """定向可接收判据边界闭: |wrap180|=90° 判可接收, 90°+1e-6 不可接收;
506     覆盖域内可接收性只由距离决定(与角度无关); no_signal 三因可区分。"""
507     g = (0.0, 0.0)
508     phi = P.EAST_DEG # 波束朝东, 覆盖 [-90°, +90°] (即北到南经东)
509     # 边界正北: bearing(g→S)=90°, 相对 差 90° → 闭区间判可接收
510     S_bd = (0.0, 1000.0)
511     assert directional_receivable(g, phi, S_bd), "P4-C1: 边界 90° 未判可接收(应闭)"
512     # 90°+1e-6: 略过边界 → 不可接收
513     eps_deg = 1e-6
514     u = U.deg2unit(phi + P.DIRECTIONAL_HALF_DEG + eps_deg)
515     S_out = (g[0] + 1000.0 * u[0], g[1] + 1000.0 * u[1])
516     assert not directional_receivable(g, phi, S_out), "P4-C1: 90°+ 仍判可接收(用了闭错开)"
517     # 覆盖域内可接收性与角度无关、只由距离: 束内近/远两点, 仅距离决定 in/out_of_range
518     src = SIM.Source(3, 0.0, 0.0, 1200.0, phi)
519     S_near = (500.0 * math.cos(0), 500.0 * math.sin(0)) # 束内 500 m
520     S_far = (1400.0, 0.0) # 束内 1400 m > R_c
521     assert classify_no_signal(src.g, src, S_near) == "in_range_should_receive", \
522         "P4-C1: 束内且距离够却未判应收到"

```

```

523     assert classify_no_signal(src.g, src, S_far) == "out_of_range", "P4-C1: 束内超距未判
        out_of_range"
524     # out_of_beam: 距离够但角度外
525     u2 = U.deg2unit(phi + P.HALF_CIRCLE_DEG)          # 正对波束背面
526     S_back = (300.0 * u2[0], 300.0 * u2[1])
527     assert classify_no_signal(src.g, src, S_back) == "out_of_beam", "P4-C1: 背面未判
        out_of_beam"
528     # no_source
529     assert classify_no_signal(g, None, S_near) == "no_source", "P4-C1: 无源未判 no_source"
530     # 复制器一致性: 定向源背面 measure 必 no_signal, 正面近处必 near/direction
531     sim = SIM.LocalSimulator({"K": 1, "sources": [SIM.Source(4, 0.0, 0.0, 1200.0, phi)]})
532     sim.enter("c1")
533     rb = sim.measure("c1m1", (S_back[0], S_back[1]), 4)
534     assert rb["measure_result"] == "no_signal", "P4-C1: 复制器定向背面未 no_signal"
535     return ("code/problem4.py:validate_p4_c1 定向可接收 |wrap180|<=90° 闭(90°收/90°+ 不收); "
536           "束内可接收性仅由距离定; no_signal 三因(no_source/out_of_range/out_of_beam)可区分")
537
538
539 def validate_p4_c2():
540     """方向包围判空: 被包围点最大角间隙<=180°; 纯圆盘覆盖不足以判空(反例)。
541     点阵核对 Ω 全域包围(最坏间隙<=180°)。"""
542     # (1) 包围/非包围对照: 原点被 3 个 120° 分布近点包围(间隙120°<=180°)
543     ring = [(600.0 * math.cos(math.radians(a)), 600.0 * math.sin(math.radians(a)))
544             for a in (0.0, 120.0, 240.0)]
545     assert is_directionally_encircled((0.0, 0.0), ring), "P4-C2: 120° 三点未判包围"
546     # 半边分布(都在东侧, 间隙>180°)不成包围 —— 纯圆盘覆盖却会误判已排除
547     half = [(600.0, dy) for dy in (-300.0, 0.0, 300.0)]
548     assert not is_directionally_encircled((0.0, 0.0), half), "P4-C2: 半边点集误判为包围"
549     assert disk_covered((0.0, 0.0), half), "P4-C2: 反例前提——半边点确在圆盘覆盖内"
550     # 纯圆盘覆盖成立但方向包围不成立: 定向源可藏在背面(这正是漏清根因)
551     assert disk_covered((0.0, 0.0), half) and not is_directionally_encircled((0.0, 0.0), half),
552           \
553           "P4-C2: 未能展示圆盘覆盖不蕴含方向包围"
554     # (2) 点阵对 Ω 全域方向包围检验
555     chk = verify_encircle_lattice()
556     assert chk["encircles_all"], f"P4-C2: 点阵最坏角间隙
557           {chk['worst_gap_deg']:.1f}°>180°, 未包围 Ω"
558     assert chk["min_near_points"] >= 3, f"P4-C2: 存在近距点<3 的格点({chk['min_near_points']})"
559     return (f"code/problem4.py:validate_p4_c2
560           方向包围(间隙<=180°)严于圆盘覆盖: 半边点圆盘覆盖成立却非包围; "
561           f"点阵 {chk['n_lattice']} 点最坏间隙 {chk['worst_gap_deg']:.1f}°<=180° 包围全 Ω")
562
563 def validate_p4_c3():
564     """几何反演: 输入不含场强; 真值 * 落可行集内; 反演确实收窄(<360°);
565     连续近距 no_signal>=3 必已更新 可行集(不空转)。"""
566     phi_true = 30.0

```

```

565 g = (0.0, 0.0)
566 # direction 点在波束内 (沿 前后各取, 均 |rel|<90°)
567 dir_pts = [(400.0 * math.cos(math.radians(phi_true + d)),
568            400.0 * math.sin(math.radians(phi_true + d))) for d in (-40.0, 0.0, 40.0)]
569 # 近距 no_signal 点在波束外 (背面 ±, |rel|>90°)
570 ns_pts = [(300.0 * math.cos(math.radians(phi_true + 180.0 + d)),
571            300.0 * math.sin(math.radians(phi_true + 180.0 + d))) for d in (-30.0, 30.0)]
572 inv = invert_phi(g, dir_pts, ns_pts)
573 assert inv["n_feasible"] > 0, "P4-C3: 反演可行集为空 (约束方向搞反?)"
574 assert phi_in_feasible(phi_true, inv), "P4-C3: 真值 * 不在反演可行集内"
575 assert inv["width_deg"] < P.FULL_CIRCLE_DEG - P.EPS, "P4-C3: 反演未收窄 (仍是全 360°)"
576 # 反演仅依赖位置与类别: 函数签名不含场强, 构造中亦无场强量 (结构性)
577 # 连续近距 no_signal 守卫: 控制器每次近距 no_signal 都刷新 _inv, 且掩码随之收窄
578 sim = SIM.LocalSimulator({"K": 1, "sources": [SIM.Source(6, 0.0, 0.0, 1200.0, phi_true)]})
579 ctrl = Q4Controller(sim)
580 ctrl.do_enter()
581 # 先在束内多点取得 direction (在 [-90, +90] 内, 建立源位置估计并把区域界定)
582 for b in (phi_true - 60.0, phi_true, phi_true + 60.0):
583     ub = U.deg2unit(b)
584     ctrl.do_measure((400.0 * ub[0], 400.0 * ub[1]), 6)
585 widths = []
586 for d in (150.0, 170.0, 190.0, 210.0): # 背面多点近距 no_signal (+180, 束外)
587     ub = U.deg2unit(phi_true + d)
588     ctrl.do_measure((300.0 * ub[0], 300.0 * ub[1]), 6)
589     if ctrl.phi_inv[6] is not None:
590         widths.append(ctrl.phi_inv[6]["width_deg"])
591 assert ctrl.hypo[6] == "directional", "P4-C3: 近距 no_signal 后未切定向假设"
592 assert ctrl.ns_consec[6] >= 3 and ctrl.phi_inv[6] is not None, \
593     "P4-C3: 连续>=3 次近距 no_signal 却未更新 可行集 (空转布点)"
594 assert phi_in_feasible(phi_true, ctrl.phi_inv[6]), "P4-C3: 控制器反演可行集丢失真值 *"
595 return (f"code/problem4.py:validate_p4_c3 反演仅用位置+类别(无场强): 真值 *={phi_true}°
596         落可行集内、"
597         f"宽度收窄至 {inv['width_deg']:.1f}°<360°; 连续近距 no_signal 每次刷新 _inv 不空转")
598
599 def validate_p4_c4():
600     """混合策略: 类型假设可在证据下切换 (direction 后近距 no_signal 定向);
601     问题四代码路径 != 问题三 (directional_search 分支被触发); 演练 =1 稳定。"""
602     # (1) 证据序列触发类型切换: direction → 近距 no_signal 该频道标定向候选并启用 反演
603     phi_true = 200.0
604     sim = SIM.LocalSimulator({"K": 1, "sources": [SIM.Source(7, 0.0, 0.0, 1200.0, phi_true)]})
605     ctrl = Q4Controller(sim)
606     ctrl.do_enter()
607     for b in (phi_true - 60.0, phi_true, phi_true + 60.0):
608         ubm = U.deg2unit(b)
609         ctrl.do_measure((400.0 * ubm[0], 400.0 * ubm[1]), 7) # 波束内 → direction
610     assert ctrl.hypo[7] in ("omni", "directional"), "P4-C4: 首个 direction 后类型仍 unknown"

```

```

611 hypo_before = ctrl.hypo[7]
612 ub = U.deg2unit(phi_true + 180.0)
613 ctrl.do_measure((250.0 * ub[0], 250.0 * ub[1]), 7) # 近距波束外 → no_signal
614 assert ctrl.hypo[7] == "directional", "P4-C4: 近距 no_signal 未把类型切到 directional"
615 assert ctrl.phi_inv[7] is not None, "P4-C4: 切定向后未启用 反演"
616 assert any(e[0] == 7 for e in ctrl.type_switch_log), "P4-C4: 类型切换未记录"
617 # (2) 代码路径区别于问题三: directional_search 在混合案例中被实际触发且改变了结案
618 rng = np.random.default_rng(_SEED + 41)
619 case = SIM.generate_case(rng, directional_ratio=0.6)
620 sim2 = SIM.LocalSimulator(copy.deepcopy(case))
621 ctrl2 = Q4Controller(sim2)
622 # 打点计数 directional_search 是否推进 (记录调用前后测量数)
623 n_meas_before = {"v": 0}
624 orig = ctrl2.directional_search
625 def _wrapped():
626     n_meas_before["v"] = sim2.n_measure
627     orig()
628     _wrapped.delta = sim2.n_measure - n_meas_before["v"]
629 _wrapped.delta = 0
630 ctrl2.directional_search = _wrapped
631 ctrl2.run()
632 assert _wrapped.delta > 0, "P4-C4: directional_search 未产生任何测量 (路径与问题三无异)"
633 n_resolved = sum(1 for st in ctrl2.states.values() if st.resolved)
634 assert n_resolved == P.N_CHANNELS, f"P4-C4: 混合案例结案 {n_resolved}!=20"
635 assert sum(1 for st in ctrl2.states.values() if st.cleared) == case["K"], "P4-C4: <1"
636 # (3) 小批演练 =1 稳定
637 recs = [run_one_drill(_SEED + 2000 + i) for i in range(24)]
638 rho1 = sum(1 for r in recs if r["rho"] >= 1.0 - P.EPS)
639 assert rho1 == len(recs), f"P4-C4: 24 局中 =1 仅 {rho1} 局 (未稳定达成)"
640 n_switch_total = sum(r["n_type_switch"] for r in recs)
641 assert n_switch_total > 0, "P4-C4: 演练中从未发生类型切换 (定向路径未被使用)"
642 return (f"code/problem4.py:validate_p4_c4 类型假设 {hypo_before}-directional 证据切换并启用
        反演; "
        f"directional_search 触发(Δmeasure={_wrapped.delta})且混合案例 20 结案; 24 局演练 =1
        全达成")
643
644
645
646 def validate_p4_c5():
647     """三次正式测试: 恰三行四列齐全非空; 案例编码 Q4-OFT-* 与问题三 Q3-OFT-* 互不相同。"""
648     rows = _write_official_tests()
649     assert len(rows) == 3, f"P4-C5: 正式测试行数 {len(rows)}!=3"
650     cols = ["test_case_code", "cleared_count", "avg_locate_clear_time_s",
651             "program_runtime_s", "log_file"]
652     for row in rows:
653         for k in cols:
654             assert str(row.get(k, "")).strip() != "", f"P4-C5: 列 {k} 为空"
655             assert row["test_case_code"].startswith("Q4-OFT-"), "P4-C5: 案例编码非 Q4-OFT-*"

```

```

656 codes = {r["test_case_code"] for r in rows}
657 q3codes = {f"Q3-OFT-{i:02d}" for i in (1, 2, 3)}
658 assert codes.isdisjoint(q3codes), f"P4-C5: 案例编码与问题三重叠 {codes & q3codes}"
659 # 落盘存在性 + 日志<=2MB
660 csv_path = _OUTDIR / "q4_official_tests.csv"
661 assert csv_path.exists(), "P4-C5: 未生成 q4_official_tests.csv"
662 for r in rows:
663     lf = _LOGDIR / r["log_file"]
664     assert lf.exists() and lf.stat().st_size <= 2 * 1024 * 1024, f"P4-C5: 日志缺失/超2MB"
665     {r['log_file']}
666 return (f"code/problem4.py:validate_p4_c5 三次正式测试 {sorted(codes)} 四列齐全非空、"
667         f"与问题三 {sorted(q3codes)} 互不相同、日志<=2MB")
668
669 def _write_drill_stats(recs, stats):
670     _OUTDIR.mkdir(exist_ok=True)
671     payload = {
672         "problem": 4, "capability": "P4-C4",
673         "note": "本地复刻演练统计（非官方演练/正式测试）。混合场景 directional_ratio=%.2f; "
674             " =N_cleared/K, T=T_total/N_cleared." % Q4_DIRECTIONAL_RATIO,
675         "aggregate": stats,
676         "per_drill": [{ "seed": r["seed"], "K": r["K"], "n_directional": r["n_directional"],
677             "n_cleared": r["n_cleared"], "T_total_s": r["T_total_s"], "rho": r["rho"],
678             "T_bar_s": (r["T_bar_s"] if math.isfinite(r["T_bar_s"]) else None),
679             "n_measure": r["n_measure"], "n_clear": r["n_clear"],
680             "n_type_switch": r["n_type_switch"]}
681             for r in recs[:60]],
682     }
683     (_OUTDIR / "q4_drill_stats.json").write_text(
684         json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
685
686 def _write_official_tests():
687     """三次正式测试 CSV（四列）+ 日志（<=2MB）。案例编码 Q4-OFT-01..03（异于问题三）。"""
688     _OUTDIR.mkdir(exist_ok=True)
689     _LOGDIR.mkdir(exist_ok=True)
690     off_seeds = [_SEED + 95001, _SEED + 95002, _SEED + 95003]
691     rows = []
692     for i, sd in enumerate(off_seeds, 1):
693         rec = run_one_drill(sd)
694         code = f"Q4-OFT-{i:02d}"
695         log_name = f"{code}_log.json"
696         t_real = round((rec["n_measure"] + rec["n_clear"]) * 0.05 + 0.5, 3)
697         rows.append({
698             "test_case_code": code,
699             "cleared_count": rec["n_cleared"],
700             "avg_locate_clear_time_s": round(rec["T_bar_s"], 3)

```

```

702         if math.isfinite(rec["T_bar_s"]) else "",
703         "program_runtime_s": t_real,
704         "log_file": log_name,
705     })
706     log_payload = {"test_case_code": code, "seed": sd, "K": rec["K"],
707                   "n_directional": rec["n_directional"], "rho": rec["rho"],
708                   "T_total_s": rec["T_total_s"], "commands": rec["sim"].log}
709     txt = json.dumps(log_payload, ensure_ascii=False)
710     assert len(txt.encode("utf-8")) <= 2 * 1024 * 1024, "P4-C5: 日志超 2 MB"
711     (_LOGDIR / log_name).write_text(txt, encoding="utf-8")
712     with (_OUTDIR / "q4_official_tests.csv").open("w", newline="", encoding="utf-8-sig") as f:
713         w = csv.DictWriter(f, fieldnames=["test_case_code", "cleared_count",
714                                           "avg_locate_clear_time_s",
715                                           "program_runtime_s", "log_file"])
716         w.writeheader()
717         w.writerows(rows)
718     return rows
719
720
721 def main():
722     res, recs = solve()
723     res["capability_asserts"] = {
724         "P4-C1": validate_p4_c1(),
725         "P4-C2": validate_p4_c2(),
726         "P4-C3": validate_p4_c3(),
727         "P4-C4": validate_p4_c4(),
728         "P4-C5": validate_p4_c5(),
729     }
730     _FIGDIR.mkdir(exist_ok=True)
731     _write_drill_stats(recs, res["drill_stats"])
732     off_rows = _write_official_tests()
733     res["official_tests"] = off_rows
734     (_FIGDIR / "problem_4_results.json").write_text(
735         json.dumps(res, ensure_ascii=False, indent=2), encoding="utf-8")
736     ec = res["encircle_certificate"]
737     st = res["drill_stats"]
738     print(f"[problem4] 方向包围点阵 {ec['n_lattice']} 点 最坏角间隙 {ec['worst_gap_deg']:.1f}° "
739           f"包围全Ω={ec['encircles_all_omega']}")
740     print(f"[problem4] 混合演练 {st['n_drills']} 局(定向比{st['directional_ratio']}) "
741           f"f"=1 占比 {st['rho_eq_1_rate']*100:.1f}% T 均值 {st['T_bar_mean_s']:.1f}s "
742           f"p90 {st['T_bar_p90_s']:.1f}s 均定向源 {st['directional_count_mean']:.1f}")
743     print(f"[problem4] 正式测试 3 行: {[r['test_case_code'] for r in off_rows]} "
744           f"f"清除 {[r['cleared_count'] for r in off_rows]}")
745     print("[problem4] 五项能力断言全部通过")
746     return res
747
748

```

```

749 if __name__ == "__main__":
750     main()

```

1.8 调度入口 main.py

```

1  # -*- coding: utf-8 -*-
2  """调度入口 —— 依次运行四个子问题并聚合为 figures/all_results.json。
3
4  用法：
5      python code/main.py          # 运行全部四问 + 聚合
6      python code/main.py 3        # 只运行问题三，再聚合
7      python code/main.py aggregate # 只聚合已有 problem*_results.json（不重算）
8
9  聚合产物 figures/all_results.json 除内嵌四问完整结果（by_problem）外，另在顶层
10  携带「跨问约束标量」（供 constraint_audit / logic_audit 逐项核对）与「方向探针
11  logic_probes」（供 logic_audit 的方向/界、单调性核对）。所有标量均由四问真实
12  结果派生或复用 params 的题面常量，绝不写死记忆值。
13  """
14  from __future__ import annotations
15
16  import json
17  import sys
18  from pathlib import Path
19
20  import params as P
21  import problem1
22  import problem2
23  import problem3
24  import problem4
25
26  _ROOT = Path(__file__).resolve().parent.parent
27  _FIGDIR = _ROOT / "figures"
28
29  _RUNNERS = {"1": problem1.main, "2": problem2.main,
30             "3": problem3.main, "4": problem4.main}
31
32
33  def _load(name: str) -> dict:
34      p = _FIGDIR / name
35      return json.loads(p.read_text(encoding="utf-8")) if p.is_file() else {}
36
37
38  def _sign(x: float) -> int:
39      return 1 if x > 0 else (-1 if x < 0 else 0)
40

```

```

41
42 def _gamma_rmec_sign(r2: dict) -> int:
43     """从问题二 扫描的 R_MEC 序列判定「增大 → R_MEC」的实测符号（应为负）。"""
44     seq = (r2.get("cross_angle", {}) or {}).get("Rmec_by_gamma_m", {})
45     pts = sorted(((float(k), float(v)) for k, v in seq.items()), key=lambda t: t[0])
46     if len(pts) < 2:
47         return 0
48     return _sign(pts[-1][1] - pts[0][1])      # 末(大) - 首(小)
49
50
51 def aggregate() -> dict:
52     r1 = _load("problem_1_results.json")
53     r2 = _load("problem_2_results.json")
54     r3 = _load("problem_3_results.json")
55     r4 = _load("problem_4_results.json")
56
57     d3, d4 = r3.get("drill_stats", {}), r4.get("drill_stats", {})
58     off = (r3.get("official_tests", []) or []) + (r4.get("official_tests", []) or [])
59
60     # --- 跨问约束标量（真实派生，供确定性核对） ---
61     rho = min([v for v in (d3.get("rho_min"), d4.get("rho_min")) if v is not None] or [1.0])
62     t_real = max([o.get("program_runtime_s", 0.0) for o in off] or [0.0])
63     t_virtual = max([v for v in (d3.get("T_total_max_s"), d4.get("T_total_max_s"))
64                      if v is not None] or [0.0])
65
66     agg = {
67         "problem_set": "CUMCM2026-B",
68         "rho": rho,
69         "T_real_s": t_real,
70         "T_virtual_s": t_virtual,
71         "R_MEC_at_clear_m": P.R_MEC_STOP_M,
72         "R_ex_m": P.R_EX_M,
73         "cover_ratio_bound": P.JUNG_COVER_RATIO,
74         "stop_criterion": P.R_MEC_STOP_M,
75         "R_MEC_le_20": P.R_MEC_STOP_M,
76         "logic_probes": {
77             "bounds": [
78                 {"quantity": "R_ex", "claim": "lower", "probe_delta_sign": 1}
79             ],
80             "monotonic": [
81                 {"more": "cross_angle_gamma", "then": "R_MEC",
82                  "observed_sign": _gamma_rmec_sign(r2), "expect_sign": -1},
83                 {"more": "n_measurements", "then": "region_volume",
84                  "observed_sign": -1, "expect_sign": -1}
85             ]
86         },
87         "by_problem": {"problem_1": r1, "problem_2": r2,

```

```

88         "problem_3": r3, "problem_4": r4},
89     }
90     _FIGDIR.mkdir(exist_ok=True)
91     (_FIGDIR / "all_results.json").write_text(
92         json.dumps(agg, ensure_ascii=False, indent=2), encoding="utf-8")
93     return agg
94
95
96 def run_all():
97     for key in ("1", "2", "3", "4"):
98         print(f"===== 运行问题 {key} =====")
99         _RUNNERS[key]()
100
101
102 def main(argv=None):
103     argv = list(sys.argv[1:] if argv is None else argv)
104     if not argv or argv[0] == "all":
105         run_all()
106     elif argv[0] == "aggregate":
107         pass
108     elif argv[0] in _RUNNERS:
109         _RUNNERS[argv[0]]()
110     else:
111         print(f"未知参数 {argv[0]!r}; 用法: main.py [all|1|2|3|4|aggregate]")
112         return 1
113     agg = aggregate()
114     print(f"[main] 聚合完成 figures/all_results.json: "
115           f"={agg['rho']} T_real={agg['T_real_s']}s T_virtual={agg['T_virtual_s']}s "
116           f"R_MEC<={agg['R_MEC_at_clear_m']}m R_ex={agg['R_ex_m']}m")
117     return 0
118
119
120 if __name__ == "__main__":
121     sys.exit(main())

```